

长广溪智能制造（无锡）有限公司

Changguangxi Intelligent Manufacturing (Wuxi) Co., Ltd.

协作机器人二次开发接口文档 （C#）

文档版本：v2.20e-full

发布日期：2025 年 9 月 16 日

文档编号：CGXi-QM-CR-004-F002A



长广溪智能制造（无锡）有限公司

Changguangxi Intelligent Manufacturing(Wuxi) Co., Ltd.



长广溪智能制造（无锡）有限公司（简称长广溪智造）保留对此文件修改之权利且不另行通知。长广溪智能制造（无锡）有限公司所提供之信息相信为正确且可靠之信息，但并不保证本文件中绝无错误。请于向长广溪智能制造（无锡）有限公司提出订单前，自行确定所使用之相关技术文件及规格为最新之版本。若因贵公司使用本公司之文件或产品，而涉及第三人之专利或著作权等智能财产权之应用及配合时，则应由贵公司负责取得同意及授权，本公司仅单纯贩售产品，上述关于同意及授权，非属本公司应为保证之责任。又未经长广溪智能制造（无锡）有限公司之正式书面许可，本公司之所有产品不得使用于医疗器材，维持生命系统及飞航等相关设备。

注意事项

本手册仅适用于长广溪智能制造（无锡）有限公司 G 系列协作机器人。

目录

一、SDK 介绍	1
二、数据结构	1
2.1 接口调用返回值	1
2.2 坐标系选择	2
2.3 点转换类型	2
2.4 位姿变换	2
2.5 运动触发模式	3
2.6 TCP 偏移量	3
2.7 运动参数	3
2.8 简化版运动参数	4
2.9 圆弧运动参数	5
2.10 运动模式	5
2.11 关节状态	5
2.12 机械臂状态	6
2.13 安全模式	6
2.14 程序状态	7
2.15 弹窗信息	7
2.16 版本信息	8
2.17 Lua 变量	8
2.18 变量元素	8
2.19 变量信息	9
2.20 消息来源	9
2.21 警告等级	9
2.22 世界时间	10
2.23 日志信息	10
2.24 版本信息	10
2.25 负载	11
2.26 校验码检查	11
2.27 串口通信	11
2.28 波特率	11
2.29 串口配置	12
2.30 IO 类型	12
2.31 安装角度	13
2.32 系统功能输入	13
2.33 系统功能输出	13
2.34 IO 选项卡输出	14
2.35 输出信号状态 (自动复位)	14
2.36 输入信号	14
2.37 输出信号	15

2.38 工具 IO 类型	15
2.39 模拟量类型	15
2.40 模拟量输入	15
2.41 模拟量输出	15
2.42 编码器配置	16
2.43 工具电压	16
2.44 BOOL 寄存器	16
2.45 寄存器	16
2.46 工具端信息	17
2.47 点坐标系	17
2.48 线坐标系	17
2.49 面坐标系	18
2.50 机器人限值等级	18
2.51 碰撞后处理模式	18
2.52 关节限值模式	18
2.53 安全限值模式	19
2.54 安全限值	19
2.55 关节限值	19
2.56 安全平面	20
2.57 安全 IO 输入信号	20
2.58 安全 IO 输出信号	20
2.59 控制柜网络配置	20
2.60 MODBUS 地址映射功能	21
2.61 MODBUS 地址映射	21
2.62 MODBUS 连接状态	21
2.63 MODBUS 参数配置	22
2.64 通讯变量选择	22
2.65 脚本程序状态	22
2.66 串口类型	22
2.67 轨迹点数据	23
2.68 轨迹参数	23
2.69 轨迹数据	23
2.70 下载轨迹数据	23
2.71 轨迹记录参数	24
2.72 轨迹记录状态	24
2.73 轨迹运行状态	24
2.74 控制模式	24
2.75 力控设置	24
2.76 力控传感器 tcp	25
2.77 力控传感器负载	25
2.78 力控传感器信息	25

2.79 通讯配置	25
2.80 力控传感器工具设置	26
2.81 力控配置	26
2.82 力控控制参数	26
2.83 力控数据	27
2.84 机器人状态数据	27
2.85 机械臂和扩展轴数据	27
2.86 扩展轴关节移动参数	28
2.87 扩展轴点动运动参数	28
2.88 扩展轴运动参数	29
2.89 扩展轴类型	29
2.90 扩展轴配置	29
2.91 ethercat 从站信息	30
2.92 Esc 状态	30
2.93 Ect 数据类型	30
2.94 pdo 类型	31
2.95 pdo 数据	31
2.96 CANOpen pdo 类型	31
2.97 CANOpen pdo 数据	31
三、接口函数	32
3.1 基础	32
3.1.1 基础接口	32
3.1.1.1 机器人创建连接	32
3.1.1.2 机器人断开连接	33
3.1.1.3 机器人关机	34
3.1.1.4 机械臂上电	34
3.1.1.5 机械臂断电	35
3.1.1.6 机械臂使能	36
3.1.1.7 机械臂关使能	37
3.1.1.8 机械臂故障复位	38
3.1.1.9 获取控制柜状态	39
3.1.1.10 写机械臂速度百分比	40
3.1.1.11 读机械臂速度百分比	41
3.1.1.12 获取 SDK 版本号	42
3.1.1.13 获取机械臂状态数据	42
3.1.2 程序控制	44
3.1.2.1 下载脚本程序	44
3.1.2.2 下载脚本程序 (加密)	45
3.1.2.3 上传脚本程序	46
3.1.2.4 运行程序	47
3.1.2.5 停止程序运行	48

3.1.2.6 暂停程序运行	49
3.1.2.7 读取脚本当前运行行号	50
3.1.2.8 读取脚本运行状态	51
3.1.2.9 上传工程文件	52
3.1.2.10 下载工程文件	53
3.1.2.11 是否存在弹窗	54
3.1.2.12 读取弹窗信息	55
3.1.2.13 设置输入弹窗信息	56
3.1.2.14 关闭弹窗	57
3.2 运动模块	57
3.2.1 运动接口	57
3.2.1.1 轴空间运动 moveJ	58
3.2.1.2 轴空间运动 movejoint	59
3.2.1.3 直线运动 moveL	60
3.2.1.4 直线运动 moveline	61
3.2.1.5 点动运动	63
3.2.1.6 圆弧运动	64
3.2.1.7 运动控制	67
3.2.1.8 设置阻塞移动中参数阈值	68
3.2.1.9 PointControlParaSimple 数据转换	69
3.2.1.10 设置软件自由驱动启动	70
3.2.1.11 读机械臂运动状态	71
3.2.1.12 读取移动缓存区中移动指令的数量	71
3.2.2 运动状态信息	72
3.2.2.1 读实际 TCP 位置	72
3.2.2.2 读目标 TCP 位置	73
3.2.2.3 读实际 TCP 速度	74
3.2.2.4 读目标 TCP 速度	75
3.2.2.5 读实际 TCP 加速度	76
3.2.2.6 读当前使用的 TCP 偏移信息	77
3.2.2.7 读取当前使用负载信息	78
3.2.3 运动配置	79
3.2.3.1 读所有 TCP 偏移信息列表	79
3.2.3.2 读取所有负载信息	80
3.2.3.3 获取指定 TCPoffset 的位姿	81
3.2.4 关节状态	82
3.2.4.1 读实际关节位置	82
3.2.4.2 读目标关节位置	83
3.2.4.3 读实际关节速度	84
3.2.4.4 读目标关节速度	85
3.2.4.5 读实际关节加速度	86

3.2.4.6 读目标关节加速度	87
3.2.4.7 读实际关节电机电流	88
3.2.4.8 读目标关节电机电流	89
3.2.4.9 读实际关节转矩	90
3.2.4.10 读实际关节采集的母线电压	91
3.2.4.11 读关节温度	92
3.2.4.12 读关节模式	93
3.2.5 轨迹复现	93
3.2.5.1 读取控制柜当前存在的轨迹索引	94
3.2.5.2 上传轨迹	95
3.2.5.3 下载轨迹	96
3.2.5.4 轨迹控制	105
3.2.5.5 轨迹数据转换为轨迹文件	107
3.2.5.6 轨迹文件转换为轨迹数据	108
3.2.5.7 设置轨迹记录参数	109
3.2.5.8 获取当前轨迹记录状态	110
3.2.5.9 获取当前轨迹运行状态	111
3.2.6 扩展轴运动	113
3.2.6.1 扩展轴关节移动	113
3.2.6.2 扩展轴直线移动	114
3.2.6.3 扩展轴点动运动	115
3.2.6.4 扩展轴运动	116
3.3 系统属性	117
3.3.1 硬件信息	117
3.3.1.1 读取控制柜硬件序列号	117
3.3.1.2 读取整臂序列号	118
3.3.1.3 获取版本信息	119
3.3.2 本体参数	120
3.3.2.1 读机械臂 DH 参数	120
3.3.2.2 读机械臂标准 DH 参数	121
3.3.2.3 读机械臂动力学参数	122
3.3.2.4 读机械臂零位补偿值	123
3.3.2.5 读机械臂减速比	124
3.3.3 系统信息	125
3.3.3.1 读机械臂当前状态	125
3.3.3.2 获取日志数量	126
3.3.3.3 获取机械臂实时日志	127
3.3.3.4 获取机械臂历史所有日志	128
3.3.3.5 日志功能是否开启	129
3.3.3.6 日志大小设置	130
3.3.3.7 读系统时间	130

3.3.3.8 读取机型型号	131
3.3.3.9 读取控制器型号	132
3.3.3.10 触发黑匣子数据	133
3.3.3.11 获取配置改变标志	134
3.3.3.12 触发系统更新	135
3.3.4 系统设置	136
3.3.4.1 开启运动仿真功能	136
3.3.4.2 开启干涉检测功能	137
3.3.4.3 开启动力学计算	138
3.3.4.4 开启碰撞检测功能	139
3.4 变量	140
3.4.1 数字量	140
3.4.1.1 读控制柜标准数字输出	140
3.4.1.2 写控制柜标准数字输出	141
3.4.1.3 读控制柜可配置数字输出	142
3.4.1.4 读控制柜所有可配置数字输出	143
3.4.1.5 写控制柜可配置数字输出	144
3.4.1.6 读工具端数字输出	145
3.4.1.7 写工具端数字输出	146
3.4.1.8 读控制柜标准数字输入	147
3.4.1.9 读控制柜可配置数字输入	148
3.4.1.10 读控制柜所有可配置数字输入	149
3.4.1.11 读工具端数字输入	149
3.4.2 模拟量	150
3.4.2.1 读控制柜模拟量输出	150
3.4.2.2 写控制柜模拟量输出	151
3.4.2.3 读工具端模拟量输入	152
3.4.2.4 读控制柜模拟量输入	153
3.4.2.5 读机械臂控制柜和工具端上所有输入数字量和模拟量	154
3.4.2.6 读机械臂控制柜和工具端上所有输出数字量和模拟量	156
3.4.3 通讯变量	157
3.4.3.1 读 bool 寄存器	158
3.4.3.2 写 bool 寄存器	159
3.4.3.3 读 int16 寄存器	160
3.4.3.4 写 int16 寄存器	161
3.4.3.5 读 int32 寄存器	162
3.4.3.6 写 int32 寄存器	163
3.4.3.7 读 float 寄存器	164
3.4.3.8 写 float 寄存器	165
3.4.4 脚本变量	166
3.4.4.1 读安装变量	166

3.4.4.2 修改安装变量	167
3.4.4.2 读取程序变量数据	168
3.4.4.3 修改程序变量数据	169
3.4.5 其他 IO	170
3.4.5.1 获取编码器计数数量	170
3.4.5.2 设置监控界面的工具输出电压	171
3.4.5.3 读取监控界面的工具输出电压	172
3.5 数学模块	173
3.5.1 位姿变换	173
3.5.1.1 位姿转换	173
3.5.1.2 位姿求逆	174
3.5.1.3 轴角转欧拉角	175
3.5.1.4 欧拉角转轴角	176
3.5.1.5 TCP 位姿转法兰位姿	177
3.5.1.6 基坐标系下的位姿转到用户坐标系下	177
3.5.1.7 目标点旋转变换	178
3.5.2 运动学	179
3.5.2.1 正解	179
3.5.2.2 逆解	180
3.6 配置	181
3.6.1 安装	181
3.6.1.1 设置安装角度	181
3.6.1.2 读取安装角度	182
3.6.2 TCP	183
3.6.2.1 读取 TCP 个数	183
3.6.2.2 读取 TCP 数据	184
3.6.2.3 读取 TCP 数据(通过名称)	185
3.6.2.4 增加 TCP	186
3.6.2.5 删除 TCP	187
3.6.2.6 编辑 TCP 数据	188
3.6.2.7 读取当前激活的 TCP 索引	189
3.6.2.8 激活 TCP	190
3.6.2.9 计算 TCP 位置	191
3.6.2.10 计算 TCP 角度	192
3.6.3 负载	193
3.6.3.1 读取负载个数	194
3.6.3.2 读取负载数据	194
3.6.3.3 读取负载数据(通过名称)	195
3.6.3.4 增加负载	196
3.6.3.5 删除负载	198
3.6.3.6 编辑负载	198

3.6.3.7 读取当前激活负载索引	200
3.6.3.8 激活负载	201
3.6.4 初始位/包装位	201
3.6.4.1 设置初始位	202
3.6.4.2 读取初始位	203
3.6.4.3 删除初始位	204
3.6.4.4 读取包装位	204
3.6.5 启动	205
3.6.5.1 设置机器人是否自动上电使能	205
3.6.5.2 读取机器人自动上电状态	206
3.6.6 控制参数	207
3.6.6.1 设置机器人拖动阻尼	207
3.6.6.2 读取机器人拖动阻尼	208
3.6.6.3 设置转矩异常门限倍率	209
3.6.6.4 读转矩异常门限倍率	210
3.6.6.5 设置扩展轴步长及速度倍率	211
3.6.6.6 读扩展轴步长及速度倍率	212
3.6.7 机器人限值	213
3.6.7.1 设置限制模式	213
3.6.7.2 读取限制模式	214
3.6.7.3 设置自定义模式下参数	215
3.6.7.4 读取自定义模式下参数	216
3.6.7.5 设置碰撞后处理方式	217
3.6.7.6 读取碰撞后处理方式	218
3.6.8 关节限值	219
3.6.8.1 设置关节限值	219
3.6.8.2 读取关节限值	220
3.6.9 安全平面	221
3.6.9.1 读取安全平面个数	221
3.6.9.2 删除安全平面	222
3.6.9.3 读取安全平面信息	223
3.6.9.4 增加安全平面	224
3.6.9.5 修改安全平面数据	225
3.6.10 安全 I/O	226
3.6.10.1 设置可配置输入信号	227
3.6.10.2 读取可配置输入信号	228
3.6.10.3 设置可配置输出信号	229
3.6.10.4 读取可配置输出信号	230
3.6.11 示教器控制	230
3.6.11.1 设置是否启用示教器	231
3.6.11.2 读取当前是否启用示教器	232

3.6.12 IO 变量	232
3.6.12.1 设置数字输入信号配置	233
3.6.12.2 读取数字输入信号配置	234
3.6.12.3 设置数字输出信号配置	235
3.6.12.4 读取数字输出信号配置	236
3.6.12.5 设置模拟量输入信号类型	238
3.6.12.6 设置工具 IO 类型	239
3.6.12.7 读取工具 IO 类型	240
3.6.12.8 读取模拟量输入信号类型	241
3.6.12.9 设置模拟量输出信号类型	242
3.6.12.10 读取模拟量输出信号类型	243
3.6.12.11 设置工具电压输出值	244
3.6.12.12 读取工具电压输出值	245
3.6.12.13 设置编码器配置	246
3.6.12.14 读取编码器配置	247
3.6.13 通讯变量	248
3.6.13.1 设置位寄存器配置	248
3.6.13.2 读取位寄存器配置	249
3.6.13.3 读取所有位寄存器配置	250
3.6.13.4 设置 16 位整数寄存器变量名	251
3.6.13.5 读取 16 位整数寄存器变量名	252
3.6.13.6 读取所有 16 位整数寄存器变量名	253
3.6.13.7 设置 32 位整数寄存器变量名	254
3.6.13.8 读取 32 位整数寄存器变量名	255
3.6.13.9 读取所有 32 位整数寄存器变量名	256
3.6.13.10 设置浮点寄存器变量名	257
3.6.13.11 读取浮点寄存器变量名	258
3.6.13.12 读取所有浮点寄存器变量名	259
3.6.14 安装变量	260
3.6.14.1 读取安装变量个数	260
3.6.14.2 读取安装变量数据	261
3.6.14.3 增加安装变量	262
3.6.14.4 删除安装变量	263
3.6.14.5 修改安装变量	264
3.6.15 坐标系	265
3.6.15.1 读取点坐标系个数	265
3.6.15.2 读取点坐标系数据	266
3.6.15.3 读取点坐标系数据(通过名称)	267
3.6.15.4 增加点坐标系	268
3.6.15.5 删除点坐标系	269
3.6.15.6 修改点坐标系数据	270

3.6.15.7 读取线坐标系个数	272
3.6.15.8 读取线坐标系数据	273
3.6.15.9 读取线坐标系数据(通过名称)	274
3.6.15.10 增加线坐标系	275
3.6.15.11 删除线坐标系	277
3.6.15.12 修改线坐标系数据	278
3.6.15.13 读取面坐标系个数	281
3.6.15.14 读取面坐标系数据	282
3.6.15.15 读取面坐标系数据(通过名称)	283
3.6.15.16 增加面坐标系	284
3.6.15.17 删除面坐标系	286
3.6.15.18 修改面坐标系数据	287
3.6.15.19 读取基坐标系数据	290
3.6.15.20 读取工具坐标系数据	291
3.6.15.21 计算点坐标系	292
3.6.15.22 计算线坐标系	293
3.6.15.23 计算面坐标系	294
3.6.16 串口通讯	296
3.6.16.1 设置串口配置	296
3.6.16.2 读取串口配置	297
3.6.17 网络	298
3.6.17.1 设置控制柜网络配置	298
3.6.17.2 读取控制柜网络配置	299
3.6.17.3 设置控制柜从站配置	300
3.6.17.4 读取控制柜从站配置	301
3.6.18 modbus 主站	301
3.6.18.1 读取从站个数	302
3.6.18.2 读取从站数据	303
3.6.18.3 添加从站	304
3.6.18.4 删除从站	305
3.6.18.5 修改从站数据	306
3.6.18.6 从站连接/断开	307
3.6.18.7 获取从站连接状态	308
3.6.18.8 读取地址映射数量	309
3.6.18.9 读取地址映射数据	310
3.6.18.10 添加地址映射	311
3.6.18.11 删除地址映射	312
3.6.18.12 修改地址映射数据	313
3.6.19 力控	314
3.6.19.1 设置力控配置	314
3.6.19.2 读力控配置	317

3.6.19.3 开启力控并下发相关设置	318
3.6.19.4 关闭力控	319
3.6.19.5 设置力控控制参数	320
3.6.19.6 读力控控制参数	322
3.6.19.7 读力控数据	323
3.6.20 扩展轴	324
3.6.20.1 设置扩展轴使能状态	324
3.6.20.2 读取扩展轴使能状态	325
3.6.20.3 读取机械臂和扩展轴数据	326
3.6.20.4 设置扩展轴配置	326
3.6.20.5 获取扩展轴配置	328
3.6.20.6 增加扩展轴	329
3.6.20.7 删除扩展轴	330
3.7 通讯	331
3.7.1 EtherCAT	331
3.7.1.1 ethercat 扫描	331
3.7.1.2 读 SDO	332
3.7.1.3 写 SDO	333
3.7.1.4 读 ESC 寄存器	335
3.7.1.5 写 ESC 寄存器	336
3.7.1.6 读 FOE	337
3.7.1.7 写 FOE	338
3.7.1.8 读 EEPROM	340
3.7.1.9 写 EEPROM	341
3.7.1.10 读 ESC 状态	342
3.7.1.11 写 ESC 状态	343
3.7.1.12 周期通讯激活	344
3.7.1.13 读取数据从 PDO 映射地址	345
3.7.1.14 写入数据到 PDO 映射地址	346
3.7.1.15 读多个 Pdo 映射地址数据	348
3.7.2 CANOpen	349
3.7.2.1 CANopen 写 pdo 数据	349
3.7.2.2 CANopen 读 pdo 数据	350
四、综合示例	351
4.1 机械臂使能	351
4.2 工程控制	353
4.3 移动控制	355
4.4 轨迹再现	360
4.5 安装变量操作	365
4.6 点坐标系操作	367
4.7 输出弹窗	372

4.8 数学变量	374
4.9 TCP 操作	379
附录 A 速度和加速度范围	I

一、SDK 介绍

CGXi 的 CR SDK 是基于网络通信协议 TCP/IP 实现的，提供了用于操作机械臂的接口，支持 python，C/C++，C# 三种语言。

本文是基于 C# 开发，提供操作机械臂的接口。该 SDK 可以实现多机械臂连接（在连接过程中句柄具有唯一性，与机械臂一一对应），用户可根据具体需求二次开发。

二、数据结构

方便接口函数的理解，特将一些常用的数据结构做详细的解释：接口中的长度单位统一为毫米（mm），角度单位统一为度（°）。

2.1 接口调用返回值

```
public enum CRresult
{
    success = 0,           ///<成功；分别有以下 3 种：连接、获取参数、写入参数
    success = 0,           ///<成功；分别有以下 3 种：连接、获取参数、写入参数
    error = 1,             ///<一般错误
    thread_running = 2,    ///<通讯连接重复建立
    operate_timeout = 3,   ///<操作超时，无返回值；注：处理超时导致无结果输出
    result_invalid = 4,    ///<返回结果无效；原因：rpc 获取结果为空
    out_of_range = 5,      ///<输入数值越界；需输入在范围内的值
    mutex_invalid = 6,     ///<内部错误
    para_error = 7,        ///<参数无效；需检查输入参数是否为空
    no_result = 8,         ///<无对应数值
    no_assignTCPindex = 9, ///<未找到指定索引号 TCP 偏移量
    no_handle = 10,        ///<未创建相关句柄
    handle_repeat = 11,    ///<重复建立相同的句柄
    repeat_name = 12,      ///<重命名
    delete_invalid = 13,  ///<删除唯一的 TCP 或者负载
    set_bit_reg_invalid = 14, ///<同时设置位寄存器输入与输出
    repeat_id = 15,        ///
```



```

    ///力控 SDK 无效的指令：包括 forceEna 等于 1,0 以外的值；forcebasetype 除 0 以外的值；forcectrltype 除 1,0 以外的值；auto/manual 1,0 以外的值
    forcectrl_parablocked = 21,
    ///力控 SDK freedrive 模式下部分参数失效被屏蔽
    forcectrl_cmdblocked = 22,
    ///力控 SDK 机器人运动时或已在力控模式下时禁止再打开力控
    forcectrl_configblocked = 23,
    ///力控 SDK 机器人运动时或已在力控模式下时禁止下发配置
    forcectrl_notenable = 24,           ///力控 SDK 机器人使能之前不能打开传感器
    forcectrl_wrongsensor = 25,        ///力控 SDK 传感器型号选择错误
    outputrule_error = 26,             ///未设置系统输出，禁止启用自复位
    analoginputtype_error = 27
    ///模拟量输入类型错误：注：工具端模拟量输入类型只有电压
}

```

2.2 坐标系选择

```

public enum CoordinateType
{
    baseCoordinate = 0,           ///<基坐标系
    jointCoordinate = 1,          ///<关节坐标系
    PointCoordinate = 2,          ///<点坐标系
    LineCoordinate = 3,           ///<线坐标系
    PlaneCoordinate = 4,          ///<面坐标系
    ToolBaseCoordinate = 5,        ///<工具坐标系
    visionCoordinate = 6          ///<视角坐标系
}

```

2.3 点转换类型

```

public enum PointTransType
{
    pointTransStop = 0,           ///<过渡点停止
    pointTransConstantSpeed = 1,  ///<匀速过渡
    pointTransSmooth = 2          ///<柔顺过渡（不匀速，不停止）
}

```

2.4 位姿变换

```

public enum PoseTranType           ///<预留参数
{
    poseTranMoveToTargetPose = 0,
}

```

```
poseTranKeepStartPose = 1
}
```

2.5 运动触发模式

```
public enum MotiontriggerMode
{
    MovetriggerbySafetySingnal = 0,    ///<安全同步信号触发
    MovetriggerbyOnlyRpc = 1,          ///<指令直接触发
    MovetriggerbyMoveCache = 2         ///<移动指令会进入缓存区
}
```

2.6 TCP 偏移量

```
[StructLayout(LayoutKind.Sequential)]
public struct TCPMsg
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 20)]
    public char[] tcpName;                ///

```

2.7 运动参数

```
[StructLayout(LayoutKind.Sequential)]
public struct PointControlPara
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] pose;
    ///<坐标点向量: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] jointpos;
    ///<关节角度: 关节 1~6 (单位: °)
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] tcpOffset;
    ///

```

```

public double[] coordinatePose;
///<参考坐标系位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
[MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
public double[] speed;
///<speed 可以是 tcp 速度或者是关节速度,当 coordinateType 设置为 jointCoordinate 时,
该速度为关节速度, 其它情况是 tcp 速度
///<作为 tcp 速度时, 单位为: mm/s; 作为关节速度时, 单位为: °/s
[MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
public double[] acc;                                     ///<加速度
///<acc 可以是 tcp 加速度或者是关节加速度,当 coordinateType 设置为 jointCoordinate
时, 该速度为关节加速度, 其它情况是 tcp 加速度
///<作为 tcp 加速度时, 单位为: mm/s²; 作为关节加速度时, 单位为: °/s²
[MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
public double[] jerk;                                     ///<加加速度, 预留参数
public PointTransType pointTransType;                   ///<位置过渡方式
public double pointTransRadius;                         ///<过渡半径
public PoseTranType poseTranType;                       ///<姿态变换方式, 预留参数
public MotiontriggerMode motiontriggerMode;            ///<运动轨迹触发方式
}

```

2.8 简化版运动参数

```

[StructLayout(LayoutKind.Sequential)]
public struct PointControlParaSimple
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] pose;
    ///<坐标点向量: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] jointpos;
    ///<关节角度: 关节 1~6 (单位: °)
    public double[] tcpOffset;
    ///<tcp 偏移量: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
    public CoordinateType coordinateType;                ///<坐标系类型
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] coordinatePose;
    ///<参考坐标系位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] speed;                                ///<速度
    ///<speed 可以是 tcp 速度或者是关节速度,当 coordinateType 设置为 jointCoordinate 时,
    该速度为关节速度, 其它情况是 tcp 速度
    ///<作为 tcp 速度时, 单位为: mm/s; 作为关节速度时, 单位为: °/s
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]

```

```

public double[] acc; //<加速度
//<acc 可以是 tcp 加速度或者是关节加速度,当 coordinateType 设置为 jointCoordinate
时,该速度为关节加速度,其它情况是 tcp 加速度
//<作为 tcp 加速度时,单位为: mm/s2; 作为关节加速度时,单位为: %s2
public PointTransType pointTransType; //<位置过渡方式
public MotiontriggerMode motiontriggerMode; //<运动轨迹触发方式
}

```

2.9 圆弧运动参数

```

[StructLayout(LayoutKind.Sequential)]
public struct PointControlParaList
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 2)]
    public PointControlPara[] pointcontrolpara; //<过渡点和终点
    public int fixedrot; //<方向模式 0-无约束 1-固定
    public double centralangle; //<圆弧角,单位: °
}

```

2.10 运动模式

```

public enum MoveType
{
    DecStop = 0, //<当前运动减速停止
    ImdStop = 1, //<当前运动立即停止
    Run = 2 //<运动
}

```

2.11 关节状态

```

public enum JointModes
{
    JOINT_MODE_POWER_OFF = 1, //<断电
    JOINT_MODE_BOOTING = 2, //<机械臂上电至空闲状态下
    JOINT_MODE_IDLE = 3, //<上电完成,空闲状态 (ethercat——>op)
    JOINT_MODE_BACK_DRIVE = 4, //<反向驱动状态
    JOINT_MODE_RELEASE_BRAKE = 5, //<机械臂使能,松抱闸过程中
    JOINT_MODE_CSP_STOP = 7, //<关节使能静止(位置模式)
    JOINT_MODE_CSP_MOVING = 8, //<关节使能运动(位置模式)
    JOINT_MODE_CST_STOP = 9, //<关节使能静止(转矩模式)
    JOINT_MODE_CST_MOVING = 10, //<关节使能运动(转矩模式)
}

```

```
JOINT_MODE_FAULT = 11,           ///<错误状态
JOINT_MODE_READY_FOR_POWER_OFF = 12    ///<准备断电
}
```

2.12 机械臂状态

```
public enum RobotModes
{
    Closed = 0,                    ///<关闭
    Disconnect = 1,               ///<断开连接状态
    ConfirmSafty = 2,             ///<安全确认状态
    Booting = 3,                  ///<控制柜正在启动
    ControlerIdle = 4,            ///<控制柜启动完成，空闲状态
    ControlerUpdataFirmWare = 5,  ///<固件更新
    JointPowerOff = 6,            ///<控制柜启动完成，本体未上电
    JointPowerOn = 7,             ///<机器人上电过程
    JointIdle = 8,                ///<机器人上电完成，空闲
    BackDrive = 9,                ///<反向驱动
    ReleaseBrake = 10,            ///<机器人使能、松抱闸中
    Enable = 11,                  ///<机器人使能完成，并已完全松开抱闸
    CloseBrake = 12,              ///<机器人断使能、关抱闸中
    Jog = 100,                    ///<点动
    Teach = 101,                  ///<拖动示教
    ForceControlTest = 102,       ///<力控测试
    ProgramStop = 103,            ///<使能完成，程序停止
    ProgramPause = 104,           ///<使能完成，程序暂停
    ProgramStopping = 105,        ///<使能完成，程序和运动正在停止
    ProgramPauseing = 106,        ///<使能完成，程序和运动正在暂停
    ProgramRun_MotionStop = 107,  ///<程序运行，运动停止
    ProgramRun_MotionReducing = 108, ///<程序运行，运动减速中
    ProgramRun_MotionMoving = 109, ///<程序运行
    ProgramRun_MotionCanBlend = 110, ///<程序运行，机器人进入到可交融区域
    Imdstop = 111,                ///<紧急停止
    ProtectiveStop = 112          ///<防护停止
}
```

2.13 安全模式

```
public enum SafetyModes
{
    SAFETY_MODE_UNDEFINED_SAFETY_MODE = 1,    ///<未定义
}
```

```

SAFETY_MODE_VALIDATE_JOINT_ID = 2,          ///<验证关节的 ID
SAFETY_MODE_FAULT = 3,
///<安全错误, 其他错误出现一段时间后, 最终保持的错误状态
SAFETY_MODE_VIOLATION = 4,                ///<在安全平面处越界
SAFETY_MODE_ROBOT_EMERGENCY_STOP = 5,      ///<急停按钮按下
SAFETY_MODE_SYSTEM_EMERGENCY_STOP = 6,
///<可配置 IO 输入任意两路设置为紧急停机, 断开后出现的错误状态
SAFETY_MODE_SAFEGUARD_STOP = 7,           ///<安全 IO 断开
SAFETY_MODE_RECOVERY = 8,
///<在违反安全限制后, 机器人停止, 重启后, 进入恢复模式
SAFETY_MODE_PROTECTIVE_STOP = 9,
///<保护性停止, 运动过程中, 外部强力推动机械手
SAFETY_MODE_REDUCED = 10,                 ///<限制模式
SAFETY_MODE_NORMAL = 11                   ///<正常运行模式
}

```

2.14 程序状态

```

public enum ProgramState
{
    PROGRAM_STATE_STOP = 0,          ///<程序停止
    PROGRAM_STATE_PAUSE = 1,         ///<程序暂停
    PROGRAM_STATE_RUN = 2             ///<程序运行
}

```

2.15 弹窗信息

```

public struct PopUpMsg
{
    public int popupType;              ///<弹窗类型, 0-输出弹窗, 1-输入弹窗
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 1024)]
    public byte[] var_data;           ///<弹窗显示的字符串, 字符串类型
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 1024)]
    public byte[] var_title;          ///<弹窗标题, 字符串类型
    public int iswarning;              ///<是否是警告信息, bool 类型
    public int iserror;               ///<是否是报警信息, bool 类型
    public int isblocking;            ///<是否暂停程序运行, bool 类型
    public VariableType inVarType;     ///<输入弹窗时, 变量类型
    public double inVarMinValue;       ///<输入弹窗时, 变量数值范围最小值
    public double inVarMaxValue;       ///<输入弹窗时, 变量数值范围最大值
}

```

2.16 版本信息

```
public struct Version
{
    [MarshalAs(UnmanagedType.ByValTStr, SizeConst = 20)]
    public string deviceType;           ///<机器人类型
    [MarshalAs(UnmanagedType.ByValTStr, SizeConst = 20)]
    public string versionNo;           ///<版本信息
    public int bugfix;                 ///

```

2.17 Lua 变量

```
public enum VariableType
{
    LUA_TNIL = 0,                    ///<空
    LUA_TBOOLEAN = 1,               ///<布尔
    LUA_TLIGHTUSERDATA = 2,         ///<预留
    LUA_TNUMBER = 3,                ///<数字
    LUA_TSTRING = 4,                ///<字符串
    LUA_TTABLE = 5,                 ///<表
    LUA_TFUNCTION = 6,              ///<函数
    LUA_TUSERDATA = 7,              ///<自定义类型
    LUA_TTHREAD = 8,                ///<线程
    LUA_NUMTAGS = 9                  ///<预留
}
```

2.18 变量元素

```
[StructLayout(LayoutKind.Sequential)]
public struct VarItemData
{
    public VariableType varItemType;    ///<数据类型
    public int boolValue;               ///

```

```
}
```

2.19 变量信息

```
[StructLayout(LayoutKind.Sequential)]
```

```
public struct VariableMsg
```

```
{
```

```
    [MarshalAs(UnmanagedType.ByValTStr, SizeConst = 100)]
```

```
    public string variableName;           ///<变量名
```

```
    public VariableType variableType;     ///<数据类型
```

```
    public int variableID;                ///<变量 id
```

```
    public int boolValue;                 ///
```

```
    public double numberValue;            ///
```

```
    [MarshalAs(UnmanagedType.ByValTStr, SizeConst = 100)]
```

```
    public string stringValue;            ///
```

```
    public int tableValueCount;           ///




```

```
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 500)]
```

```
    public VarItemData[] tableValue;      ///




```

```
    [MarshalAs(UnmanagedType.ByValTStr, SizeConst = 10000)]
```

```
    public string tableValueStr;          ///




```

```
}
```

2.20 消息来源

```
public enum CommMessageSource
```

```
{
```

```
    Base = 0,                            ///<基座关节（关节 1）
```

```
    Shoulder = 1,                        ///<肩部关节（关节 2）
```

```
    Elbow = 2,                           ///<肘部关节（关节 3）
```

```
    Wrist1 = 3,                          ///<腕关节 1（关节 4）
```

```
    Wrist2 = 4,                          ///<腕关节 2（关节 5）
```

```
    Wrist3 = 5,                          ///<腕关节 3（关节 6）
```

```
    Tool = 6,                            ///<工具模块
```

```
    Controller = 100,                    ///<主控制柜
```

```
    SafetyProcessorB = 101,              ///<协处理器
```

```
    TeachPendant = 200                   ///<示教器
```

```
}
```

2.21 警告等级

```
public enum WarningLevel
```

```
{
```



```

MESSAGE_WARNING_LEVEL_INFO = 1,           ///<正常消息
MESSAGE_WARNING_LEVEL_WARNING = 2,         ///<警告不停机
MESSAGE_WARNING_LEVEL_VIOLATION = 3,       ///<违反限制停机
MESSAGE_WARNING_LEVEL_FAULT = 4,           ///<系统故障，紧急停止
MESSAGE_WARNING_LEVEL_USER = 5             ///<用户打印输出
}

```

2.22 世界时间

```

public struct WorldTime
{
    public uint year;           ///<年
    public uint month;          ///<月
    public uint day;            ///<日
    public uint week;           ///<周
    public uint hour;           ///<时
    public uint minute;         ///<分
    public uint second;         ///<秒
    public uint milliSecond;    ///<毫秒
}

```

2.23 日志信息

```

[StructLayout(LayoutKind.Sequential)]
public struct RobotLogMsg
{
    public uint robotMessageCode;           ///<消息代码
    public uint robotMessageType;           ///<消息类型，预留
    public CommMessageSource source;        ///<消息来源
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 100)]
    public byte[] textMessage;              ///<消息信息
    public WorldTime worldTime;             ///<时间戳
    public WarningLevel warningLevel;       ///<警告等级
    public RobotModes robotMode;            ///<机器人状态
    public ulong timeStamp_us;              ///<精确时间戳
    public ulong messageIndex;              ///<消息序号
}

```

2.24 版本信息

```

[StructLayout(LayoutKind.Sequential, CharSet = CharSet.Ansi)]
public struct SysVersion

```

```
{
    public Version sjqVersion;          ///<示教器版本
    public Version kzqVersion;          ///<控制柜版本
    public Version jpmVersion;          ///<本体放电板版本
    public Version saftyCtrlVersion;    ///<安全板版本
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public Version[] jointVersion;      ///<关节版本
    public Version etfVersion;          ///<工具法兰版本
}
```

2.25 负载

```
[StructLayout(LayoutKind.Sequential)]
public struct PayLoad
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 20)]
    public char[] payloadName;          ///<负载名称
    public double toolPayload;          ///<负载重量: kg
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 3)]
    public double[] centerOfGravity;    ///<负载中心（CX, CY, CZ）（单位: mm）
}
```

2.26 校验码检查

```
public enum ParityCheck
{
    no = 0,          ///<无校验
    odd = 1,         ///<奇校验
    even = 2         ///<偶校验
}
```

2.27 串口通信

```
public enum SerialCommuType
{
    FreeComm = 0,          ///<自由格式
    Modbus_RTU = 1,        ///

```

2.28 波特率

```
public enum BaudRate
```

```

{
    BaudRate300 = 0,           ///<波特率为 300Baud
    BaudRate600 = 1,           ///<波特率为 600Baud
    BaudRate1200 = 2,          ///<波特率为 1200Baud
    BaudRate2400 = 3,          ///<波特率为 2400Baud
    BaudRate4800 = 4,          ///<波特率为 4800Baud
    BaudRate9600 = 5,          ///<波特率为 9600Baud
    BaudRate19200 = 6,         ///<波特率为 19200Baud
    BaudRate28800 = 7,         ///<波特率为 28800Baud
    BaudRate38400 = 8,         ///<波特率为 38400Baud
    BaudRate57600 = 9,         ///<波特率为 57600Baud
    BaudRate115200 = 10,        ///<波特率为 115200Baud
    BaudRate192000 = 11,        ///<波特率为 192000Baud
    BaudRate256000 = 12,        ///<波特率为 256000Baud
    BaudRate288000 = 13,        ///<波特率为 288000Baud
    BaudRate384000 = 14,        ///<波特率为 384000Baud
    BaudRate512000 = 15,        ///<波特率为 512000Baud
    BaudRate576000 = 16,        ///<波特率为 576000Baud
    BaudRate768000 = 17,        ///<波特率为 768000Baud
    BaudRate1000000 = 18,       ///<波特率为 1000000Baud
    BaudRate1200000 = 19,       ///<波特率为 1200000Baud
    BaudRate1500000 = 20,       ///<波特率为 1500000Baud
    BaudRate2400000 = 21,       ///<波特率为 2400000Baud
    BaudRate3000000 = 22,       ///<波特率为 3000000Baud
}

```

2.29 串口配置

```

public struct SerialCommuStruct
{
    public BaudRate baudRate;           ///<波特率
    public ParityCheck parity;           ///<奇偶校验
    public int dataBits;                 ///<数据位
    public double stopBits;              ///<停止位
    public int retryNumber;              ///<重试次数
    public int timeout;                  ///<超时时间，单位：ms
    public SerialCommuType serialCommType; ///<串口通信类型
    public int modbusSlaveNo;            ///

```

2.30 IO 类型

```
public enum VarDigitalIOType
{
    Invaild = 0,           ///<无效
    Digital = 1,           ///<标准数字量
    ConfigureableDigital = 2, ///<可配置数字量
    ToolDigital = 3,       ///<工具
    Analog = 4,            ///<模拟量
    ToolAnalog = 5         ///<工具模拟量
}
```

2.31 安装角度

```
public struct InstallAngle
{
    public double baseAngle;    ///<基座旋转角度，单位：°
    public double tiltAngle;    ///<基座倾斜角，单位：°
}
```

2.32 系统功能输入

```
public enum InputAction
{
    InputAction_NONE = 0,           ///<无动作
    InputAction_RobotON = 1,        ///<本体上电使能
    InputAction_MovetoHome = 2,     ///<回初始点
    InputAction_RunProgram = 3,     ///<运行程序
    InputAction_SuspendedProgram = 4, ///<暂停程序
    InputAction_StopProgram = 5,    ///<停止程序
    InputAction_FreeDrive = 6,      ///<自由拖动
    InputAction_DownEnable = 7,     ///<下使能
    InputAction_ClearFault = 8      ///<清除故障
}
```

2.33 系统功能输出

```
public enum OutputAction
{
    OutputAction_NONE = 0,           ///<无动作
    OutputAction_RobotON_HIGH = 1,   ///<机器人使能时为高
    OutputAction_RobotON_LOW = 2,    ///<机器人使能时为低
    OutputAction_RobotPowerOn_HIGH = 3, ///<已上电时为高
    OutputAction_RobotPowerOn_LOW = 4,  ///<已上电时为低
}
```

```

OutputAction_PowerOn_HIGH = 5,           ///<已开机时为高
OutputAction_PowerOn_LOW = 6,            ///<已开机时为低
OutputAction_WarnErr_HIGH = 7,           ///<有系统错误时为高
OutputAction_WarnErr_LOW = 8,            ///<有系统错误时为低
OutputAction_ProgramStatus_Stop_HIGH = 10, ///<程序停止时为高
OutputAction_ProgramStatus_Stop_LOW = 11, ///<程序停止时为低
OutputAction_Fault_Stop_HIGH = 12,       ///<异常停止时为高
OutputAction_Fault_Stop_LOW = 13,        ///<异常停止时为低
OutputAction_ProgramStatus_Pause_HIGH = 14, ///<程序暂停输出为高
OutputAction_ProgramStatus_Pause_LOW = 15, ///<程序暂停输出为低
OutputAction_MotionStatus_Moving_HIGH = 20, ///<运动时为高
OutputAction_MotionStatus_Moving_LOW = 21, ///<运动时为低
OutputAction_MovetoHome_HIGH = 22
///<回初始位到位信号，到位时为高
}

```

2.34 IO 选项卡输出

```

public enum OutputOptions           ///

```

2.35 输出信号状态（自动复位）

```

public enum OutputRule
{
    OutputRule_Disable = 0,         ///<禁用
    OutputRule_Enable = 1           ///<启用
}

```

2.36 输入信号

```

public struct IOInputConfiguration
{
    public int inputFilteringTime;    ///<输入信号滤波时间，单位:ms
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 32)]
    public char[] inputName;         ///<输入信号名称
    public InputAction inputActions;  ///<输入信号操作
}

```

2.37 输出信号

```
public struct IOOutputConfiguration
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 32)]
    public char[] outputName;           ///<输出信号名称
    public OutputAction outputActions;  ///<输出信号条件
    public OutputOptions outputOptions; ///<输出信号操作控制选项
    public OutputRule outputRule;       ///<输出信号状态（自动复位）
}
```

2.38 工具 IO 类型

```
public enum ToolIoType
{
    IO_OUTPUT = 0,           ///<输出
    IO_INPUT = 1             ///<输入
}
```

2.39 模拟量类型

```
public enum AnalogType
{
    Current = 0,             ///<电流
    Voltage = 1              ///<电压
}
```

2.40 模拟量输入

```
public struct AnalogInputConfiguration
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 32)]
    public char[] analogInputName;    ///<模拟量输入通道名称
    public AnalogType analogInputType; ///<模拟量输入类型
}
```

2.41 模拟量输出

```
public struct AnalogOutputConfiguration
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 32)]
    public char[] analogOutputName;    ///<模拟量输出通道名称
}
```

```

public AnalogType analogOutputType;          ///<模拟量输出类型
}

```

2.42 编码器配置

```

public struct EncoderConfiguration
{
    public int eqepType;          ///<类型
    public int eqepCountMode;     ///<模式
    public int strobeInputMode;   ///<探针
    public int eqepEnabled;      ///<控制
}

```

2.43 工具电压

```

public enum ToolPower
{
    Power_invaild = 0,           ///<无效值
    Power_on = 1,                ///<24V
    Power_off = 2,               ///<0V
}

```

2.44 BOOL 寄存器

```

public struct BitRegisterConfiguration
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 32)]
    public char[] GeneralPurposeBOOLeanRegisterName;
    ///<BOOL 型寄存器名称
    public InputAction GeneralPurposeBOOLeanRegisterInputActions;
    ///<BOOL 型寄存器系统功能输入
    public OutputAction GeneralPurposeBOOLeanRegisterOutputActions;
    ///<BOOL 型寄存器系统功能输出
    public OutputRule GeneralPurposeBOOLeanRegisterOutputRule;
    ///<BOOL 型寄存器自复位
}

```

2.45 寄存器

```

public struct RegisterConfiguration
{
    public char[] GeneralPurposeRegisterName;          ///<寄存器名称
}

```

2.46 工具端信息

```
public struct PoseMessage
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] toolPosition;
    ///<位姿: x, y, z (单位: mm) ; Rx, Ry, Rz (单位: °)
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] toolAxisAngle;
    ///<关节角度: 关节 1~6 (单位: °)
}
```

2.47 点坐标系

```
public struct PointCSNode
{
    public int isValid;           ///<是否有效
    public int id;                ///<id 号
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 32)]
    public char[] name;          ///<点坐标系名字
    public PoseMessage point;     ///<点坐标系数据
}
```

2.48 线坐标系

```
public struct LineCSNode
{
    public int isValid;           ///<是否有效
    public int id;                ///<id 号
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 32)]
    public char[] name;          ///<线坐标系名字
    public PointCSNode firstPoint; ///<第一个点的数据相关信息
    public PointCSNode secondPoint; ///<第二个点的数据相关信息
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] coordinatePose;
    ///<线坐标系位姿数据: x, y, z (单位: mm) ; Rx, Ry, Rz (单位: °)
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] coordinateJointPos;
    ///<线坐标系关节数据: 关节 1~6 (单位: °)
}
```


2.49 面坐标系

```
public struct PlaneCSNode
{
    public int isValid;           ///<是否有效
    public int id;               ///

```

2.50 机器人限值等级

```
public enum SafetyLimitsValuesType
{
    Userdefined = 0,           ///<自定义
    LimitLevel_1 = 1,         ///<等级 1（限制最多）
    LimitLevel_2 = 2,         ///<等级 2（限制偏多）
    LimitLevel_3 = 3,         ///<等级 3（限制偏少）
    LimitLevel_4 = 4,         ///<等级 4（限制最少）
}
```

2.51 碰撞后处理模式

```
public enum SafetyCollisionHandleMode
{
    Collision_EnterReboundMode = 0, ///<进入反弹模式
    Collision_ProgramPause = 1,      ///<暂停
    Collision_ProgramStop = 2,       ///<停止
}
```

2.52 关节限值模式

```
public enum LimitsBoundaryPlaneMode
```

```

{
    Disabled = 0,           ///<禁用
    Normal = 1,             ///<正常
    Reduced = 2,            ///<缩减
    Both = 3,               ///<二者都是
    Trigger = 4             ///<触发器缩减模式
}

```

2.53 安全限值模式

```

public enum SafetyLimitsMode
{
    invalid = 0,             ///<禁用
    mode_Normal = 1,         ///<正常
    mode_Reduced = 2         ///<缩减
}

```

2.54 安全限值

```

public struct SafetyLimitsValues
{
    public double maxTcpSpeed;           ///

```

2.55 关节限值

```

public struct SafetyLimitsJointAngle
{
    public double maxJointSpeed;          ///<最大角速度,单位: ° /s
    public double minJointPosition;       ///<最小位置, 单位: °
    public double maxJointPosition;       ///<最大位置, 单位: °
}

```

2.56 安全平面

```
public struct SafetyLimitsBoundaryPlane
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 32)]
    public char[] name;           ///<安全平面名字
    public int id;                 ///
```

2.57 安全 IO 输入信号

```
public enum SafetyIOInput
{
    input_invalid = 0,           ///<未分配
    emergencyStop = 1,           ///<紧急停机
    reducedMode = 2,             ///<缩减模式
    safeguardStop = 3,           ///<安全停止
    safeguardReset = 6           ///<安全重置
}
```

2.58 安全 IO 输出信号

```
public enum SafetyIOOutput
{
    output_invalid = 0,           ///<未分配
    systemEmergencyStop = 1,       ///<紧急停止输出
    robotMoving = 2,              ///<机器人运动
    robotNotStopping = 3          ///<机器人未停止
}
```

2.59 控制柜网络配置

```
public struct IPConfig
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 4)]
```

```
public int[] ipAddr;           ///
```

2.60 MODBUS 地址映射功能

```
public enum ModbusMasterFunctionNo
{
    ReadBool = 0,               ///<读位寄存器
    WriteBool = 1,              ///<写位寄存器
    ReadInt16 = 2,              ///<读 16 位整数寄存器
    WriteInt16 = 3,             ///<写 16 位整数寄存器
    ReadInt32 = 4,              ///<读 32 位整数寄存器
    WriteInt32 = 5,             ///<写 32 位整数寄存器
    ReadFloat = 6,              ///<读浮点寄存器
    WriteFloat = 7,             ///<写浮点寄存器
    ReadWriteBool = 8,          ///<预留
    ReadWriteInt16 = 9,         ///<预留
    ReadWriteInt32 = 10,        ///<预留
    ReadWriteFloat = 11,        ///<预留
}
```

2.61 MODBUS 地址映射

```
public struct ModbusAddrMap
{
    public ModbusMasterFunctionNo functionNo;    ///
```

2.62 MODBUS 连接状态

```
public enum ModbusMasterOperate
{
    Start = 0,           ///<开始连接
    Stop = 1,            ///<断开连接
}
```

2.63 MODBUS 参数配置

```
public struct ModbusMasterConfig
{
    public int id; //<从站 id 号
    public int slaveNo; //<从站站号
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 4)]
    public int[] slaveIP; //<modbus-tcp 时，从站 ip
    public int scanTime; //<扫描周期
    public ModbusMasterOperate modbusMasterOperate; //< modbus 连接状态
    public int autoConnect; //<自动连接
}
```

2.64 通讯变量选择

```
public enum CommuVarType
{
    CommuVarType_Bool = 0, //<bool 类型
    CommuVarType_Int16 = 1, //<16bit 整数类型
    CommuVarType_Int32 = 2, //<32bit 整数类型
    CommuVarType_Float = 3 //<32bit 浮点类型
}
```

2.65 脚本程序状态

```
public enum Lua_ScriptStatus
{
    lua_Script_NoneOP = 0, //<脚本为空
    lua_Script_stop = 1, //<脚本程序停止
    lua_Script_pause = 2, //<脚本程序暂停
    lua_Script_run = 3, //<脚本程序运行
    lua_Script_load = 4 //<脚本程序正在加载
}
```

2.66 串口类型

```
public enum SerialType
{
    rs485_tool_CommSettings = 0, //<工具 485
    rs485_1_CommSettings = 1, //<485-1
    rs485_2_CommSettings = 2, //<485-2
    rs232_CommSettings = 3, //<rs232
}
```

```
    ethernet_CommSettings = 4    ///  
}
```

2.67 轨迹点数据

```
public struct PathPoint  
{  
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]  
    public double[] pose;  
    ///  
    ///  
    ///  
    ///  
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]  
    public double[] jointpos;  
    ///  
    ///  
    ///  
}
```

2.68 轨迹参数

```
public struct PathPara  
{  
    public int index;    ///  
    public int moveType;    ///  
    public double speed;    ///  
    public double acc;    ///  
    public double blendRadius;    ///  
}
```

2.69 轨迹数据

```
public struct PathData  
{  
    public IntPtr pathPoints;    ///  
    public int pathPointsNum;    ///  
    public int moveTime;    ///  
    ///  
}
```

2.70 下载轨迹数据

```
public struct PathDownloadData  
{  
    public PathData pathData;    ///  
    public PathPara pathPara;    ///  
}
```

2.71 轨迹记录参数

```
public struct RecordPathPara
{
    public int sampleTime;        ///<采样时间 recordControl 为 1 时有效，单位：ms
    public int recordControl;     ///<记录控制，0-停止，1-启动，2-暂停
}
```

2.72 轨迹记录状态

```
public struct PathRecordStatus
{
    public int recordStatus;      ///<0-记录完成，1-记录中，2-暂停记录
    public int waypointNumber;    ///<已记录点数
}
```

2.73 轨迹运行状态

```
public struct PathRunMsg
{
    public int pathrunstatus;     ///<0 或 10001-轨迹控制停止 1-轨迹运行中
    public int pointIndex;        ///<当前运行轨迹索引
}
```

2.74 控制模式

```
public enum ControlModes
{
    CONTROL_MODE_POSITION = 0,    ///<控制器启动后的各种 RobotModes
    CONTROL_MODE_TEACH = 1,
    ///
```

2.75 力控设置

```
public struct ForceSetting
{
    public int forceBaseType;
    ///<力控类型： 0-基于 TCP 力传感器,1-基于动力学,2-基于底座力传感器
}
```

```
public int forceCtrlType;    ///<力控用法: 0-运动力控,1-力控拖动
[MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
public int[] flexibleAxis;    ///<力控轴
}
```

2.76 力控传感器 tcp

```
public struct ForceTCPMsg
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 20)]
    public byte[] forceTcpName;    ///<力控传感器 tcp 的名称
    public int forceTcpId;    ///<力控传感器 tcp 的 ID
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] forceTcpOffset;    ///<力控传感器 tcp 偏移量
}
```

2.77 力控传感器负载

```
public struct ForcePayload
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 20)]
    public byte[] forcePayloadName;    ///<力控传感器负载名称
    public int forcePayloadId;    ///<力控传感器负载 ID
    public double forceToolPayload;
    ///<力控传感器负载重量（单位：kg）
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 3)]
    public double[] forceCenterOfGravity;    ///<负载中心（CX，CY，CZ）
}
```

2.78 力控传感器信息

```
public struct SensorMessage
{
    public int venderNo;    ///<力传感器厂家号: 0-CGX,1-蓝点,2-鑫精诚
    public int productNo;    ///<力传感器产品号
    public int sequenceNo;    ///<力传感器通讯拓扑顺序号
}
```

2.79 通讯配置

```
public struct CommuConfig
{

```



```

public int autoConnect;          ///<自动连接  不处理-0  连接-1  关闭-2
public int mannaulOperate;       ///<手动连接  不处理-0  连接-1  关闭-2
public int paraBuf;              ///<预留参数
}

```

2.80 力控传感器工具设置

```

public struct ForceToolSetting
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 20)]
    public byte[] activeForceTCP_name;          ///<激活的力控 tcp 名称
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 20)]
    public byte[] activeForcePayload_name;       ///<激活的力控负载名称
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 20)]
    public ForceTCPMsg[] availableForceTCP;      ///<可以使用的力控 TCP
    public int availableForceTCPLen;             ///<可以使用的力控 TCP 数量
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 20)]
    public ForcePayLoad[] availableForcePayLoad; ///<可以使用的力控负载
    public int availableForcePayLoadLen;         ///<可以使用的力控负载数量
}

```

2.81 力控配置

```

public struct ForceConfig
{
    public SensorMessage sensorMessage;          ///<传感器信息
    public ForceToolSetting forceCtrlTcpImpl;     ///<力控工具设置
    public CommuConfig commuConfig;              ///<力控通讯配置
}

```

2.82 力控控制参数

```

public struct ForceCtlPara
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] taskFrame;                  ///<力控坐标系
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] wrench;                    ///<力给定
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] limits;                    ///<力控保护限制（范围：大于 0）
    public int limitsLen;                      ///<力控保护限制长度 0 或 6
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
}

```

```

public double[] mass;           ///<力控虚拟质量 (范围: 0.5 ~ 10)
public int massLen;             ///<力控虚拟质量长度 0、1 或 6
[MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
public double[] damping;        ///<力控虚拟阻尼 (范围: 0.1 ~ 10)
public int dampingLen;          ///<力控虚拟阻尼长度 0、1 或 6
[MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
public double[] stiffness;       ///<力控虚拟刚度 (范围: 0 ~ 10)
public int stiffnessLen;         ///<力控虚拟刚度长度 0、1 或 6
}

```

2.83 力控数据

```

public struct ForceData
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] ftRaw;          ///<力传感器原始值
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] ftCtrl;         ///<力传感器控制值
    public ControlModes controlMode; ///<控制模式
}

```

2.84 机器人状态数据

```

public struct RobotStateData
{
    public double[] tcpActualPose;    ///<实际 tcp 位置
    public double[] jointActualPos;    ///<实际关节位置
    public RobotModes robotMode;       ///<机械臂当前状态
    public int robotMoveStatus;        ///<机械臂运动状态
    public int robotSpeedPercent;      ///<机械臂速度百分比
    public int[] configurableDigitalOutput; ///<控制器可配置数字输出
}

```

2.85 机械臂和扩展轴数据

```

public struct RobotExjData
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 10)]
    public double[] actualExjointPos;    ///<扩展轴实际位置
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] actualJointPos;      ///<实际关节角度
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
}

```

```

public double[] actualTcpVector;          ///<实际 TCP 位姿
}

```

2.86 扩展轴关节移动参数

```

public struct MoveExjPara
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 12)]
    public double[] pose;
    ///<前六个代表关节姿态 (x, y, z (单位: mm); Rx, Ry, Rz (单位: °)), 后
    六个代表扩展轴位置
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 12)]
    public double[] jointPos;
    ///< 前六个代表关节 1~6 关节角度, 后六个代表扩展轴位置
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 12)]
    public double[] speed;
    ///< 前六个代表机械臂速度, 当坐标系类型为关节坐标系时, 分别代表一到六关节
    的速度°/s, 其他时候代表 tcp 速度 mm/s, 第七个代表扩展轴直线速度 mm/s, 第八个代
    表扩展轴旋转速度°/s
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 12)]
    public double[] acc;
    ///< 前六个代表关节加速度, 后六个无效
    ///< acc 可以是 tcp 加速度或者是关节加速度, 当 coordinateType 设置为
    jointCoordinate 时, 该速度为关节加速度, 其它情况是 tcp 加速度
    ///< 作为 tcp 加速度时, 单位为: mm/s2; 作为关节加速度时, 单位为: °/s2
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    double tcpOffset[ROB_AXIS_NUM];
    ///< tcp 偏移量:x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
    public double[] tcpOffset;          ///< tcp 的 ID 号
    public CoordinateType coordinateType;    ///< 坐标系类型
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 6)]
    public double[] coordinatePose;
    ///<参考坐标系位姿:x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
    public double pointTransRadius;      ///<过渡半径
    public MotiontriggerMode motiontriggerMode;    ///<运动轨迹触发方式
}

```

2.87 扩展轴点动运动参数

```

public struct MoveJogExjPara
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 10)]

```

```

public double[] jointPos;           ///<扩展轴位置
[MarshalAs(UnmanagedType.ByValArray, SizeConst = 10)]
public double[] speed;             ///<扩展轴速度
[MarshalAs(UnmanagedType.ByValArray, SizeConst = 10)]
public double[] acc;               ///<扩展轴加速度
}

```

2.88 扩展轴运动参数

```

public struct ExjMovePara
{
    public int exjCount;             ///<扩展轴数量
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 10)]
    public int[] index;             ///<扩展轴索引
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 10)]
    public double[] speed;          ///<扩展轴速度
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 10)]
    public double[] position;        ///<扩展轴位置
    public double acctime;           ///<扩展轴加速时间
    public MotiontriggerMode motiontriggerMode; ///<运动轨迹触发方式
}

```

2.89 扩展轴类型

```

public enum ExternalJointType
{
    line_externalJoint = 0,          ///<直线
    rotation_externalJoint = 1       ///<旋转
}

```

2.90 扩展轴配置

```

public struct ExjConfig
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 50)]
    public byte[] name;             ///< 扩展轴名称
    public ExternalJointType type;   ///< 扩展轴类型
    public double ratio;            ///< 传动比
    public double minlimit;         ///< 最小值
    public double maxlimit;         ///< 最大值
    public long zeroPosition;        ///< 零位(pulse)
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 50)]
}

```

```

public double[] exjToRef;           ///< 扩展轴相对于参考系的关系
[MarshalAs(UnmanagedType.ByValArray, SizeConst = 50)]
public byte[] masterExjName;        ///< 主轴名称
public double masterSlaveRate;      ///< 主从比
}

```

2.91 ethercat 从站信息

```

public struct EtherCATSlaveData
{
    public int index;                ///< 索引
    public int venderID;              ///< 版本 ID
    public int productCode;           ///< 产品号
    public int revisionNo;            ///< 软件版本
}

```

2.92 Esc 状态

```

public enum EscState
{
    ECT_STATE_INIT = 1,              ///< Init
    ECT_STATE_PREOP = 2,              ///< Pre-OP
    ECT_STATE_BOOT = 3,               ///< Bootstrap
    ECT_STATE_SAFEOP = 4,             ///< Safe-OP
    ECT_STATE_OP = 8,                 ///< OP
}

```

2.93 Ect 数据类型

```

public enum Ect_DataType
{
    Ect_BOOL = 1,                     ///< bool
    Ect_SINT = 2,                     ///< sint
    Ect_INT = 3,                      ///< int
    Ect_DINT = 4,                     ///< dint
    Ect_USINT = 5,                    ///< usint
    Ect_UINT = 6,                     ///< uint
    Ect_UDINT = 7,                    ///< uint
    Ect_REAL = 8,                     ///< real
    Ect_LREAL = 17,                   ///< lreal
    Ect_LINT = 21,                     ///< lint
    Ect_ULINT = 27,                    ///< ulint
    Ect_BYTE = 30,                     ///< byte
    Ect_WORD = 31,                     ///< word
}

```

```
Ect_DWORD = 32          ///
```

2.94 pdo 类型

```
public enum Ect_PdoType
{
    Ect_TxPdo = 1,          ///
```

2.95 pdo 数据

```
public struct PdoTypeData
{
    public int pathData;          ///<轨迹数据
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 8)]
    public byte[] pdoData;        ///<读出的数据
    public int pdoDataLen;        ///<读出的数据字节长度
    public Ect_DataType pdoDataType; ///<数据类型
    public Ect_PdoType pdoType;    ///
```

2.96 CANOpen pdo 类型

```
public enum CoPdoType
{
    CO_TX_PDO = 1,          ///
```

2.97 CANOpen pdo 数据

```
public enum CoDataType
{
    EDS_DT_BOOL = 1,          ///<0x0001
    EDS_DT_S8 = 2,            ///<0x0002
    EDS_DT_S16 = 3,           ///<0x0003
    EDS_DT_S32 = 4,           ///<0x0004
    EDS_DT_U8 = 5,            ///<0x0005
    EDS_DT_U16 = 6,           ///<0x0006
    EDS_DT_U32 = 7,           ///<0x0007
}
```

```
EDS_DT_FLOAT = 8,           ///<0x0008
EDS_DT_STRING = 9,          ///<0x0009; 一个字符占 8 位
EDS_DT_OCTET_STRING = 0xA,  ///<0x000A; 一个字符占 8 位
EDS_DT_UNICODE_STRING = 0xB, ///<0x000B; 一个字符占 16 位
EDS_DT_DOMAIN = 0xF,        ///<0x000F
EDS_DT_DOUBLE = 0x11,       ///<0x0011
EDS_DT_INT64 = 0x15,        ///<0x0015
EDS_DT_UINT64 = 0x1B        ///<0x001B
}
```

三、接口函数

3.1 基础

3.1.1 基础接口

3.1.1.1 机器人创建连接

函数

```
CRresult cr_create_robot(ref int robotHandle, string ipAddr, int port, string passwd)
```

简介

创建机器人连接。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	ref int	输出	机械臂实例句柄
ipAddr	string	输入	控制柜 IP 地址
port	int	输入	控制柜端口号为 2323
passwd	string	输入	预留

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
```

```

}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

3.1.1.2 机器人断开连接

函数

CRresult cr_destroy_robot(int robotHandle)

简介

断开机器人连接。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
Console.WriteLine(result.ToString());

```


3.1.1.3 机器人关机

函数

CRresult cr_shutdown(int robotHandle)

简介

机器人关机。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_shutdown(robotHandle);
//机器人关机
if (result != CRresult.success)//判断机器人关机是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.1.4 机械臂上电

函数

CRresult cr_poweron(int robotHandle)

简介

机械臂上电。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_poweron(robotHandle);
//机械臂上电
if (result != CRresult.success)//判断机械臂上电是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.1.5 机械臂断电

函数

CRresult cr_poweroff(int robotHandle)

简介

机械臂断电。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_poweroff(robotHandle);
//机械臂断电
if (result != CResult.success)//判断机械臂断电是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.1.6 机械臂使能

函数

CResult cr_enable(int robotHandle)

简介

机械臂使能。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
```

```
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_enable(robotHandle);
//机械臂使能
if (result != CRresult.sucess)//判断机械臂使能是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.1.7 机械臂关使能

函数

CRresult cr_disable(int robotHandle)

简介

机械臂关使能。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_disable(robotHandle);
```

```
//机械臂关使能
if (result != CRresult.sucess)//判断机械臂关使能是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.1.8 机械臂故障复位

函数

CRresult cr_FaultReset(int robotHandle)

简介

机械臂故障复位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_FaultReset(robotHandle);
//机械臂故障复位
if (result != CRresult.sucess)//判断机械臂故障复位是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
```

```
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.1.9 获取控制柜状态

函数

CResult cr_get_controlMode(int robotHandle, ref int controlmode)

简介

获取控制柜的自动或手动状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
controlmode	ref int	输出	控制柜状态：0-自动，1-手动

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int controlmode = -1;
result = CS.cr_get_controlMode(robotHandle, ref controlmode);
//获取控制柜状态
if (result != CResult.sucess)//判断获取控制柜状态是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(controlmode.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
```

```
Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.1.10 写机械臂速度百分比

函数

CRresult cr_set_robotSpeedPercent(int robotHandle, uint speedPercent)

简介

写机械臂速度百分比。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
speedPercent	uint	输入	机械臂速度百分比 (范围: 1-100)

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
uint speedPercent = 20;
result = CS.cr_set_robotSpeedPercent(robotHandle, speedPercent);
//写机械臂速度百分比
if (result != CRresult.success)//判断写机械臂速度百分比是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(speedPercent.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.1.11 读机械臂速度百分比

函数

CRresult cr_get_robotSpeedPercent(int robotHandle, ref uint speedPercent)

简介

读机械臂速度百分比。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
speedPercent	ref uint	输出	机械臂速度百分比 (范围: 1-100)

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
uint speedPercent = 1;
result = CS.cr_get_robotSpeedPercent(robotHandle, ref speedPercent);
//读机械臂速度百分比
if (result != CRresult.sucess)//判断读机械臂速度百分比是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(speedPercent.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```


3.1.1.12 获取 SDK 版本号

函数

CRresult cr_get_sdk_version(byte[] version)

简介

获取 SDK 版本号。

参数

参数名称	数据类型	输入/输出	描述
version	byte[]	输出	SDK 版本信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
byte[] version = new byte[20];
CRresult result = CS.cr_get_sdk_version(version);
if (result != CRresult.success)//判断获取 SDK 版本号是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string str = Encoding.UTF8.GetString(version);
Console.WriteLine(str);
```

3.1.1.13 获取机械臂状态数据

函数

CRresult cr_get_robotStateData(int robotHandle, ref **RobotStateData** robotStateData);

简介

获取机械臂状态数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
robotStateData	ref RobotStateData	输出	机械臂状态数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
```

```
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
RobotStateData robotStateData = new RobotStateData();
result = CS.cr_get_robotStateData(robotHandle, ref robotStateData); //获取机械臂状态数据
if (result != CResult.sucess) //判断获取机械臂状态数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("实际 tcp 位置: ");
for (int i = 0; i < 6; i++)
{
    Console.WriteLine(robotStateData.tcpActualPose[i].ToString());
    if (i < 5)
        Console.WriteLine(",");
}
Console.WriteLine("\n");
Console.WriteLine("实际关节位置: ");
for (int i = 0; i < 6; i++)
{
    Console.WriteLine(robotStateData.jointActualPos[i].ToString());
    if (i < 5)
        Console.WriteLine(",");
}
Console.WriteLine("\n");
Console.WriteLine("机械臂当前状态: " + robotStateData.robotMode.ToString());
Console.WriteLine("机械臂运动状态: " + robotStateData.robotMoveStatus.ToString());
Console.WriteLine("机械臂速度百分比: " + robotStateData.robotSpeedPercent.ToString());
Console.WriteLine("控制器可配置数字输出: ");
for (int i = 0; i < 8; i++)
{
    Console.WriteLine(robotStateData.configurableDigitalOutput[i].ToString());
    if (i < 7)
        Console.WriteLine(",");
}
Console.WriteLine("\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.2 程序控制

3.1.2.1 下载脚本程序

函数

CRresult cr_downloadProgram(int robotHandle, string programfile)

简介

下载脚本程序。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
programfile	string	输入	需下载的脚本程序

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
string programfile = @"function mainFuncProgram()
    --start robotConfig
    TCP_1 = { 0,0,0,0,0,0 }
    Payload_1= { 0,{0,0,0} }
    set_tcp_payload(Payload_1[1], Payload_1[2])
    --end robotConfig
    function RobotProgram()
    while (true)
    do
    --output: CDO_0 = true
    set_standard_digital_out(0, true)
    wait(1) --sync
    end
    end
    RobotProgram_Result = task_create(RobotProgram)
    function PauseFuncProgram()
    while (true)
    do
```

```

        wait(100) --sync
    end
end
PauseFuncProgram_Result = task_create(PauseFuncProgram)
end
mainFuncProgram_Result = task_create(mainFuncProgram)";
result = CS.cr_downloadProgram(robotHandle, programfile);
//加载程序
if (result != CRresult.sucess)//判断加载程序是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
}

```

3.1.2.2 下载脚本程序（加密）

函数

CRresult cr_downloadProgram_encryption(int robotHandle, string programfile)

简介

下载脚本程序（加密）。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
programfile	string	输入	需下载的脚本程序

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
}

```

```

string programfile = @"function mainFuncProgram()
    --start robotConfig
    TCP_1 = { 0,0,0,0,0,0 }
    Payload_1= { 0,{0,0,0} }
    set_tcp_payload(Payload_1[1], Payload_1[2])
    --end robotConfig
    function RobotProgram()
    while (true)
    do
    --output: CDO_0 = true
    set_standard_digital_out(0, true)
    wait(1)  --sync
    end
    end
    RobotProgram_Result = task_create(RobotProgram)
    function PauseFuncProgram()
    while (true)
    do
    wait(100)  --sync
    end
    end
    PauseFuncProgram_Result = task_create(PauseFuncProgram)
    end
    mainFuncProgram_Result = task_create(mainFuncProgram)";

result = CS.cr_downloadProgram_encryption(robotHandle, programfile);
//下载脚本程序（加密）
if (result != CRresult.sucess)//判断下载脚本程序（加密）是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

3.1.2.3 上传脚本程序

函数

CRresult cr_uploadProgram(int robotHandle, byte[] programfile)

简介

上传脚本程序。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
programfile	byte[]	输出	上传的脚本程序

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] programfile = new byte[1024 * 75 * 10];
string str = "";
result = CS.cr_uploadProgram(robotHandle, programfile);
//上传程序
if (result != CResult.success)//判断上传程序是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
str = Encoding.UTF8.GetString(programfile);
Console.WriteLine(Encoding.UTF8.GetString(programfile));
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.2.4 运行程序

函数

CResult cr_play(int robotHandle)

简介

运行当前程序。若当前无已加载的程序，请先通过 cr_downloadProgram 加载一个程序。

参数

参数名称	数据类型	输入/输出	描述
------	------	-------	----

robotHandle	int	输入	机械臂实例句柄
-------------	-----	----	---------

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_play(robotHandle);
//运行当前程序
if (result != CRresult.sucess)//判断运行当前程序是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.2.5 停止程序运行

函数

CRresult cr_stop(int robotHandle)

简介

停止运行。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
```

```
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_stop(robotHandle);
//停止运行
if (result != CRresult.sucess)//判断机停止程序运行是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.2.6 暂停程序运行

函数

CRresult cr_pause(int robotHandle)

简介

暂停运行。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
```



```
result = CS.cr_pause(robotHandle);
//暂停运行
if (result != CRresult.sucess)//判断暂停程序运行是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.2.7 读取脚本当前运行行号

函数

CRresult cr_script_current_line_get(int robotHandle, int[] currentLine, ref int lineLen)

简介

读取脚本当前运行行号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
currentLine	int[]	输出	脚本当前行号
lineLen	ref int	输出	脚本行号数组长度

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int[] currentLine = new int[20];
int lineLen = 0;
result = CS.cr_script_current_line_get(robotHandle, currentLine, ref lineLen);
//读取脚本当前运行行号
```

```
if (result != CRresult.sucess)//判断读取脚本当前运行行号是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(currentLine[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.2.8 读脚本运行状态

函数

CRresult cr_get_lua_scriptstatus(int robotHandle, ref **Lua_ScriptStatus** scriptstatus)

简介

读脚本运行状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
scriptstatus	ref Lua_ScriptStatus	输出	当前脚本运行状态

返回值

数据类型为 **CRresult**，若为 **sucess** 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
Lua_ScriptStatus scriptstatus = Lua_ScriptStatus.lua_Script_load;
result = CS.cr_get_lua_scriptstatus(robotHandle, ref scriptstatus);
//读脚本运行状态
if (result != CRresult.sucess)//判断读脚本运行状态是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
```

```
Console.WriteLine(scriptstatus.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.2.9 上传工程文件

函数

CRresult cr_uploadProject(int robotHandle, string FilePath, string filename)

简介

上传 crp 文件和脚本程序。如果先使用 cr_downloadProgram 下载脚本程序后再上传工程文件，则只有脚本程序有效，crp 文件为空。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
FilePath	string	输入	文件保存路径
filename	string	输入	.crp 和脚本程序的文件名

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
string FilePath = "C:/software/v1.6c/teachpendant_x64/program";
string filename = "脚本测试 demo 用";
result = CS.cr_uploadProject(robotHandle, FilePath, filename);
//工程文件上传
if (result != CRresult.sucess)//判断工程文件上传是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
```

```
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.2.10 下载工程文件

函数

CRresult cr_downloadProject(int robotHandle, string crpFilepathname, string crscriptFilepathname)

简介

下载 crp 文件和脚本程序，下载成功后可从示教器上看到目前处于工程暂停状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
crpFilepathname	string	输入	.crp 文件路径名
crscriptFilepathname	string	输入	脚本程序路径名

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
string crpFilepathname = "../test.crp";
string crscriptFilepathname = "../test.crscript";
//工程文件下载
result = CS.cr_downloadProject(robotHandle, crpFilepathname, crscriptFilepathname);
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

```
}
```

3.1.2.11 是否存在弹窗

函数

CRresult cr_script_popup_exist(int robotHandle, ref bool hasPopup)

简介

是否存在弹窗。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
hasPopup	ref bool	输出	弹窗结果：0-不存在 1-存在

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
bool hasPopup = false;
result = CS.cr_script_popup_exist(robotHandle, ref hasPopup);
//是否存在弹窗
if (result != CRresult.success)//判断是否存在弹窗是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(hasPopup.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.2.12 读取弹窗信息

函数

CRresult cr_script_popup_msg_get(int robotHandle, ref **PopUpMsg** popupMsg)

简介

读取弹窗信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
popupMsg	ref PopUpMsg	输出	弹窗信息

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PopUpMsg popupMsg = new PopUpMsg();
result = CS.cr_script_popup_msg_get(robotHandle, ref popupMsg);
//读取弹窗信息
if (result != CRresult.sucess)//判断读取弹窗信息是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string utf8String = Encoding.UTF8.GetString(popupMsg.var_data);
Console.WriteLine(utf8String);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.2.13 设置输入弹窗信息

函数

CRresult cr_script_popup_msg_set(int robotHandle, IntPtr invarMsg)

简介

设置输入弹窗信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
invarMsg	IntPtr	输入	弹窗信息

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
VariableMsg invarMsg = new VariableMsg();
invarMsg.variableType = VariableType.LUA_TSTRING;
invarMsg.variableID = 4;
invarMsg.variableName = "abc213";
invarMsg.stringValue = "This is Message";
int size = Marshal.SizeOf(invarMsg);
IntPtr ptr = Marshal.AllocHGlobal(size);
Marshal.StructureToPtr(invarMsg, ptr, true);
result = CS.cr_script_popup_msg_set(robotHandle, ptr);
//设置输入弹窗信息
if (result != CRresult.sucess)//判断设置输入弹窗信息是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(invarMsg.variableName);
Marshal.FreeHGlobal(ptr);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
```

```
Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.1.2.14 关闭弹窗

函数

CRresult cr_script_popup_close(int robotHandle)

简介

关闭弹窗。只能关闭输入弹窗，不能关闭输出弹窗。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_script_popup_close(robotHandle);
//关闭弹窗
if (result != CRresult.success)//判断关闭弹窗是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2 运动模块

3.2.1 运动接口

3.2.1.1 轴空间运动 moveJ

函数

CRresult cr_moveJ(int robotHandle, **PointControlPara** pointControlPara)

简介

轴空间运动，推荐使用 cr_move_joint 接口。移动相关速度和加速度范围见附录 A 速度和加速度范围。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pointControlPara	PointControlPara	输入	运动相关参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PointControlPara pointControlPara = new PointControlPara();
pointControlPara.speed = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlPara.acc = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlPara.pose = new double[6] { -66, -438, 887, -57.34, 0, -149 };
//在 MoveJ 中无影响
pointControlPara.jointpos = new double[6] { -60, -60, 60, -60, -60, -60 };
//目标点示例数据
pointControlPara.tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlPara.coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlPara.jerk = new double[6] { 60, 60, 60, 60, 60, 60 };
pointControlPara.tcpID = -1;
pointControlPara.coordinateType = CoordinateType.jointCoordinate;
pointControlPara.pointTransType = PointTransType.pointTransStop;
pointControlPara.pointTransRadius = 0;
pointControlPara.poseTranType = PoseTranType.poseTranMoveToTargetPose;
pointControlPara.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
result = CS.cr_moveJ(robotHandle, pointControlPara);
//轴空间运动
if (result != CRresult.sucess)//判断轴空间运动是否成功
```

```
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(pointControlPara.tcpID.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.1.2 轴空间运动 movejoint

函数

CRresult cr_move_joint(int robotHandle, **PointControlPara** pointControlPara, bool isBlock)

简介

轴空间运动。移动相关速度和加速度范围见附录 A 速度和加速度范围。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pointControlPara	PointControlPara	输入	运动相关参数
isBlock	bool	输入	true-阻塞 false-非阻塞

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PointControlParaSimple pointControlParaSimple = new PointControlParaSimple();
PointControlPara pointControlPara = new PointControlPara();

bool isBlock = true;
pointControlParaSimple.speed = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlParaSimple.acc = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlParaSimple.pose = new double[6] { -66, -438, 887, -57.34, 0, -149 };
//在 MoveJ 中无影响
```

```
pointControlParaSimple.jointpos = new double[6] { -60, -60, 60, -60, -60, -60 };
//目标点示例数据
pointControlParaSimple.tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple.coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple.coordinateType = CoordinateType.jointCoordinate;
pointControlParaSimple.pointTransType = PointTransType.pointTransStop;
pointControlParaSimple.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
CS.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple, ref
pointControlPara);

result = CS.cr_move_joint(robotHandle, pointControlPara, isBlock);
//轴空间运动
if (result != CRresult.sucess)//判断轴空间运动是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(pointControlPara.tcpID.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.1.3 直线运动 moveL

函数

CRresult cr_moveL(int robotHandle, **PointControlPara** pointControlPara)

简介

直线运动，推荐使用 cr_move_line 接口。移动相关速度和加速度范围见附录 A 速度和加速度范围。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pointControlPara	PointControlPara	输入	运动相关参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
```

```

//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PointControlPara pointControlPara = new PointControlPara();
pointControlPara.speed = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlPara.acc = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlPara.jointpos = new double[6] { 60, 60, -60, 60, 60, 60 };
//起始点示例数据
pointControlPara.pose = new double[6] { 0, -437, 887, -57.34, 0, -149 };
//目标点示例数据
pointControlPara.tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlPara.coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlPara.jerk = new double[6] { 60, 60, 60, 60, 60, 60 };
pointControlPara.tcpID = -1;
pointControlPara.coordinateType= CoordinateType.baseCoordinate;
pointControlPara.pointTransType = PointTransType.pointTransStop;
pointControlPara.pointTransRadius = 0;
pointControlPara.poseTranType = PoseTranType.poseTranMoveToTargetPose;
pointControlPara.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
result = CS.cr_moveL(robotHandle, pointControlPara);
//直线运动
if (result != CRresult.sucess)//判断直线运动是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(pointControlPara.tcpID.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

3.2.1.4 直线运动 moveline

函数

CRresult cr_move_line(int robotHandle, **PointControlPara** pointControlPara, bool isBlock)

简介

直线运动。移动相关速度和加速度范围见[附录 A 速度和加速度范围](#)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pointControlPara	PointControlPara	输入	运动相关参数
isBlock	bool	输入	true-阻塞 false-非阻塞

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PointControlPara pointControlPara = new PointControlPara();
PointControlParaSimple pointControlParaSimple = new PointControlParaSimple();

bool isBlock = true;
pointControlParaSimple.speed = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlParaSimple.acc = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlParaSimple.jointpos = new double[6] { 60, 60, -60, 60, 60, 60 };
//起始点示例数据
pointControlParaSimple.pose = new double[6] { 0, -437, 887, -57.34, 0, -149 };
//目标点示例数据
pointControlParaSimple.tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple.coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple.coordinateType = CoordinateType.baseCoordinate;
pointControlParaSimple.pointTransType = PointTransType.pointTransStop;
pointControlParaSimple.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;

CS.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple, ref
pointControlPara);
result = CS.cr_move_line(robotHandle, pointControlPara, isBlock);
//直线运动
if (result != CRresult.sucess)//判断直线运动是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(pointControlPara.tcpID.ToString());
result = CS.cr_destroy_robot(robotHandle);
```

```
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.1.5 点动运动

函数

CRresult cr_moveJog(RobotHandle robotHandle, **PointControlPara** pointControlPara)

简介

点动运动。连续点动控制变化时位置一次最多改变 100mm，角度最多改变 20°。移动相关速度和加速度范围见附录 A 速度和加速度范围。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pointControlPara	PointControlPara	输入	运动相关参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PointControlPara pointControlPara = new PointControlPara();
pointControlPara.speed = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlPara.acc = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlPara.pose = new double[6] { 0, 5, 0, 0, 0, 0 };
//沿坐标系 y 轴运动 5mm
pointControlPara.jointpos = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlPara.tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlPara.coordinatePose = new double[6] { 114.3, -390.498, 215.347, -180, 0, 0 };
//点坐标系
pointControlPara.jerk = new double[6] { 60, 60, 60, 60, 60, 60 };
pointControlPara.tcpID = -1;
pointControlPara.coordinateType = CoordinateType.PointCoordinate;
//基于点坐标系进行点动
```

```
pointControlPara.pointTransType = PointTransType.pointTransStop;
pointControlPara.pointTransRadius = 0;
pointControlPara.poseTranType = PoseTranType.poseTranMoveToTargetPose;
pointControlPara.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
result = CS.cr_moveJog(robotHandle, pointControlPara);
//点动运动
if (result != CRresult.sucess)//判断点动运动是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(pointControlPara.tcpID.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.1.6 圆弧运动

函数

CRresult cr_move_circle(int robotHandle, **PointControlParaList** pointControlParaList, bool isBlock);

简介

圆弧运动。移动相关速度和加速度范围见附录 A 速度和加速度范围。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pointControlParaList	PointControlParaList	输入	圆弧运动相关参数
isBlock	bool	输入	true-阻塞 false-非阻塞

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
```

```
PointControlParaSimple pointControlParaSimple0 = new PointControlParaSimple();
//初始点简化运动参数
PointControlPara pointControlPara0 = new PointControlPara();
//初始点运动参数
pointControlParaSimple0.speed = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlParaSimple0.acc = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlParaSimple0.pose = new double[6] { 0, 0, 0, 0, 0, 0 };
//初始点, 但在 MoveJ 中无影响
pointControlParaSimple0.jointpos = new double[6] { 60, 60, -60, 60, 60, 60 };
//初始点关节角
pointControlParaSimple0.tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple0.coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple0.coordinateType = CoordinateType.jointCoordinate;
pointControlParaSimple0.pointTransType = PointTransType.pointTransStop;
pointControlParaSimple0.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
result = CS.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple0, ref
pointControlPara0); //PointControlParaSimple0 数据转换为 PointControlPara0
if (result != CRresult.sucess)
//判断 PointControlParaSimple0 数据转换为 PointControlPara0 是否成功
{
    Console.WriteLine("PointControlParaSimple0 转化失败" + "\r\n");
}

PointControlParaSimple pointControlParaSimple1 = new PointControlParaSimple();
//中间点简化运动参数
PointControlPara pointControlPara1 = new PointControlPara();
//中间点运动参数
pointControlParaSimple1.speed = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlParaSimple1.acc = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlParaSimple1.jointpos = new double[6] { 60, 55, -65, 60, 60, 60 };
//中间点关节角
pointControlParaSimple1.pose = new double[6] { 0, 0, 0, 0, 0, 0 };
//中间点位姿
CS.cr_kineForward(robotHandle, pointControlParaSimple1.jointpos,
pointControlParaSimple1.pose);
//正解求中间点位姿
pointControlParaSimple1.tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple1.coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple1.coordinateType = CoordinateType.baseCoordinate;
pointControlParaSimple1.pointTransType = PointTransType.pointTransStop;
pointControlParaSimple1.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
result = CS.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple1, ref
pointControlPara1); //PointControlParaSimple1 数据转换为 PointControlPara1
```



```
if (result != CRresult.sucess)
//判断 PointControlParaSimple1 数据转换为 PointControlPara1 是否成功
{
    Console.WriteLine("PointControlParaSimple1 转化失败" + "\r\n");
}

PointControlParaSimple pointControlParaSimple2 = new PointControlParaSimple();
//终点简化运动参数
PointControlPara pointControlPara2 = new PointControlPara();
//终点运动参数
pointControlParaSimple2.speed = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlParaSimple2.acc = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlParaSimple2.jointpos = new double[6] { 60, 50, -70, 60, 60, 60 };
//终点关节角
pointControlParaSimple2.pose = new double[6] { 0, 0, 0, 0, 0, 0 };
//终点位姿
CS.cr_kineForward(robotHandle, pointControlParaSimple2.jointpos,
pointControlParaSimple2.pose);
//正解求终点位姿
pointControlParaSimple2.tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple2.coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple2.coordinateType = CoordinateType.baseCoordinate;
pointControlParaSimple2.pointTransType = PointTransType.pointTransStop;
pointControlParaSimple2.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
result = CS.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple2, ref
pointControlPara2); //PointControlParaSimple2 数据转换为 PointControlPara2
if (result != CRresult.sucess)
//判断 PointControlParaSimple2 数据转换为 PointControlPara2 是否成功
{
    Console.WriteLine("PointControlParaSimple2 转化失败" + "\r\n");
}

bool isBlock = true;
PointControlParaList pointControlParaList = new PointControlParaList();
pointControlParaList.pointcontrolpara = new PointControlPara[] { pointControlPara1,
pointControlPara2 };

pointControlParaList.fixedrot = 0;
pointControlParaList.centralangle = 0;
CS.cr_move_joint(robotHandle, pointControlPara0, isBlock);
//运动到起始点
result = CS.cr_move_circle(robotHandle, pointControlParaList, isBlock);
//圆弧运动
if (result != CRresult.sucess)//判断圆弧运动是否成功
```

```
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.1.7 运动控制

函数

CResult cr_moveControl(int robotHandle, **MoveType** moveType)

简介

运动控制。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
moveType	MoveType	输入	运动控制参数

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
MoveType moveType = MoveType.ImdStop;
result = CS.cr_moveControl(robotHandle, moveType);
//运动控制
if (result != CResult.sucess)//判断运动控制是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
```

```
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.1.8 设置阻塞移动中参数阈值

函数

CRresult cr_move_block_threshold_set(int robotHandle, double poseDelt, double jointDelt)

简介

设置阻塞移动中参数阈值。在阻塞移动中，当前位姿/关节角度与目标点的位姿/关节角度的相差值小于位姿/关节阈值时，则被判定为已移动到目标点。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
poseDelt	double	输入	位姿阈值 默认 0.02
jointDelt	double	输入	关节阈值 默认 0.1

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_move_block_threshold_set(robotHandle, 0.05, 0.05);
//将阻塞移动中的位姿阈值设置为 0.05，关节阈值设置为 0.05
if (result != CRresult.sucess)//判断设置阻塞移动中参数阈值是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
```

```
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.1.9 PointControlParaSimple 数据转换

函数

CRresult cr_move_pointControlPara_transfer(int robotHandle, **PointControlParaSimple** pointControlParaSimple, ref **PointControlPara** pointControlPara)

简介

PointControlParaSimple 转换为 PointControlPara。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pointControlParaSimple	PointControlParaSimple	输入	简化运动参数
pointControlPara	ref PointControlPara	输出	运动参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PointControlParaSimple pointControlParaSimple = new PointControlParaSimple();
pointControlParaSimple.acc = new double[6] { 10, 10, 10, 10, 10, 10 };
pointControlParaSimple.speed = new double[6] { 10, 10, 10, 10, 10, 10 };
pointControlParaSimple.tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple.coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple.coordinateType = CoordinateType.jointCoordinate;
pointControlParaSimple.pose = new double[6] { 60, 60, -60, 60, 60, 60 };
PointControlPara pointControlPara = new PointControlPara();
result = CS.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple, ref
pointControlPara);
//PointControlParaSimple 转换为 PointControlPara
if (result != CRresult.sucess)//判断 PointControlParaSimple 数据转换是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
```

```
}

Console.WriteLine(pointControlPara.pose[2].ToString());
```

3.2.1.10 设置软件自由驱动启动

函数

CRresult cr_set_softFreeDriveEnabled(int robotHandle, int softFreeDriveEnabled)

简介

设置软件自由驱动启动。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
softFreeDriveEnabled	int	输入	true 为进入拖拽模式，false 为退出拖拽模式

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_set_softFreeDriveEnabled(robotHandle, 1);
//设置软件自由驱动启动
if (result != CRresult.success)//判断设置软件自由驱动启动是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.1.11 读机械臂运动状态

函数

CRresult cr_get_robotMoveStatus(int robotHandle, ref int isRobotMoving)

简介

读机械臂运动状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
isRobotMoving	ref int	输出	机械臂当前是否运动：0-停止，1-运动

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int isRobotMoving = 1;
result = CS.cr_get_robotMoveStatus(robotHandle, ref isRobotMoving);
//读机械臂运动状态
if (result != CRresult.sucess)//判断/读机械臂运动状态是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(isRobotMoving.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.1.12 读取移动缓存区中移动指令的数量

函数

CRresult cr_move_cache_num_get(int robotHandle, ref int cacheNum)

简介

读取当前移动缓存区中移动指令的数量，当 PointControlPara 的 MotiontriggerMode 参数为 MovetriggerbyMoveCache 时，对应的移动指令会进入缓存区。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
cacheNum	ref int	输出	缓存区中移动指令数量

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int cacheNum = 0;
result = CS.cr_move_cache_num_get(robotHandle, ref cacheNum);
//读取移动缓存区中移动指令的数量
if (result != CRresult.sucess)//判断读取移动缓存区中移动指令的数量是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(cacheNum.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.2 运动状态信息

3.2.2.1 读实际 TCP 位置

函数

```
CRresult cr_get_tcpActualPose(int robotHandle, [MarshalAs(UnmanagedType.LPArray,
SizeConst = 6)] double[] pose)
```

简介

读实际 TCP 位置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pose	double[]	输出	待读取的值: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)

返回值

数据类型为 CRresult, 若为 sucess 或 0, 则表示成功, 其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败, 查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] pose = new double[6];
result = CS.cr_get_tcpActualPose(robotHandle, pose);
//读实际 tcp 位置
if (result != CRresult.sucess)//判断读实际 tcp 位置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

Console.WriteLine(pose[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接, 释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.2.2 读目标 TCP 位置

函数

```
CRresult cr_get_tcpTargetPose(int robotHandle, [MarshalAs(UnmanagedType.LPArray,
SizeConst = 6)] double[] pose)
```


简介

读目标 TCP 位置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pose	double[]	输出	待读取的值: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)

返回值

数据类型为 CRresult, 若为 sucess 或 0, 则表示成功, 其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败, 查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] pose = new double[6];
result = CS.cr_get_tcpTargetPose(robotHandle, pose);
//读目标 tcp 位置
if (result != CRresult.sucess)//判断读目标 tcp 位置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(pose[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接, 释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.2.3 读实际 TCP 速度

函数

```
CRresult cr_get_tcpActualSpeed(int robotHandle, [MarshalAs(UnmanagedType.LPArray,
SizeConst = 6)] double[] speed)
```

简介

读实际 TCP 速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
speed	double[]	输出	待读取的值：x，y，z（单位：mm/s）；Rx，Ry，Rz（单位：°/s）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] speed = new double[6];
result = CS.cr_get_tcpActualSpeed(robotHandle, speed);
//读实际 tcp 速度
if (result != CRresult.sucess)//判断读实际 tcp 速度是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(speed[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.2.4 读目标 TCP 速度

函数

```
CRresult cr_get_tcpTargetSpeed(int robotHandle, [MarshalAs(UnmanagedType.LPArray,
SizeConst = 6)] double[] speed)
```

简介

读目标 TCP 速度。

参数

参数名称	数据类型	输入/输出	描述
------	------	-------	----

robotHandle	int	输入	机械臂实例句柄
speed	double[]	输出	待读取的值: x, y, z (单位: mm/s) ; Rx, Ry, Rz (单位: °/s)

返回值

数据类型为 CResult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
int robotHandle = -1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败, 查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] speed = new double[6];
CS.cr_get_tcpTargetSpeed(robotHandle, speed);//读目标 tcp 速度
if (result != CResult.success)//判断读目标 tcp 速度是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(speed[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接, 释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.2.5 读实际 TCP 加速度

函数

CResult cr_get_tcpActualAcceleration(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] acceleration)

简介

读实际 TCP 加速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	Int	输入	机械臂实例句柄
acceleration	double[]	输出	待读取的值: x, y, z (单位: mm/s ²) ; Rx, Ry, Rz (单位: °/s ²)

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] acceleration = new double[6];
CS.cr_get_tcpActualAcceleration(robotHandle, acceleration);//读实际 tcp 加速度
if (result != CRresult.sucess)//判断读实际 tcp 加速度是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(acceleration[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.2.6 读当前使用的 TCP 偏移信息

函数

CRresult cr_get_currentTCPmsg(int robotHandle, ref TCPMsg tcpMsg)

简介

读当前使用的 TCP 偏移信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
tcpMsg	ref TCPMsg	输出	当前使用的 TCP 偏移信息

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
```

```

CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
TCPMsg tcpMsg = new TCPMsg();
result = CS.cr_get_currentTCPmsg(robotHandle, ref tcpMsg);
//读当前使用的 TCP 偏移信息
if (result != CRresult.sucess)//判断读当前使用的 TCP 偏移信息是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(tcpMsg.tcpName[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
    
```

3.2.2.7 读取当前使用负载信息

函数

CRresult cr_get_currentPayloadmsg(int robotHandle, ref **PayLoad** payloadMsg)

简介

读取当前使用负载信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
payloadMsg	ref PayLoad	输出	读取当前负载信息

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    
```

```

        Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
    }
    Payload payloadMsg = new Payload();
    result = CS.cr_get_currentPayloadmsg(robotHandle, ref payloadMsg);
    //读取当前使用负载信息
    if (result != CRresult.sucess)//判断读取当前使用负载信息是否成功
    {
        Console.WriteLine("接口运行失败" + "\r\n");
    }
    Console.WriteLine(payloadMsg.toolPayload.ToString());
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }
}

```

3.2.3 运动配置

3.2.3.1 读所有 TCP 偏移信息列表

函数

CRresult cr_get_allTCPmsg(int robotHandle, IntPtr tcpMsgList, ref int validNumber)

简介

读所有的 TCP 偏移信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
tcpMsgList	IntPtr	输出	所有 TCP 偏移信息所在的数组
validNumber	ref int	输出	TCP 偏移信息数组中有效数据的长度

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}

```

```

}
TCPMsg[] tcpMsgList = new TCPMsg[20];
int size = Marshal.SizeOf(typeof(TCPMsg));
IntPtr ptr = Marshal.AllocHGlobal(size * 20);
int validNumber = 0;
result = CS.cr_get_allTCPmsg(robotHandle, ptr, ref validNumber);
//读所有 TCP 偏移信息列表
if (result != CRresult.sucess)//判断读所有 TCP 偏移信息列表是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
for (int i = 0; i < validNumber; i++)
{
    IntPtr ptr_s = ptr + i * size;
    tcpMsgList[i] = (TCPMsg)Marshal.PtrToStructure(ptr_s, typeof(TCPMsg));
}
Marshal.FreeHGlobal(ptr);
Console.WriteLine(tcpMsgList[0].tcpOffset[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
    
```

3.2.3.2 读取所有负载信息

函数

CRresult cr_get_allPayloadmsg(int robotHandle, IntPtr payloadMsgList, ref int validNumber)

简介

读取所有负载信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
payloadMsgList	IntPtr	输出	读取所有负载所在的数组
validNumber	ref int	输出	读取负载数组中有效数据的长度

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
```

```

CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
Payload[] payloadMsgList = new Payload[20];
int size = Marshal.SizeOf(typeof(Payload));
IntPtr ptr = Marshal.AllocHGlobal(size * 20);
int validNumber = 0;
result = CS.cr_get_allPayloadmsg(robotHandle, ptr, ref validNumber); ;
//读取所有负载信息
if (result != CRresult.sucess)//判断读取所有负载信息是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
for (int i = 0; i < validNumber; i++)
{
    IntPtr ptr_s = ptr + i * size;
    payloadMsgList[i] = (Payload)Marshal.PtrToStructure(ptr_s, typeof(Payload));
}
Marshal.FreeHGlobal(ptr);
Console.WriteLine(payloadMsgList[0].toolPayload.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
    
```

3.2.3.3 获取指定 TCPoffset 的位姿

函数

```

CRresult cr_get_AssignTCP_Pose(int robotHandle, [MarshalAs(UnmanagedType.LPArray,
SizeConst = 6)] double[] dstTCPoffset, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)]
double[] dstPose)
    
```

简介

获取指定 TCPoffset 的位姿。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
dstTCPoffset	double[]	输入	需变换后 TCP 偏移量：x，y，z（单位：

			mm) ; Rx, Ry, Rz (单位: °)
dstPose	double[]	输出	变化为 dstTCPoffset 后的位姿: x, y, z (单位: mm) ; Rx, Ry, Rz (单位: °)

返回值

数据类型为 CResult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败, 查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] dstTCPoffset = new double[6];
dstTCPoffset[0] = 10;
double[] dstPose = new double[6];
result = CS.cr_get_AssignTCP_Pose(robotHandle, dstTCPoffset, dstPose);
//获取指定 TCPoffset 的位姿
if (result != CResult.success)//判断获取指定 TCPoffset 的位姿是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(dstPose[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接, 释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.4 关节状态

3.2.4.1 读实际关节位置

函数

```
CResult cr_get_jointActualPos(int robotHandle, [MarshalAs(UnmanagedType.LPArray,
SizeConst = 6)] double[] pos)
```

简介

读实际关节位置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pos	double[]	输出	待读取的值，关节 1~6（单位：°）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] actualpos = new double[6];
result = CS.cr_get_jointActualPos(robotHandle, actualpos);
//读实际关节位置
if (result != CResult.success)//判断读实际关节位置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(actualpos[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.4.2 读目标关节位置

函数

CResult cr_get_jointTargetPos(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] pos)

简介

读目标关节位置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

pos	double[]	输出	待读取的值，关节 1~6（单位：°）
-----	----------	----	--------------------

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] pos = new double[6];
result = CS.cr_get_jointTargetPos(robotHandle, pos);
//读目标关节位置
if (result != CRresult.sucess)//判断读目标关节位置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(pos[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.4.3 读实际关节速度

函数

CRresult cr_get_jointActualVelocity(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] velocity)

简介

读实际关节速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
velocity	double[]	输出	待读取的值，关节 1~6（单位：° /s）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] velocity = new double[6];
result = CS.cr_get_jointActualVelocity(robotHandle, velocity);
//读实际关节速度
if (result != CRresult.sucess)//判断读实际关节速度是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(velocity[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.4.4 读目标关节速度

函数

CRresult cr_get_jointTargetVelocity(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] velocity)

简介

读目标关节速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
velocity	double[]	输出	待读取的值，关节 1~6（单位：°/s）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
```

```
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] velocity = new double[6];
result = CS.cr_get_jointTargetVelocity(robotHandle, velocity);//读目标关节速度
if (result != CRresult.sucess)//判断读目标关节速度是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(velocity[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.4.5 读实际关节加速度

函数

CRresult cr_get_jointActualAcceleration(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] acceleration)

简介

读实际关节加速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
acceleration	double[]	输出	待读取的值，关节 1~6（单位：°/s ² ）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
```

```
}
double[] acceleration = new double[6];
CS.cr_get_jointActualAcceleration(robotHandle, acceleration); //读实际关节加速度
if (result != CRresult.sucess) //判断读实际关节加速度是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(acceleration[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess) //判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.4.6 读目标关节加速度

函数

CRresult cr_get_jointTargetAcceleration(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] acceleration)

简介

读目标关节加速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
acceleration	double[]	输出	待读取的值，关节 1~6（单位：°/s ² ）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess) //判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] acceleration = new double[6];
CS.cr_get_jointTargetAcceleration(robotHandle, acceleration); //读目标关节加速度
if (result != CRresult.sucess) //判断读目标关节加速度是否成功
```

```
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(acceleration[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.4.7 读实际关节电机电流

函数

CRresult cr_get_jointActualCurrent(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] current)

简介

读实际关节电机电流（数值和关节转矩等效，三关节符号相反）。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
current	double[]	输出	待读取的值，关节 1~6（单位：mA）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] current = new double[6];
result = CS.cr_get_jointActualCurrent(robotHandle, current);
//读实际关节电机电流
if (result != CRresult.sucess)//判断读实际关节电机电流是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(current[0].ToString());
```

```
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.4.8 读目标关节电机电流

函数

CRresult cr_get_jointTargetCurrent(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] current)

简介

读目标关节电机电流（数值和关节转矩等效，三关节符号相反）。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
current	double[]	输出	待读取的值，关节 1~6（单位：mA）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] current = new double[6];
result = CS.cr_get_jointTargetCurrent(robotHandle, current);
//读目标关节电机电流
if (result != CRresult.sucess)//判断读目标关节电机电流是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(current[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
```



```
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.4.9 读实际关节转矩

函数

CRresult cr_get_jointActualTorque (int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] torque)

简介

读实际关节转矩。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
torque	double[]	输出	待读取的值，关节 1~6 (单位：额定转矩/1000)

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] torque = new double[6];
result = CS.cr_get_jointActualTorque(robotHandle, torque);
//读实际关节转矩
if (result != CRresult.success)//判断读实际关节转矩是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(torque[0].ToString());
result = CS.cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.4.10 读实际关节采集的母线电压

函数

```
CRresult cr_get_jointActualVoltage(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] voltage)
```

简介

读实际关节采集的母线电压。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
voltage	double[]	输出	待读取的值，关节 1~6（单位：V）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] voltage = new double[6];
result = CS.cr_get_jointActualVoltage(robotHandle, voltage);
//读实际关节采集的母线电压
if (result != CRresult.success)//判断读实际关节采集的母线电压是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(voltage[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.4.11 读关节温度

函数

```
CRresult cr_get_jointTemperature(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] temperature)
```

简介

读关节温度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
temperature	double[]	输出	待读取的值，关节 1~6（单位：℃）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] temperature = new double[6];
result = CS.cr_get_jointTemperature(robotHandle, temperature);
//读关节温度
if (result != CRresult.success)//判断读关节温度是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(temperature[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.4.12 读关节模式

函数

```
CRresult cr_get_jointMode(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] JointModes[] jointMode)
```

简介

读关节模式。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
jointMode	JointModes[]	输出	关节模式

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
JointModes[] jointMode = new JointModes[6];
result = CS.cr_get_jointMode(robotHandle, jointMode);
//读关节模式
if (result != CRresult.success)//判断读关节模式是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(jointMode[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.5 轨迹复现

3.2.5.1 读取控制柜当前存在的轨迹索引

函数

CRresult cr_path_all_index_get(int robotHandle,int[] allPathIndex,ref int pathIndexLen)

简介

读取控制柜当前存在的轨迹索引。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
allPathIndex	int[]	输出	所有轨迹索引,最大为 51
pathIndexLen	ref int	输出	轨迹索引数组有效长度

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}

int[] allPathIndex = new int[51];
int pathIndexLen = -1 ;
result = CS.cr_path_all_index_get(robotHandle, allPathIndex, ref pathIndexLen);
//读取控制柜当前存在的轨迹索引

if (result != CRresult.sucess)//判断读取控制柜当前存在的轨迹索引是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(pathIndexLen + "\r\n");

result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.5.2 上传轨迹

函数

CRresult cr_path_upload(int robotHandle,int index, ref **PathData** pathData)

简介

上传轨迹。运行前可通过 **cr_path_all_index_get** 先判断 **index** 是否存在。上传轨迹的请求索引的取值范围为：-1~49。其中 0~49 索引下的轨迹保存在控制器 Flash 中，-1 索引下的轨迹数据是上一次轨迹记录控制功能中记录下的轨迹数据，存放在控制器 RAM 中，下电清空。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	轨迹索引：-1~49
pathData	ref PathData	输出	上传的轨迹数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}

PathData pathData = new PathData();
pathData.pathPoints = new IntPtr();
int size = Marshal.SizeOf(typeof(PathPoint));
pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);
PathPoint[] pathPoints = new PathPoint[10000];

result = CS.cr_path_upload(robotHandle, 4, ref pathData);
//上传索引为 4 的轨迹，使用前需判断索引为 4 的轨迹是否存在
if (result != CRresult.sucess)//上传轨迹是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

for (int points_count = 0; points_count < 10000; points_count++)
```

```
{
    IntPtr newPtr = IntPtr.Add(pathData.pathPoints, points_count * size);
    pathPoints[points_count] = (PathPoint)Marshal.PtrToStructure(newPtr, typeof(PathPoint));
    Console.WriteLine($"{points_count}");
}

Marshal.FreeHGlobal(pathData.pathPoints);

result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.5.3 下载轨迹

函数

CRresult cr_path_download(int robotHandle, **PathDownloadData** pathDownloadData)

简介

下载轨迹。该功能会覆盖控制柜已存在的轨迹，建议运行前通过 **cr_path_all_index_get** 接口先判断该索引是否存在。轨迹下载接口是多个功能复用的接口，其支持 TCP 轨迹复现、关节轨迹复现和分段 **movex** 功能。当 **moveType** 为 0 时，调用该接口时机器人会直接运行，提供的轨迹数据需要按照 2ms 规划，运行轨迹之前需将机器人移动到轨迹的第一个点位；当 **moveType** 为 1 时为 TCP 轨迹复现，需要调用 **cr_path_action** 接口来运行轨迹；当 **moveType** 为 4 时为关节轨迹复现，需要调用 **cr_path_action** 接口来运行轨迹；当 **moveType** 为 3 时，**movex** 运动，需要设置速度、加速度、交融半径，需要调用 **cr_path_action** 接口来运行轨迹。

注：在使用相关功能时，需要保证机械臂已经移动到轨迹起始点对应的位置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pathDownloadData	PathDownloadData	输入	下载轨迹数据

返回值

数据类型为 **CRresult**，若为 **sucess** 或 0，则表示成功，其它均为失败。

示例 1：下载轨迹

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
```

```
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PathDownloadData pathDownloadData = new PathDownloadData();
pathDownloadData.pathData = new PathData();
pathDownloadData.pathPara = new PathPara();

pathDownloadData.pathData.pathPoints = new IntPtr();
int size = Marshal.SizeOf(typeof(PathPoint));
pathDownloadData.pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);

//轨迹文件转换为轨迹数据，为指针开辟大小
PathData pathData = new PathData();
pathData.pathPoints = new IntPtr();
pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);
PathPoint[] pathPoints = new PathPoint[10000];

string filePath = "D:/file/path.crpah";
result = CS.cr_path_file2pathData(filePath, ref pathData); //轨迹文件转换为轨迹数据
if (result != CResult.sucess)//判断轨迹文件转换为轨迹数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

for (int points_count = 0; points_count < 10000; points_count++)//轨迹点指针内容转化
{
    IntPtr newPtr = IntPtr.Add(pathData.pathPoints, points_count * size);
    pathPoints[points_count] = (PathPoint)Marshal.PtrToStructure(newPtr, typeof(PathPoint));
    Console.WriteLine($"{points_count}");
}

pathDownloadData.pathData.moveTime = pathData.moveTime;
pathDownloadData.pathData.pathPointsNum = pathData.pathPointsNum;
pathDownloadData.pathPara.index = 1; //轨迹下载参数设置
pathDownloadData.pathPara.moveType = 1;

for (int points_count = 0; points_count < 10000; points_count++)//将新的轨迹点位转回指针
{
    Marshal.StructureToPtr(pathPoints[points_count],
    IntPtr.Add(pathDownloadData.pathData.pathPoints, points_count * size), false);
}

result = CS.cr_path_download(robotHandle, pathDownloadData); //下载轨迹到索引 1
```



```

if (result != CRresult.sucess)//判断下载轨迹是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

Marshal.FreeHGlobal(pathDownloadData.pathData.pathPoints);
Marshal.FreeHGlobal(pathData.pathPoints);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

示例 2：TCP 轨迹复现

- (1) 调用 `cr_path_download` 接口将所有 TCP 位置和关节位置下载到机械臂，运动轨迹类型 `movetype` 写 1，运动时间 `moveTime` 写对应执行周期
- (2) 调用 `cr_path_action` 执行对应索引的轨迹，`runContol` 参数写 1

```

int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PathDownloadData pathDownloadData = new PathDownloadData();
pathDownloadData.pathData = new PathData();
pathDownloadData.pathPara = new PathPara();
pathDownloadData.pathData.pathPoints = new IntPtr();
int size = Marshal.SizeOf(typeof(PathPoint));
pathDownloadData.pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);
//轨迹文件转换为轨迹数据，为指针开辟大小
PathData pathData = new PathData();
pathData.pathPoints = new IntPtr();
pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);
PathPoint[] pathPoints = new PathPoint[10000];
string filePath = "D:/file/path.crpah";
result = CS.cr_path_file2pathData(filePath, ref pathData); //轨迹文件转换为轨迹数据
if (result != CRresult.sucess)//判断轨迹文件转换为轨迹数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
for (int points_count = 0; points_count < 10000; points_count++)//轨迹点指针内容转化
{
    IntPtr newPtr = IntPtr.Add(pathData.pathPoints, points_count * size);

```

```

    pathPoints[points_count] = (PathPoint)Marshal.PtrToStructure(newPtr, typeof(PathPoint));
    Console.WriteLine($"{points_count}");
}

pathDownloadData.pathData.moveTime = 100;
pathDownloadData.pathData.pathPointsNum = pathData.pathPointsNum;
pathDownloadData.pathPara.index = 1; //轨迹下载参数设置
pathDownloadData.pathPara.moveType = 1;
for (int points_count = 0; points_count < 10000; points_count++)//将新的轨迹点位转回指针
{
    Marshal.StructureToPtr(pathPoints[points_count],
        IntPtr.Add(pathDownloadData.pathData.pathPoints, points_count * size), false);
}
result = CS.cr_path_download(robotHandle, pathDownloadData); //下载轨迹到索引 1
if (result != CRresult.sucess)//判断下载轨迹是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
PointControlParaSimple pointControlParaSimple = new PointControlParaSimple();
pointControlParaSimple.pose = new double[6];
pointControlParaSimple.jointpos = new double[6];
pointControlParaSimple.tcpOffset = new double[6];
pointControlParaSimple.coordinatePose = new double[6];
pointControlParaSimple.speed = new double[6];
pointControlParaSimple.acc = new double[6];
for (int i = 0; i < 6; i++)
{
    pointControlParaSimple.pose[i] = pathPoints[0].pose[i];
    pointControlParaSimple.jointpos[i] = pathPoints[0].jointpos[i];
    pointControlParaSimple.tcpOffset[i] = 0;
    pointControlParaSimple.coordinatePose[i] = 0;
    pointControlParaSimple.speed[i] = 30;
    pointControlParaSimple.acc[i] = 30;
}
pointControlParaSimple.coordinateType = CoordinateType.jointCoordinate;
pointControlParaSimple.pointTransType = PointTransType.pointTransStop;
pointControlParaSimple.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
PointControlPara pointControlPara = new PointControlPara();
result = CS.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple, ref
pointControlPara);
// pointControlParaSimple 转化为 pointControlPara
if (result != CRresult.sucess)
    //判断 pointControlParaSimple 转化为 pointControlPara 是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

```

```

}
result = CS.cr_move_joint(robotHandle, pointControlPara, true); //移动至第一个点位
if (result != CResult.sucess) //判断移动至第一个点位是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_path_action(robotHandle, pathDownloadData.pathPara.index, 1);
//运行轨迹索引 1
if (result != CResult.sucess) //判断运行轨迹是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Marshal.FreeHGlobal(pathDownloadData.pathData.pathPoints);
Marshal.FreeHGlobal(pathData.pathPoints);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

示例 3：关节轨迹复现

- (1) 调用 `cr_path_download` 接口将所有关节位置下载到机械臂，运动轨迹类型 `movetype` 写 4，运动时间 `moveTime` 写对应的执行周期
- (2) 调用 `cr_path_action` 执行对应索引的轨迹，`runContol` 参数写 1

```

int robotHandle = -1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PathDownloadData pathDownloadData = new PathDownloadData();
pathDownloadData.pathData = new PathData();
pathDownloadData.pathPara = new PathPara();
pathDownloadData.pathData.pathPoints = new IntPtr();
int size = Marshal.SizeOf(typeof(PathPoint));
pathDownloadData.pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);
//轨迹文件转换为轨迹数据，为指针开辟大小
PathData pathData = new PathData();
pathData.pathPoints = new IntPtr();
pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);
PathPoint[] pathPoints = new PathPoint[10000];
string filePath = "D:/file/path.crpath";
result = CS.cr_path_file2pathData(filePath, ref pathData); //轨迹文件转换为轨迹数据

```

```
if (result != CResult.sucess)//判断轨迹文件转换为轨迹数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
for (int points_count = 0; points_count < 10000; points_count++)//轨迹点指针内容转化
{
    IntPtr newPtr = IntPtr.Add(pathData.pathPoints, points_count * size);
    pathPoints[points_count] = (PathPoint)Marshal.PtrToStructure(newPtr, typeof(PathPoint));
    Console.WriteLine($"{points_count}");
}

pathDownloadData.pathData.moveTime = 100;
pathDownloadData.pathData.pathPointsNum = pathData.pathPointsNum;
pathDownloadData.pathPara.index = 1; //轨迹下载参数设置
pathDownloadData.pathPara.moveType = 4;
for (int points_count = 0; points_count < 10000; points_count++)//将新的轨迹点位转回指针
{
    Marshal.StructureToPtr(pathPoints[points_count],
        IntPtr.Add(pathDownloadData.pathData.pathPoints, points_count * size), false);
}
result = CS.cr_path_download(robotHandle, pathDownloadData); //下载轨迹到索引 1
if (result != CResult.sucess)//判断下载轨迹是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

PointControlParaSimple pointControlParaSimple = new PointControlParaSimple();
pointControlParaSimple.pose = new double[6];
pointControlParaSimple.jointpos = new double[6];
pointControlParaSimple.tcpOffset = new double[6];
pointControlParaSimple.coordinatePose = new double[6];
pointControlParaSimple.speed = new double[6];
pointControlParaSimple.acc = new double[6];
for (int i = 0; i < 6; i++)
{
    pointControlParaSimple.pose[i] = pathPoints[0].pose[i];
    pointControlParaSimple.jointpos[i] = pathPoints[0].jointpos[i];
    pointControlParaSimple.tcpOffset[i] = 0;
    pointControlParaSimple.coordinatePose[i] = 0;
    pointControlParaSimple.speed[i] = 30;
    pointControlParaSimple.acc[i] = 30;
}

pointControlParaSimple.coordinateType = CoordinateType.jointCoordinate;
pointControlParaSimple.pointTransType = PointTransType.pointTransStop;
pointControlParaSimple.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
PointControlPara pointControlPara = new PointControlPara();
```

```

result = CS.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple, ref
pointControlPara);
// pointControlParaSimple 转化为 pointControlPara
if (result != CRresult.sucess)
    //判断 pointControlParaSimple 转化为 pointControlPara 是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_move_joint(robotHandle, pointControlPara, true); //移动至第一个点位
if (result != CRresult.sucess) //判断移动至第一个点位是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_path_action(robotHandle, pathDownloadData.pathPara.index, 1);
//运行轨迹索引 1
if (result != CRresult.sucess) //判断运行轨迹是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Marshal.FreeHGlobal(pathDownloadData.pathData.pathPoints);
Marshal.FreeHGlobal(pathData.pathPoints);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

示例 4：分段 moveX

(1) 调用 `cr_path_download` 接口将部分 TCP 位置和关节位置下载到机械臂，运动轨迹类型 `movetype` 写 3，运动时间 `moveTime` 写 2，速度 `speed` 写 `movex` 的运动速度，加速度 `acc` 写 `movex` 的加速度，交融半径 `blendRadius` 写 `movex` 的交融半径

(2) 调用 `cr_path_action` 执行对应索引的分段 `Movex`，`runControl` 参数写 1

(3) 多次调用 `cr_path_download` 接口写入后续的 TCP 位置和关节位置轨迹，参数和第一步中的参数一致

(4) 调用 `cr_path_action` 关闭分段 `Movex`，`runControl` 参数写 0

```

int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PathDownloadData pathDownloadData = new PathDownloadData();

```

```

pathDownloadData.pathData = new PathData();
pathDownloadData.pathPara = new PathPara();
pathDownloadData.pathData.pathPoints = new IntPtr();
int size = Marshal.SizeOf(typeof(PathPoint));
pathDownloadData.pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);
//轨迹文件转换为轨迹数据，为指针开辟大小
PathData pathData = new PathData();
pathData.pathPoints = new IntPtr();
pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);
PathPoint[] pathPoints = new PathPoint[10000];
string filePath = "D:/file/path.crpath";
result = CS.cr_path_file2pathData(filePath, ref pathData); //轨迹文件转换为轨迹数据
if (result != CRresult.sucess)//判断轨迹文件转换为轨迹数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
for (int points_count = 0; points_count < 10000; points_count++)//轨迹点指针内容转化
{
    IntPtr newPtr = IntPtr.Add(pathData.pathPoints, points_count * size);
    pathPoints[points_count] = (PathPoint)Marshal.PtrToStructure(newPtr, typeof(PathPoint));
    Console.WriteLine($"{points_count}");
}

pathDownloadData.pathData.moveTime = pathData.moveTime;
pathDownloadData.pathData.pathPointsNum = pathData.pathPointsNum;
pathDownloadData.pathPara.index = 1; //轨迹下载参数设置
pathDownloadData.pathPara.moveType = 3; //moveType 为 3 时执行 Movex
pathDownloadData.pathPara.speed = 100; // Movex 速度 单位: mm/s
pathDownloadData.pathPara.acc = 200; // Movex 加速度 单位: mm/s^2
pathDownloadData.pathPara.blendRadius = 50; // Movex 交融半径 单位: mm

for (int points_count = 0; points_count < 10000; points_count++)//将新的轨迹点位转回指针
{
    Marshal.StructureToPtr(pathPoints[points_count],
        IntPtr.Add(pathDownloadData.pathData.pathPoints, points_count * size), false);
}
result = CS.cr_path_download(robotHandle, pathDownloadData); //下载轨迹到索引 1
if (result != CRresult.sucess)//判断下载轨迹是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
PointControlParaSimple pointControlParaSimple = new PointControlParaSimple();
pointControlParaSimple.pose = new double[6];
pointControlParaSimple.jointpos = new double[6];

```

```

pointControlParaSimple.tcpOffset = new double[6];
pointControlParaSimple.coordinatePose = new double[6];
pointControlParaSimple.speed = new double[6];
pointControlParaSimple.acc = new double[6];
for (int i = 0; i < 6; i++)
{
    pointControlParaSimple.pose[i] = pathPoints[0].pose[i];
    pointControlParaSimple.jointpos[i] = pathPoints[0].jointpos[i];
    pointControlParaSimple.tcpOffset[i] = 0;
    pointControlParaSimple.coordinatePose[i] = 0;
    pointControlParaSimple.speed[i] = 30;
    pointControlParaSimple.acc[i] = 30;
}
pointControlParaSimple.coordinateType = CoordinateType.jointCoordinate;
pointControlParaSimple.pointTransType = PointTransType.pointTransStop;
pointControlParaSimple.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
PointControlPara pointControlPara = new PointControlPara();
result = CS.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple, ref
pointControlPara);
// pointControlParaSimple 转化为 pointControlPara
if (result != CRresult.sucess)
    //判断 pointControlParaSimple 转化为 pointControlPara 是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_move_joint(robotHandle, pointControlPara, true); //移动至第一个点位
if (result != CRresult.sucess) //判断移动至第一个点位是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_path_action(robotHandle, pathDownloadData.pathPara.index, 1);
//运行轨迹索引 1
if (result != CRresult.sucess) //判断运行轨迹是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Marshal.FreeHGlobal(pathDownloadData.pathData.pathPoints);
Marshal.FreeHGlobal(pathData.pathPoints);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

3.2.5.4 轨迹控制

函数

CRresult cr_path_action(int robotHandle,int pathIndex,int runControl)

简介

轨迹控制。运行前可通过 cr_path_all_index_get 先判断轨迹索引 pathIndex 是否存在。启动轨迹之前需将机器人移动到轨迹的第一个点位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pathIndex	int	输入	轨迹索引 0~49
runControl	int	输入	0-停止, 1-启动, 2-暂停

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PathData pathData = new PathData();
pathData.pathPoints = new IntPtr();
int size = Marshal.SizeOf(typeof(PathPoint));
pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);
PathPoint[] pathPoints = new PathPoint[10000];

result = CS.cr_path_upload(robotHandle, 4, ref pathData);
//上传索引为 4 的轨迹，使用前需判断索引为 4 的轨迹是否存在
if (result != CRresult.success)//上传轨迹是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

for (int points_count = 0; points_count < 10000; points_count++)
{
    IntPtr newPtr = IntPtr.Add(pathData.pathPoints, points_count * size);
    pathPoints[points_count] = (PathPoint)Marshal.PtrToStructure(newPtr, typeof(PathPoint));
}
```



```
Console.WriteLine($"{points_count}");
}

PointControlParaSimple pointControlParaSimple = new PointControlParaSimple();
//初始点简化运动参数
PointControlPara pointControlPara = new PointControlPara();
//初始点运动参数
pointControlParaSimple.speed = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlParaSimple.acc = new double[6] { 30, 30, 30, 30, 30, 30 };
pointControlParaSimple.pose = new double[6] { pathPoints[0].pose[0],
                                              pathPoints[0].pose[1],
                                              pathPoints[0].pose[2],
                                              pathPoints[0].pose[3],
                                              pathPoints[0].pose[4],
                                              pathPoints[0].pose[5]};

//初始点, 但在 MoveJ 中无影响
pointControlParaSimple.jointpos = new double[6] { pathPoints[0].jointpos[0],
                                                  pathPoints[0].jointpos[1],
                                                  pathPoints[0].jointpos[2],
                                                  pathPoints[0].jointpos[3],
                                                  pathPoints[0].jointpos[4],
                                                  pathPoints[0].jointpos[5]};

//初始点关节角
pointControlParaSimple.tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple.coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 };
pointControlParaSimple.coordinateType = CoordinateType.jointCoordinate;
pointControlParaSimple.pointTransType = PointTransType.pointTransStop;
pointControlParaSimple.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
result = CS.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple, ref
pointControlPara);
//PointControlParaSimple 数据转换为 PointControlPara
if (result != CResult.sucess)
//判断 PointControlParaSimple 数据转换为 PointControlPara 是否成功
{
    Console.WriteLine("PointControlParaSimple 转化失败" + "\r\n");
}

result = CS.cr_move_joint(robotHandle, pointControlPara, true); //移动至第一个点位
if (result != CResult.sucess)//判断移动至第一个点位是否成功
{
    Console.WriteLine("移动至第一个点位失败" + "\r\n");
}
Thread.Sleep(200);
result = CS.cr_path_action(robotHandle, 4, 1);
```

```
//启动轨迹索引为 4 的轨迹，使用前需判断索引为 4 的轨迹是否存在
if (result != CRresult.sucess)//判断轨迹索引为 4 的轨迹是否开启成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Marshal.FreeHGlobal(pathData.pathPoints);

result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.5.5 轨迹数据转换为轨迹文件

函数

CRresult cr_path_pathData2file(**PathData** pathData, string filePath)

简介

轨迹数据转换为轨迹文件。

参数

参数名称	数据类型	输入/输出	描述
pathData	PathData	输入	轨迹数据
filePath	string	输出	文件保存路径

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PathData pathData = new PathData();
pathData.pathPoints = new IntPtr();
int size = Marshal.SizeOf(typeof(PathPoint));
pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);
PathPoint[] pathPoints = new PathPoint[10000];
```

```

result = CS.cr_path_upload(robotHandle, 4, ref pathData);
//上传索引为 4 的轨迹，使用前需判断索引为 4 的轨迹是否存在
if (result != CRresult.sucess)//上传轨迹是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

for (int points_count = 0; points_count < 10000; points_count++)
{
    IntPtr newPtr = IntPtr.Add(pathData.pathPoints, points_count * size);
    pathPoints[points_count] = (PathPoint)Marshal.PtrToStructure(newPtr, typeof(PathPoint));
    Console.WriteLine($"{points_count}");
}

string filePath = "D:/file/path.crpah";
result = CS.cr_path_pathData2file(pathData, filePath);
if (result != CRresult.sucess)//判断轨迹数据转换为轨迹文件是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Marshal.FreeHGlobal(pathData.pathPoints);

result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
    
```

3.2.5.6 轨迹文件转换为轨迹数据

函数

CRresult cr_path_file2pathData(string filePath, ref **PathData** pathData)

简介

轨迹文件转换为轨迹数据。

参数

参数名称	数据类型	输入/输出	描述
filePath	string	输入	文件路径
pathData	ref PathData	输出	轨迹数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PathData pathData = new PathData();
pathData.pathPoints = new IntPtr();
int size = Marshal.SizeOf(typeof(PathPoint));
pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);
PathPoint[] pathPoints = new PathPoint[10000];

string filePath = "D:/file/path.crpah"
result = CS.cr_path_file2pathData(filePath, ref pathData); //轨迹文件转换为轨迹数据
if (result != CRresult.success)//判断轨迹文件转换为轨迹数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

for (int points_count = 0; points_count < 10000; points_count++)//轨迹点指针内容转化
{
    IntPtr newPtr = IntPtr.Add(pathData.pathPoints, points_count * size);
    pathPoints[points_count] = (PathPoint)Marshal.PtrToStructure(newPtr, typeof(PathPoint));
    Console.WriteLine($"{points_count}");
}
Marshal.FreeHGlobal(pathData.pathPoints);

result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.5.7 设置轨迹记录参数

函数

CRresult cr_path_recordPara_set(int robotHandle, RecordPathPara recordPathPara)

简介

设置轨迹记录参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
recordPathPara	RecordPathPara	输入	轨迹记录参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}

RecordPathPara recordPathPara = new RecordPathPara();
recordPathPara.sampleTime = 2;
recordPathPara.recordControl = 1;
result = CS.cr_path_recordPara_set(robotHandle, recordPathPara); //设置轨迹记录参数

if (result != CRresult.sucess)//判断设置轨迹记录参数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.5.8 获取当前轨迹记录状态

函数

```
CRresult cr_path_recordStatus_get(int robotHandle, ref PathRecordStatus pathRecordStatus)
```

简介

获取当前轨迹记录状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pathRecordStatus	ref PathRecordStatus	输出	当前轨迹记录状态

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}

PathRecordStatus pathRecordStatus = new PathRecordStatus();
result = CS.cr_path_recordStatus_get(robotHandle, ref pathRecordStatus);
//获取当前轨迹记录状态

if (result != CResult.success)//判断获取当前轨迹记录状态是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(pathRecordStatus.recordStatus + "\r\n");

result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.5.9 获取当前轨迹运行状态

函数

CResult cr_path_currentRunStatus_get(int robotHandle, ref **PathRunMsg** pathRunMsg)

简介

获取当前轨迹运行状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pathRunMsg	ref PathRunMsg	输出	当前轨迹运行状态

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}

PathRunMsg pathRunMsg = new PathRunMsg();
result = CS.cr_path_currentRunStatus_get(robotHandle, ref pathRunMsg);
//获取当前轨迹运行状态

if (result != CResult.success)//判断获取当前轨迹运行状态是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(pathRunMsg.pathrunstatus + "\r\n");

result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.6 扩展轴运动

3.2.6.1 扩展轴关节移动

函数

CRresult cr_move_joint_exj(int robotHandle, **MoveExjPara** moveExjPara, int isBlock)

简介

扩展轴关节移动。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
moveExjPara	MoveExjPara	输入	扩展轴关节移动相关参数
isBlock	Int	输入	1-阻塞 0-非阻塞

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int isBlock = 1;
MoveExjPara moveExjPara = new MoveExjPara();
moveExjPara.speed = new double[12] { 60, 60, 60, 60, 60, 60, 100, 200, 60, 60, 60, 60 };
moveExjPara.acc = new double[12] { 60, 60, 60, 60, 60, 60, 100, 60, 60, 60, 60, 60 };
moveExjPara.pose = new double[12] { 110, -613, 1129, -57.34, 0, 148.98, 0, 0, 0, 0, 0, 0 };
//前六个代表关节姿态
moveExjPara.jointPos = new double[12] { 60, 60, -60, 60, 60, 60, 300, 200, 0, 0, 0, 0 };
//前六个代表关节 1~6 关节角度，后六个代表扩展轴位置
moveExjPara.tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 };
moveExjPara.coordinateType = CoordinateType.jointCoordinate;
moveExjPara.coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 };
moveExjPara.pointTransRadius = 10;
moveExjPara.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
result = CS.cr_move_joint_exj(robotHandle, moveExjPara, isBlock);//扩展轴关节运动
if (result != CRresult.sucess) //判断扩展轴关节运动是否成功
{
```



```

        Console.WriteLine("接口运行失败" + "\r\n");
    }
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }

```

3.2.6.2 扩展轴直线移动

函数

CRresult cr_move_line_exj(int robotHandle, **MoveExjPara** moveExjPara, int isBlock)

简介

扩展轴直线移动。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
moveExjPara	MoveExjPara	输入	扩展轴直线移动相关参数
isBlock	int	输入	1-阻塞 0-非阻塞

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] jointpos = new double[12] { 60, 60, -60, 60, 60, 60, 100, 0, 0, 0, 0, 0 };
//前六个代表关节 1~6 关节角度
double[] pose = new double[12] { 110, -613, 1129, -57.34, 0, 148.98, 300, 200, 0, 0, 0, 0 };
//前六个代表关节姿态,后六个代表扩展轴位置
int isBlock = 1;
MoveExjPara moveExjPara = new MoveExjPara();
moveExjPara.speed = new double[12];
moveExjPara.acc = new double[12];
moveExjPara.pose = new double[12];

```

```

moveExjPara.jointPos = new double[12];
moveExjPara.tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 };
for (int index = 0; index < 12; index++)
{
    moveExjPara.pose[index] = pose[index];
    moveExjPara.jointPos[index] = jointpos[index];
    moveExjPara.speed[index] = 60;
    moveExjPara.acc[index] = 100;
}
moveExjPara.coordinateType = CoordinateType.PlaneCoordinate;
moveExjPara.coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 };
moveExjPara.pointTransRadius = 10;
moveExjPara.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
moveExjPara.tcpId = -1;
result = CS.cr_move_line_exj(robotHandle, moveExjPara, isBlock); //扩展轴直线运动
if (result != CRresult.sucess) //判断扩展轴直线运动是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess) //判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

3.2.6.3 扩展轴点动运动

函数

CRresult cr_moveJog_exj(int robotHandle, **MoveJogExjPara** moveJogExjPara)

简介

扩展轴点动运动。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
moveJogExjPara	MoveJogExjPara	输入	扩展轴点动运动相关参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
MoveJogExjPara moveJogExjPara = new MoveJogExjPara();
moveJogExjPara.speed = new double[10] { 50, 0, 0, 0, 0, 0, 0, 0, 0, 0 };
moveJogExjPara.acc = new double[10] { 100, 0, 0, 0, 0, 0, 0, 0, 0, 0 };
moveJogExjPara.jointPos = new double[10] { 20, 0, 0, 0, 0, 0, 0, 0, 0, 0 };
result = CS.cr_moveJog_exj(robotHandle, moveJogExjPara); //扩展轴点动运动
if (result != CResult.sucess) //判断扩展轴点动运动是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.2.6.4 扩展轴运动

函数

CResult cr_exj_move(int robotHandle, **ExjMovePara** exjMovePara,int isBlock)

简介

扩展轴运动。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
exjMovePara	ExjMovePara	输入	扩展轴点动运动相关参数
isBlock	int	输入	1-阻塞 0-非阻塞

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
```

```
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
ExjMovePara exjMovePara = new ExjMovePara();
int isBlock = 1;
exjMovePara.exjCount = 1;
exjMovePara.index = new int[10] { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 };
exjMovePara.speed = new double[10] { 50, 0, 0, 0, 0, 0, 0, 0, 0, 0 };
exjMovePara.position = new double[10] { 100, 0, 0, 0, 0, 0, 0, 0, 0, 0 };
exjMovePara.acctime = 100;
exjMovePara.motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc;
result = CS.cr_exj_move(robotHandle, exjMovePara, isBlock);//扩展轴运动
if (result != CRresult.sucess) //判断扩展轴运动是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3 系统属性

3.3.1 硬件信息

3.3.1.1 读取控制柜硬件序列号

函数

CRresult cr_get_productInfo(int robotHandle, byte[] productSn)

简介

读取控制柜硬件序列号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
productSn	byte[]	输出	读取的控制柜硬件序列号

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] productSn = new byte[20];
result = CS.cr_get_productInfo(robotHandle, productSn);
//读取控制柜硬件序列号
if (result != CRresult.sucess)//判断读取控制柜硬件序列号是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string str = Encoding.UTF8.GetString(productSn);
Console.WriteLine(str);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.1.2 读取整臂序列号

函数

CRresult cr_get_productSn(int robotHandle, byte[] sn)

简介

读取整臂序列号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
sn	byte[]	输出	读取整臂序列号

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] Sn = new byte[20];
result = CS.cr_get_productSn(robotHandle, Sn);
//读取整臂序列号
if (result != CResult.sucess)//判断读取整臂序列号是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string str = Encoding.UTF8.GetString(Sn);
Console.WriteLine(str);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.1.3 获取版本信息

函数

CResult cr_get_sysVersion(int robotHandle, ref SysVersion version)

简介

获取版本信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
version	ref SysVersion	输出	返回版本号信息

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
```

```
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SysVersion version = new SysVersion();
result = CS.cr_get_sysVersion(robotHandle, ref version);
//获取版本信息
if (result != CRresult.sucess)//判断获取版本信息是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(version.kzqVersion.versionNo.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.2 本体参数

3.3.2.1 读机械臂 DH 参数

函数

CRresult cr_get_DhParmeter(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 100)] double[] dhPara)

简介

读机械臂 DH 参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
dhPara	double[]	输出	机械臂 DH 参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
```

```

if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] dhPara = new double[100];
result = CS.cr_get_DhParmeter(robotHandle, dhPara);
//读机械臂 DH 参数
if (result != CRresult.sucess)//判断读机械臂 DH 参数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(dhPara[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
    
```

3.3.2.2 读机械臂标准 DH 参数

函数

CRresult cr_get_DhParmeter(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 100)] double[] dhPara)

简介

读机械臂标准 DH 参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
dhPara	double[]	输出	机械臂标准 DH 参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
    
```



```

}
double[] dhPara = new double[100];
result = CS.cr_get_stdDhParmeter(robotHandle, dhPara);
//读机械臂标准 DH 参数
if (result != CRresult.sucess)//判断读机械臂标准 DH 参数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(dhPara[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
    
```

3.3.2.3 读机械臂动力学参数

函数

CRresult cr_get_DynamicParmeter(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 100)] double[] dynamicPara)

简介

读机械臂动力学参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
dynamicPara	double[]	输出	机械臂动力学参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] dynamicPara = new double[100];
result = CS.cr_get_DynamicParmeter(robotHandle, dynamicPara);
    
```

```
//读机械臂动力学参数
if (result != CRresult.sucess)//判断读机械臂动力学参数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(dynamicPara[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.2.4 读机械臂零位补偿值

函数

```
CRresult cr_get_zeroCompensationOffset(int robotHandle,
[MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] zeroCompensationOffset)
```

简介

读机械臂零位补偿值。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
zeroCompensationOffset	double[]	输出	机械臂零位补偿值

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] zeroCompensationOffset = new double[6];
result = CS.cr_get_zeroCompensationOffset(robotHandle, zeroCompensationOffset);
//读机械臂零位补偿值
if (result != CRresult.sucess)//判断读机械臂零位补偿值是否成功
{
```

```

        Console.WriteLine("接口运行失败" + "\r\n");
    }
    Console.WriteLine(zeroCompensationOffset[0].ToString());
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }

```

3.3.2.5 读机械臂减速比

函数

CRresult cr_get_reducerRatio(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] reducerRatio)

简介

读机械臂减速比。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
reducerRatio	double[]	输出	机械臂减速比

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] reducerRatio = new double[6];
result = CS.cr_get_reducerRatio(robotHandle, reducerRatio);
//读机械臂减速比
if (result != CRresult.sucess)//判断读机械臂减速比是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(reducerRatio[0].ToString());
result = CS.cr_destroy_robot(robotHandle);

```

```
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.3 系统信息

3.3.3.1 读机械臂当前状态

函数

CRresult cr_get_robotMode(int robotHandle, ref **RobotModes** robotMode)

简介

读机械臂当前状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
robotMode	ref RobotModes	输出	机械臂当前状态

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
RobotModes robotMode = RobotModes.BackDrive;
result = CS.cr_get_robotMode(robotHandle, ref robotMode);
//读机械臂当前状态
if (result != CRresult.sucess)//判断读机械臂当前状态是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(robotMode.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
```

```
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.3.2 获取日志数量

函数

CRresult cr_cfg_log_number_get(int robotHandle, ref int logNumber)

简介

获取日志数量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
logNumber	ref int	输出	日志个数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int logNumber = 0;
result = CS.cr_cfg_log_number_get(robotHandle, ref logNumber);
if (result != CRresult.sucess)//判断读机械臂获取日志数量是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(logNumber.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.3.3 获取机械臂实时日志

函数

CRresult cr_get_logMsgs(int robotHandle, IntPtr logMsgs, ref int logMsgNumber)

简介

获取机械臂实时日志。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
logMsgs	IntPtr	输出	机械臂日志信息数组
logMsgNumber	ref int	输出	日志个数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int logMsgNumber = 10;
IntPtr ptr = IntPtr.Zero;
int size = Marshal.SizeOf(typeof(RobotLogMsg));
ptr = Marshal.AllocHGlobal(size * 100);
result = CS.cr_get_logMsgs(robotHandle, ptr, ref logMsgNumber);
//获取机械臂日志
if (result != CRresult.success)//判断获取机械臂日志是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
RobotLogMsg[] logMsgs = new RobotLogMsg[logMsgNumber];
for (int i = 0; i < logMsgNumber; i++)
{
    IntPtr ptr_s = ptr + i * size;
    logMsgs[i] = (RobotLogMsg)Marshal.PtrToStructure(ptr_s, typeof(RobotLogMsg));
}
if (logMsgs.Length > 0)
{
    byte[] tmp = new byte[100];
```

```
int indexOfZero = Array.IndexOf(logMsgs[0].textMessage, (byte)0);
Array.Copy(logMsgs[0].textMessage, tmp, indexOfZero == -1 ?
logMsgs[0].textMessage.Length : indexOfZero);

Console.WriteLine(Encoding.UTF8.GetString(tmp));
}
Marshal.FreeHGlobal(ptr);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.3.4 获取机械臂历史所有日志

函数

CRresult cr_sys_history_log_get(int robotHandle, IntPtr logMsgs, ref int logMsgNumber)

简介

获取机械臂历史所有日志。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
logMsgs	IntPtr	输出	机械臂日志信息数组
logMsgNumber	ref int	输出	日志个数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int logMsgNumber = 0;
IntPtr ptr = IntPtr.Zero;
int size = Marshal.SizeOf(typeof(RobotLogMsg));
ptr = Marshal.AllocHGlobal(size * 3000);
result = CS.cr_sys_history_log_get(robotHandle, ptr, ref logMsgNumber);
```

```
//获取机械臂历史所有日志
if (result != CRresult.sucess)//判断获取机械臂历史所有日志是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
RobotLogMsg[] logMsgs = new RobotLogMsg[logMsgNumber];
for (int i = 0; i < logMsgNumber; i++)
{
    IntPtr ptr_s = ptr + i * size;
    logMsgs[i] = (RobotLogMsg)Marshal.PtrToStructure(ptr_s, typeof(RobotLogMsg));
}
if (logMsgs.Length > 0)
{
    byte[] tmp = new byte[100];
    int indexOfZero = Array.IndexOf(logMsgs[0].textMessage, (byte)0);
    Array.Copy(logMsgs[0].textMessage, tmp, indexOfZero == -1 ?
logMsgs[0].textMessage.Length : indexOfZero);

    Console.WriteLine(Encoding.UTF8.GetString(tmp));
}
Marshal.FreeHGlobal(ptr);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.3.5 日志功能是否开启

函数

CRresult cr_log_enable(bool enable)

简介

日志功能是否开启。需要在机器人创建连接之前使用。

参数

参数名称	数据类型	输入/输出	描述
enable	bool	输入	是否开启 true-开启 false-不开启

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
bool enable = true;
CRresult result = CS.cr_log_enable(enable);
//日志功能是否开启
if (result != CRresult.sucess)//判断日志功能是否开启成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
```

3.3.3.6 日志大小设置

函数

CRresult cr_log_set_size(long size, int num)

简介

日志大小设置。需要在机器人创建连接之前使用。

参数

参数名称	数据类型	输入/输出	描述
size	long	输入	单个日志文件大小 字节
num	int	输入	日志文件数量

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
long size = 10 * 1024 * 1024;
int num = 10;
CRresult result = CS.cr_log_set_size(size, num);
//日志大小设置
if (result != CRresult.sucess)//判断日志大小设置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
```

3.3.3.7 读系统时间

函数

CRresult cr_get_SystemDateTime(int robotHandle, byte[] systemDateTime)

简介

读控制柜系统日期时间。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
systemDateTime	byte[]	输出	读取的控制柜系统日期时间

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] systemDateTime = new byte[20];
result = CS.cr_get_SystemDateTime(robotHandle, systemDateTime);
//读系统时间
if (result != CResult.success)//判断读系统时间是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string str = Encoding.UTF8.GetString(systemDateTime);
Console.WriteLine(str);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.3.8 读取机型型号

函数

CResult cr_sys_robotModel_get(int robotHandle, byte[] robotModel,int len)

简介

读取机型型号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
robotModel	byte[]	输出	机械臂型号
len	int	输入	机械臂型号数组长度

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] robotModel = new byte[10];
result = CS.cr_sys_robotModel_get(robotHandle, robotModel, 10); //读机型型号
if (result != CRresult.sucess)//判断读机型型号是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string str = Encoding.UTF8.GetString(robotModel);
Console.WriteLine(str + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.3.9 读取控制器型号

函数

CRresult cr_get_controllerType(int robotHandle, byte[] controllerType)

简介

读取控制器型号，返回码说明如下。

- 1-K20
- 2-MKE
- 3-K10
- 4-DK10

5-MK

6-MK40-H

7-MK300

8-MKCAN 参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
controllerType	byte[]	输出	控制器型号

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] controllerType = new byte[10];
result = CS.cr_get_controllerType(robotHandle, controllerType);//读取控制器型号
if (result != CRresult.sucess)//判断读取控制器型号是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string str = Encoding.UTF8.GetString(controllerType);
Console.WriteLine(str + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.3.10 触发黑匣子数据

函数

CRresult cr_sys_blackBoxData_trigger(int robotHandle)

简介

触发黑匣子数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_sys_blackBoxData_trigger(robotHandle); //触发黑匣子数据
if (result != CRresult.success)//判断触发黑匣子数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.3.11 获取配置改变标志

函数

CRresult cr_cfg_change_code_get(int robotHandle, ref int changeCode)

简介

获取配置改变标志。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
changeCode	ref int	输出	配置改变标志

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int changeCode = 0;
result = CS.cr_cfg_change_code_get(robotHandle, ref changeCode);
if (result != CRresult.sucess)//判断获取配置改变标志是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(changeCode.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.3.12 触发系统更新

函数

CRresult cr_sys_update_trigger(int robotHandle, ref int errCode)

简介

触发系统更新。错误码说明如下：

- 1-机械臂未断电
- 2-安全控制器 ECAT 未处于 OP 状态
- 3-通知安全控制器自更新失败
- 4-控制器写自更新标识失败
- 5-ECAT 网络故障
- 6-当前控制柜 boot 版本不支持软更新

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
errCode	ref int	输出	控制器返回错误码

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int errCode = 0;
result = CS.cr_sys_update_trigger(robotHandle, ref errCode);
if (result != CResult.success)//判断触发系统更新是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.4 系统设置

3.3.4.1 开启运动仿真功能

函数

CResult cr_set_motionSimulation(int robotHandle, bool isopening)

简介

开启运动仿真功能。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
isopening	bool	输入	是否开启

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
bool isopening = true;
result = CS.cr_set_motionSimulation(robotHandle, isopening);
//开启运动仿真功能
if (result != CRresult.sucess)//判断开启运动仿真功能是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.4.2 开启干涉检测功能

函数

CRresult cr_set_motionJudgeInterference(int robotHandle, bool isopening)

简介

开启干涉检测功能。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
isopening	bool	输入	是否开启

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
```



```
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
bool isopening = true;
result = CS.cr_set_motionJudgeInterference(robotHandle, isopening);
//开启干涉检测功能
if (result != CRresult.sucess)//判断开启干涉检测功能是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.4.3 开启动力学计算

函数

CRresult cr_set_motionTorqueCalcon(int robotHandle, bool isopening)

简介

开启动力学计算。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
isopening	bool	输入	是否开启

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
bool isopening = true;
result = CS.cr_set_motionTorqueCalcon(robotHandle, isopening);
```

```
//开启动力学计算
if (result != CRresult.sucess)//判断开启动力学计算是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.3.4.4 开启碰撞检测功能

函数

CRresult cr_set_motionTorqueCollisionDetection(int robotHandle, bool isopening)

简介

开启碰撞检测功能。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
isopening	bool	输入	是否开启

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
bool isopening = true;
result = CS.cr_set_motionTorqueCollisionDetection(robotHandle, isopening);
//开启碰撞检测功能
if (result != CRresult.sucess)//判断开启碰撞检测功能是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
```

```
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4 变量

3.4.1 数字量

3.4.1.1 读控制柜标准数字输出

函数

CResult cr_get_stdDigitalOut(int robotHandle, int index, ref int val)

简介

读控制柜标准数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	索引号（范围：0-7）
val	ref int	输出	标准数字输出值 (0-无效 1-有效)

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int val = 0;
result = CS.cr_get_stdDigitalOut(robotHandle, 0, ref val);
//读控制柜标准数字输出
if (result != CResult.sucess)//判断读控制柜标准数字输出是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
```

```

}
Console.WriteLine("DO_0:" + val + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

3.4.1.2 写控制柜标准数字输出

函数

CRresult cr_set_stdDigitalOut(int robotHandle, int index, int val)

简介

写控制柜标准数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	索引号（范围：0-7）
val	int	输入	标准数字输出值 (0-无效 1-有效)

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int val = 1;
result = CS.cr_set_stdDigitalOut(robotHandle, 0, val);
//写控制柜标准数字输出
if (result != CRresult.sucess)//判断写控制柜标准数字输出是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("DO_0:" + val + "\r\n");

```

```
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.1.3 读控制柜可配置数字输出

函数

CResult cr_get_configDigitalOut(int robotHandle, int index, ref int val)

简介

读控制柜可配置数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	索引号（范围：0-7）
val	ref int	输出	可配置数字输出值 （0-无效 1-有效）

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int val = 0;
result = CS.cr_get_configDigitalOut(robotHandle, 0, ref val);
//读控制柜可配置数字输出
if (result != CResult.sucess)//判断读控制柜可配置数字输出是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("CO_0:" + val + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
```

```
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.1.4 读控制柜所有可配置数字输出

函数

CRresult cr_get_all_configDigitalOut(int robotHandle, int[] arrayVal, int arraySize)

简介

读控制柜所有可配置数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
arrayVal	int[]	输出	待读取的值所存放的位置
arraySize	int	输入	读取的数量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int[] arrayVal = { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 };
result = CS.cr_get_all_configDigitalOut(robotHandle, arrayVal, 5);
if (result != CRresult.sucess)//判断读控制柜所有可配置数字输出是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(string.Join(", ", arrayVal));
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.1.5 写控制柜可配置数字输出

函数

CRresult cr_set_configDigitalOut(int robotHandle, int index, int val)

简介

写控制柜可配置数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	索引号（范围：0-7）
val	int	输入	可配置数字输出值 （0-无效 1-有效）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int val = 1;
result = CS.cr_set_configDigitalOut(robotHandle, 0, val);
//写控制柜可配置数字输出
if (result != CRresult.sucess)//判断写控制柜可配置数字输出是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("CO_0:" + val + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.1.6 读工具端数字输出

函数

CRresult cr_get_toolDigitalOut(int robotHandle, int index, ref int val)

简介

读工具端数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	索引号（范围：0-1）
val	ref int	输出	工具端数字输出值 （0-无效 1-有效）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int val = 0;
result = CS.cr_get_toolDigitalOut(robotHandle, 0, ref val);
//读工具端数字输出
if (result != CRresult.sucess)//判断读工具端数字输出是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("toolDO_0:" + val + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```


3.4.1.7 写工具端数字输出

函数

CRresult cr_set_toolDigitalOut(int robotHandle, int index, int val)

简介

写工具端数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	索引号（范围：0-1）
val	int	输入	工具端数字输出值 （0-无效 1-有效）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int val = 1;
result = CS.cr_set_toolDigitalOut(robotHandle, 0, val);
//写工具端数字输出
if (result != CRresult.sucess)//判断写工具端数字输出是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("toolDO_0:" + val + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.1.8 读控制柜标准数字输入

函数

CRresult cr_get_stdDigitalIn(int robotHandle, int index, ref int val)

简介

读控制柜标准数字输入。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	索引号（范围：0-7）
val	ref int	输出	标准数字输入值 (0-无效 1-有效)

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int val = 0;
result = CS.cr_get_stdDigitalIn(robotHandle, 0, ref val);
//读控制柜标准数字输入
if (result != CRresult.sucess)//判断读控制柜标准数字输入是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("DI_0:" + val + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.1.9 读控制柜可配置数字输入

函数

CRresult cr_get_configDigitalIn(int robotHandle, int index, ref int val)

简介

读控制柜可配置数字输入。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	索引号（范围：0-7）
val	ref int	输出	标准数字输入值 (0-无效 1-有效)

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int val = 0;
result = CS.cr_get_configDigitalIn(robotHandle, 0, ref val);
//读控制柜可配置数字输入
if (result != CRresult.sucess)//判断读控制柜可配置数字输入是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("CI_0:" + val + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.1.10 读控制柜所有可配置数字输入

函数

CRresult cr_get_all_configDigitalIn(int robotHandle, int[] arrayVal, int arraySize)

简介

读控制柜所有可配置数字输入。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
arrayVal	int[]	输出	待读取的值所存放的位置
arraySize	int	输入	读取的数量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int[] arrayVal = { 0, 0, 0, 0, 0 };
result = CS.cr_get_all_configDigitalIn(robotHandle, arrayVal, 5);
if(result != CRresult.sucess)//判断读控制柜所有可配置数字输入是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(arrayVal[0]);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.1.11 读工具端数字输入

函数

CRresult cr_get_toolDigitalIn(int robotHandle, int index, ref int val)

简介

读工具端数字输入。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	索引号（范围：0-1）
val	ref int	输出	工具端数字输入值 （0-无效 1-有效）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int val = 0;
result = CS.cr_get_toolDigitalIn(robotHandle, 0, ref val);
//读工具端数字输入
if (result != CRresult.sucess)//判断读工具端数字输入是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("toolDI_0:" + val + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.2 模拟量

3.4.2.1 读控制柜模拟量输出

函数

CRresult cr_get_stdAnalogOut(int robotHandle, int index, ref double val)

简介

读控制柜模拟量输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	索引号（范围：0）
val	ref double	输出	模拟量输出值

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double val = 0;
result = CS.cr_get_stdAnalogOut(robotHandle, 0, ref val);
//读控制柜模拟量输出
if (result != CRresult.success)//判断读控制柜模拟量输出是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("AO_0:" + val + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.2.2 写控制柜模拟量输出

函数

CRresult cr_set_stdAnalogOut(int robotHandle, int index, double val)

简介

写控制柜模拟量输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	索引号（范围：0）
val	double	输入	模拟量输出值

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double val = 7;
result = CS.cr_set_stdAnalogOut(robotHandle, 0, val);
//写控制柜模拟量输出
if (result != CRresult.sucess)//判断写控制柜模拟量输出是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("AO_0:" + val + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.2.3 读工具端模拟量输入

函数

CRresult cr_get_toolAnalogIn(int robotHandle, int index, ref double val)

简介

读工具端模拟量输入。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

index	int	输入	索引号（范围：0-1）
val	ref double	输出	模拟量输入值

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double val = 0;
result = CS.cr_get_toolAnalogIn(robotHandle, 0, ref val);
//读工具端模拟量输入
if (result != CResult.success)//判断读工具端模拟量输入是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("toolAI_0:" + val + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.2.4 读控制柜模拟量输入

函数

CResult cr_get_stdAnalogIn(int robotHandle, int index, ref double val)

简介

读控制柜模拟量输入。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	索引号（范围：0-2）
val	ref double	输出	模拟量输入值

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double val = 0;
result = CS.cr_get_stdAnalogIn(robotHandle, 0, ref val);
//读控制柜模拟量输入
if (result != CRresult.sucess)//判断读控制柜模拟量输入是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("AI_0:" + val + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.2.5 读机械臂控制柜和工具端上所有输入数字量和模拟量

函数

```
CRresult cr_get_allDAInput(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 8)] int[] controllerDI, [MarshalAs(UnmanagedType.LPArray, SizeConst = 8)] int[] controllerCI, [MarshalAs(UnmanagedType.LPArray, SizeConst = 3)] double[] controllerAI, [MarshalAs(UnmanagedType.LPArray, SizeConst = 2)] int[] toolDI, [MarshalAs(UnmanagedType.LPArray, SizeConst = 2)] double[] toolAI)
```

简介

读机械臂控制柜和工具端上所有输入数字量和模拟量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
controllerDI	int[]	输出	控制柜标准数字输入量
controllerCI	int[]	输出	控制柜可配置数字输入量

controllerAI	double[]	输出	控制柜模拟输入量
toolDI	int[]	输出	工具端标准数字输入量
toolAI	double[]	输出	工具端模拟输入量

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int[] controllerDI = new int[8];
int[] controllerCI = new int[8];
double[] controllerAI = new double[3];
int[] toolDI = new int[2];
double[] toolAI = new double[2];
result = CS.cr_get_allDAInput(robotHandle, controllerDI, controllerCI, controllerAI, toolDI,
toolAI);
//读机械臂控制柜和工具端上所有输入数字量和模拟量
if (result != CResult.success)
//判断读机械臂控制柜和工具端上所有输入数字量和模拟量是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string controldi = "";
string contrci = "";
string contraai = "";
string tooldi = "";
string toolai = "";
for (int i = 0; i < 8; i++)
{
    controldi += controllerDI[i].ToString() + ',';
    contrci += controllerCI[i].ToString() + ',';
}
for (int i = 0; i < 3; i++)
{
    contraai += controllerAI[i].ToString() + ',';
}
for (int i = 0; i < 2; i++)
{

```

```

        tooldi += toolDI[i].ToString() + ',';
        toolai += toolAI[i].ToString() + ',';
    }
    Console.WriteLine("控制柜标准数字输入量 : " + controldi + "\r\n");
    Console.WriteLine("控制柜可配置数字输入量 : " + contrci + "\r\n");
    Console.WriteLine("控制柜模拟输入量 : " + contraai + "\r\n");
    Console.WriteLine("工具端标准数字输入量 : " + tooldi + "\r\n");
    Console.WriteLine("工具端模拟输入量 : " + toolai + "\r\n");
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }
}

```

3.4.2.6 读机械臂控制柜和工具端上所有输出数字量和模拟量

函数

```

CRresult cr_get_allDAOOutput(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 8)] int[] controllerDO, [MarshalAs(UnmanagedType.LPArray, SizeConst = 8)]int[] controllerCO, [MarshalAs(UnmanagedType.LPArray, SizeConst = 1)] double[] controllerAO, [MarshalAs(UnmanagedType.LPArray, SizeConst = 2)] int[] toolDO)

```

简介

读机械臂控制柜和工具端上所有输出数字量和模拟量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
controllerDO	int[]	输出	控制柜标准数字输出量
controllerCO	int[]	输出	控制柜可配置数字输出量
controllerAO	double[]	输出	控制柜模拟输出量
toolDO	int[]	输出	工具端标准数字输出量

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{

```

```
        Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
    }
    int[] controllerDO = new int[8];
    int[] controllerCO = new int[8];
    double[] controllerAO = new double[1];
    int[] toolDO = new int[2];
    result = CS.cr_get_allIDAOutput(robotHandle, controllerDO, controllerCO, controllerAO,
    toolDO);
    //读机械臂控制柜和工具端上所有输出数字量和模拟量
    if (result != CRresult.sucess)
    //判断读机械臂控制柜和工具端上所有输出数字量和模拟量是否成功
    {
        Console.WriteLine("接口运行失败" + "\r\n");
    }
    string controldo = "";
    string contrco = "";
    string contrao = controllerAO[0].ToString();
    string tooldo = "";
    for (int i = 0; i < 8; i++)
    {
        controldo += controllerDO[i].ToString() + ',';
        contrco += controllerCO[i].ToString() + ',';
    }
    for (int i = 0; i < 2; i++)
    {
        tooldo += toolDO[i].ToString() + ',';
    }

    Console.WriteLine("控制柜标准数字输出量 : " + controldo + "\r\n");
    Console.WriteLine("控制柜可配置数字输出量 : " + contrco + "\r\n");
    Console.WriteLine("控制柜模拟输出量 : " + contrao + "\r\n");
    Console.WriteLine("工具端标准数字输出量 : " + tooldo + "\r\n");
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }
}
```

3.4.3 通讯变量

3.4.3.1 读 bool 寄存器

函数

```
CRresult cr_get_boolRegValue(int robotHandle, uint startAddress, uint offsetLength, int[] regData)
```

简介

读 bool 寄存器，单次最多读取 100 个。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
startAddress	uint	输入	寄存器起始地址（范围：0-999）
offsetLength	uint	输入	寄存器偏移地址 （范围：小于或等于 1000 - 起始地址）
regData	int[]	输出	bool 寄存器数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int[] regDat = new int[10];
result = CS.cr_get_boolRegValue(robotHandle, 0, 10, regDat);
//读 bool 寄存器
if (result != CRresult.success)//判断读 bool 寄存器是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(regDat[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.3.2 写 bool 寄存器

函数

```
CRresult cr_set_boolRegValue(int robotHandle, uint startAddress, uint offsetLength, int[] regData)
```

简介

写 bool 寄存器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
startAddress	uint	输入	寄存器起始地址（范围：0-999）
offsetLength	uint	输入	寄存器偏移地址 （范围：小于或等于 1000 - 起始地址）
regData	int[]	输入	bool 寄存器数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int[] regDat = new int[10];
regDat[0] = 1;
regDat[1] = 1;
regDat[2] = 1;
result = CS.cr_set_boolRegValue(robotHandle, 0, 10, regDat);
//写 bool 寄存器
if (result != CRresult.success)//判断写 bool 寄存器是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(regDat[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
```

```
Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.3.3 读 int16 寄存器

函数

```
CRresult cr_get_int16RegValue(int robotHandle, uint startAddress, uint offsetLength, int[]
regData)
```

简介

读 int16 寄存器，单次最多读取 100 个。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
startAddress	uint	输入	寄存器起始地址（范围：0-999）
offsetLength	uint	输入	寄存器偏移地址 （范围：小于或等于 1000 - 起始地址）
regData	int[]	输出	int16 寄存器数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int[] regDat = new int[10];
result = CS.cr_get_int16RegValue(robotHandle, 0, 10, regDat);
//读 int16 寄存器
if (result != CRresult.success)//判断读 int16 寄存器是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(regDat[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
```

```
Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.3.4 写 int16 寄存器

函数

CRresult cr_set_int16RegValue(int robotHandle, uint startAddress, uint offsetLength, int[] regData)

简介

写 int16 寄存器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
startAddress	uint	输入	寄存器起始地址（范围：0-999）
offsetLength	uint	输入	寄存器偏移地址 （范围：小于或等于 1000 - 起始地址）
regData	int[]	输入	int16 寄存器数据（范围：-32768-32767）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int[] regDat = new int[10];
regDat[0] = 1;
regDat[1] = 1;
regDat[2] = 1;
result = CS.cr_set_int16RegValue(robotHandle, 0, 10, regDat);
//写 int16 寄存器
if (result != CRresult.success)//判断写 int16 寄存器是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(regDat[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
```



```
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.3.5 读 int32 寄存器

函数

CRresult cr_get_int32RegValue(int robotHandle, uint startAddress, uint offsetLength, int[] regData)

简介

读 int32 寄存器，单次最多读取 100 个。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
startAddress	uint	输入	寄存器起始地址（范围：0-999）
offsetLength	uint	输入	寄存器偏移地址 （范围：小于或等于 1000 - 起始地址）
regData	int[]	输出	int32 寄存器数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int[] regDat = new int[10];
result = CS.cr_get_int32RegValue(robotHandle, 0, 10, regDat);
//读 int32 寄存器
if (result != CRresult.sucess)//判断读 int32 寄存器是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(regDat[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
```

```
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.3.6 写 int32 寄存器

函数

CRresult cr_set_int32RegValue(int robotHandle, uint startAddress, uint offsetLength, int[] regData)

简介

写 int32 寄存器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
startAddress	uint	输入	寄存器起始地址（范围：0-999）
offsetLength	uint	输入	寄存器偏移地址 （范围：小于或等于 1000 - 起始地址）
regData	int[]	输入	int32 寄存器数据（范围：-2147483648-2147483647）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int[] regDat = new int[10];
regDat[0] = 1;
regDat[1] = 1;
regDat[2] = 1;
result = CS.cr_set_int32RegValue(robotHandle, 0, 10, regDat);
//写 int32 寄存器
if (result != CRresult.sucess)//判断写 int32 寄存器是否成功
{
```

```

        Console.WriteLine("接口运行失败" + "\r\n");
    }
    Console.WriteLine(regDat[0].ToString());
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }

```

3.4.3.7 读 float 寄存器

函数

CRresult cr_get_floatRegValue(int robotHandle, uint startAddress, uint offsetLength, float[] regData)

简介

读 float 寄存器，单次最多读取 100 个。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
startAddress	uint	输入	寄存器起始地址（范围：0-999）
offsetLength	uint	输入	寄存器偏移地址 （范围：小于或等于 1000 - 起始地址）
regData	float[]	输出	float 寄存器数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
float[] regDat = new float[10];
result = CS.cr_get_floatRegValue(robotHandle, 0, 10, regDat);
//读 float 寄存器
if (result != CRresult.sucess)//判断读 float 寄存器是否成功
{

```

```
        Console.WriteLine("接口运行失败" + "\r\n");
    }
    Console.WriteLine(regDat[0].ToString());
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CResult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }
}
```

3.4.3.8 写 float 寄存器

函数

```
CResult cr_set_floatRegValue(int robotHandle, uint startAddress, uint offsetLength, float[]
regData)
```

简介

写 float 寄存器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
startAddress	uint	输入	寄存器起始地址（范围：0-999）
offsetLength	uint	输入	寄存器偏移地址 （范围：小于或等于 1000 - 起始地址）
regData	float[]	输入	float 寄存器数据

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
float[] regDat = new float[10];
regDat[0] = -0.1f;
regDat[1] = 0.1f;
regDat[2] = 0.1f;
result = CS.cr_set_floatRegValue(robotHandle, 0, 10, regDat);
```

```
//写 float 寄存器
if (result != CRresult.sucess)//判断写 float 寄存器是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(regDat[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.4 脚本变量

3.4.4.1 读安装变量

函数

CRresult cr_get_intallVarValue(int robotHandle, string installVarName, IntPtr installVar)

简介

读安装变量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
installVarName	string	输入	需读取安装变量名称
installVar	IntPtr	输出	读取的安装变量值

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
IntPtr ptr = IntPtr.Zero;
int size = Marshal.SizeOf(typeof(VariableMsg));
ptr = Marshal.AllocHGlobal(size);
```

```
string installVarName = "var_1";
result = CS.cr_get_intallVarValue(robotHandle, installVarName, ptr);
//读安装变量
if (result != CRresult.sucess)//判断读安装变量是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
VariableMsg installVar = (VariableMsg)Marshal.PtrToStructure(ptr, typeof(VariableMsg));
Console.WriteLine(installVar.variableName);
Marshal.FreeHGlobal(ptr);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.4.2 修改安装变量

函数

CRresult cr_get_intallVarValue(int robotHandle, string installVarName, IntPtr installVar)

简介

修改安装变量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
installVarName	string	输入	修改的安装变量名称
installVar	IntPtr	输出	修改的安装变量值

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
VariableMsg variableMsg = new VariableMsg();
variableMsg.numberValue = 1;
```

```
variableMsg.variableType = VariableType.LUA_TNUMBER;
variableMsg.variableName = "i_var_2";
string installVarName = "i_var_1";
int size = Marshal.SizeOf(variableMsg);
IntPtr ptr = Marshal.AllocHGlobal(size);
Marshal.StructureToPtr(variableMsg, ptr, true);
result = CS.cr_set_installVarValue(robotHandle, installVarName, ptr);
//修改安装变量
if (result != CRresult.sucess)//判断修改安装变量是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
VariableMsg installVar = (VariableMsg)Marshal.PtrToStructure(ptr, typeof(VariableMsg));
Console.WriteLine(installVar.variableName);
Console.WriteLine(installVar.numberValue);
Marshal.FreeHGlobal(ptr);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.4.2 读取程序变量数据

函数

CRresult cr_script_var_get(int robotHandle, byte[] varName, IntPtr variableMsg)

简介

读取程序变量数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
varName	byte[]	输入	程序变量名
variableMsg	IntPtr	输出	程序变量数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
```

```

if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] varName = new byte[32];
string str = "var_1";
varName = Encoding.Default.GetBytes(str);
IntPtr ptr = IntPtr.Zero;
int size = Marshal.SizeOf(typeof(VariableMsg));
ptr = Marshal.AllocHGlobal(size);
result = CS.cr_script_var_get(robotHandle, varName, ptr);
//读取程序变量数据
if (result != CRresult.sucess)//判断读取程序变量数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
VariableMsg variableMsg = (VariableMsg)Marshal.PtrToStructure(ptr, typeof(VariableMsg));
Console.WriteLine(variableMsg.variableName[0].ToString());
Marshal.FreeHGlobal(ptr);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
    
```

3.4.4.3 修改程序变量数据

函数

CRresult cr_script_var_set(int robotHandle, IntPtr variableMsg)

简介

修改程序变量数据。该接口只在程序运行或暂停时有效。使用该接口时需保证修改的值的类型需要与程序中的变量类型对应。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
variableMsg	IntPtr	输入	程序变量数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败, 查看本地 ip 或者是否已有其他连接" + "\r\n");
}
VariableMsg variableMsg = new VariableMsg();
variableMsg.variableName = "var_1";
variableMsg.variableID = 2;
variableMsg.variableType = VariableType.LUA_TNUMBER;
variableMsg.numberValue = 1;
int size = Marshal.SizeOf(variableMsg);
IntPtr ptr = Marshal.AllocHGlobal(size);
Marshal.StructureToPtr(variableMsg, ptr, true);
result = CS.cr_script_var_set(robotHandle, ptr);
//修改名称为 var_1 的程序变量的数据
if (result != CRresult.sucess)//判断修改程序变量数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("result:" + result + "variableName:" + variableMsg.variableName +
"variableID:" + variableMsg.variableID + "\r\n");
Marshal.FreeHGlobal(ptr);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接, 释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.5 其他 IO

3.4.5.1 获取编码器计数数量

函数

```
CRresult cr_get_encoderTickCnt(int robotHandle, int encoderChannel, ref long
encoderTickCount);
```

简介

获取编码器计数数量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
encoderChannel	int	输入	编码器通道
encoderTickCount	ref long	输出	编码器计数数量

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int encoderChannel = 0;
long encoderTickCount = 0;
result = CS.cr_get_encoderTickCnt(robotHandle, encoderChannel, ref encoderTickCount);
//获取编码器通道 0 的计数数量
if (result != CRresult.sucess)//判断获取编码器计数数量是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(encoderTickCount.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.5.2 设置监控界面的工具输出电压

函数

```
CRresult cr_set_ToolOutputVoltage(int robotHandle, ToolPower toolPower );
```

简介

设置监控界面的工具输出电压。

参数

参数名称	数据类型	输入/输出	描述
------	------	-------	----

robotHandle	int	输入	机械臂实例句柄
toolPower	ToolPower	输入	工具输出电压

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
ToolPower toolPower = ToolPower.Power_on;
result = CS.cr_set_ToolOutputVoltage(robotHandle, toolPower);//设置监控界面的工具输出电压为 24V
if (result != CRresult.sucess)//判断设置监控界面的工具输出电压是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(toolPower.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.4.5.3 读取监控界面的工具输出电压

函数

```
CRresult cr_get_ToolOutputVoltage(int robotHandle, ref ToolPower toolPower );
```

简介

读取监控界面的工具输出电压。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
toolPower	ref ToolPower	输出	工具输出电压

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
ToolPower toolPower = ToolPower.Power_invalid;
result = CS.cr_get_ToolOutputVoltage(robotHandle, ref toolPower);//读取监控界面的工具输出电压
电压
if (result != CRresult.success)//判断读取监控界面的工具输出电压是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(toolPower.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.5 数学模块

3.5.1 位姿变换

3.5.1.1 位姿转换

函数

```
CRresult cr_poseTrans([MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[]
poseFrom, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] poseTrans,
[MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] poseTo)
```

简介

位姿转换。

参数

参数名称	数据类型	输入/输出	描述
poseFrom	double[]	输入	原位姿：x, y, z（单位：mm）； Rx, Ry, Rz（单位：°）
poseTrans	double[]	输入	变换位姿：x, y, z（单位：mm）； Rx, Ry, Rz（单位：°）
poseTo	double[]	输出	结果位姿：x, y, z（单位：mm）； Rx, Ry, Rz（单位：°）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
double[] poseFrom = new double[6];
double[] poseTrans = new double[6];
double[] poseTo = new double[6];
poseFrom[0] = 1;
poseFrom[1] = 2;
poseFrom[2] = 3;
poseTrans[0] = 4;
poseTrans[1] = 4;
poseTrans[2] = 4;
CRresult result = CS.cr_poseTrans(poseFrom, poseTrans, poseTo);
//位姿转换
if (result != CRresult.sucess)//判断位姿转换是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(poseTo[0].ToString());
```

3.5.1.2 位姿求逆

函数

CRresult cr_compute_pose_inv(double[] pose, int poseLen, double[] poseInv, int poseInvLen)

简介

位姿求逆。

参数

参数名称	数据类型	输入/输出	描述
pose	double[]	输入	轴角位姿数据：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）
poseLen	int	输入	位姿的长度，至少为 6

poseInv	double[]	输出	位姿的逆：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）
poseInvLen	int	输入	位姿长度，至少为 6

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
double[] pose = new double[6] { -66, -438, 887, -57, 0, -148 };
double[] poseInv = new double[6];
CRresult result = CS.cr_compute_pose_inv(pose, 6, poseInv, 6);
//位姿求逆
if (result != CRresult.sucess)//判断位姿求逆是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(poseInv[0].ToString());
```

3.5.1.3 轴角转欧拉角

函数

CRresult cr_AxisAngle2Eule([MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] axisAnglePose, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] eulePose)

简介

轴角转欧拉角。

参数

参数名称	数据类型	输入/输出	描述
axisAnglePose	double[]	输入	轴角位姿：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）
eulePose	double[]	输出	欧拉角位姿：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
double[] axisAnglePose = new double[6];
double[] eulePose = new double[6];
axisAnglePose[3] = 86.44;
axisAnglePose[4] = 29.52;
axisAnglePose[5] = 29.52;
CRresult result = CS.cr_AxisAngle2Eule(axisAnglePose, eulePose);
```

```
//轴角转欧拉角
if (result != CRresult.sucess)//判断轴角转欧拉角是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(eulePose[3].ToString());
```

3.5.1.4 欧拉角转轴角

函数

CRresult cr_Eule2AxisAngle([MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] eulePose, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] axisAnglePose)

简介

欧拉角转轴角。

参数

参数名称	数据类型	输入/输出	描述
eulePose	double[]	输入	欧拉角位姿：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）
axisAnglePose	double[]	输出	轴角位姿：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
double[] axisAnglePose = new double[6];
double[] eulePose = new double[6];
eulePose[3] = 90.00;
eulePose[4] = 0;
eulePose[5] = 37.71;
CRresult result = CS.cr_Eule2AxisAngle(eulePose, axisAnglePose);
//欧拉角转轴角
if (result != CRresult.sucess)//判断欧拉角转轴角是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(axisAnglePose[3].ToString());
```

3.5.1.5 TCP 位姿转法兰位姿

函数

```
CRresult cr_TcpToFlangePose([MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] tcpPose, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] toolPose, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] flangePose)
```

简介

TCP 位姿转法兰位姿。

参数

参数名称	数据类型	输入/输出	描述
tcpPose	double[]	输入	tcp 位姿在用户坐标系下的描述: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
toolPose	double[]	输入	当前使用的 tcp: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
flangePose	double[]	输出	法兰在基坐标系下的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
double[] tcpPose = new double[6] { 223, 240, 550, 23, 24, 55 };
double[] toolPose = new double[6] { 22, 44, 55, 66, 77, 88 };
double[] flangePose = new double[6];
CRresult result = CS.cr_TcpToFlangePose(tcpPose, toolPose, flangePose);
//TCP 位姿转法兰位姿
if (result != CRresult.success)//判断 TCP 位姿转法兰位姿是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(flangePose[3].ToString());
```

3.5.1.6 基坐标系下的位姿转到用户坐标系下

函数

```
CRresult cr_compute_pose_base_to_user(double[] basePose, int basePoseLen, double[] userPose, int userPoseLen, double[] poseInUser, int poseInUserLen)
```

简介

基坐标系下的位姿转到用户坐标系下。

参数

参数名称	数据类型	输入/输出	描述
basePose	double[]	输入	轴角位姿数据：x, y, z（单位：mm）； Rx, Ry, Rz（单位：°）
basePoseLen	int	输入	位姿的长度，至少为 6
userPose	double[]	输入	用户坐标系位姿：x, y, z（单位：mm）； Rx, Ry, Rz（单位：°）
userPoseLen	int	输入	位姿的长度，至少为 6
poseInUser	double[]	输出	用户坐标系下的位姿：x, y, z（单位：mm）； Rx, Ry, Rz（单位：°）
poseInUserLen	int	输入	位姿的长度，至少为 6

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
double[] basePose = new double[6] { -66, -438, 887, -57, 0, -148 };
double[] userPose = new double[6] { 0, 0, 0, 0, 0, 0 };
double[] poseInUser = new double[6];
CResult result = CS.cr_compute_pose_base_to_user(basePose, 6, userPose, 6, poseInUser, 6);
//基坐标系下的位姿转到用户坐标系下
if (result != CResult.success)//判断基坐标系下的位姿转到用户坐标系下是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(poseInUser[0].ToString());
```

3.5.1.7 目标点旋转变换

函数

CResult cr_compute_rotate_around_axis(double[] axisPosition, double[] axisDirection, double angle, double[] srcPose, double[] dstPose)

简介

目标点旋转变换。

参数

参数名称	数据类型	输入/输出	描述
axisPosition	double[]	输入	旋转轴位置(x,y,z（单位：mm）)
axisDirection	double[]	输入	旋转轴方向(x,y,z（单位：mm）)
angle	double[]	输入	旋转角度（°）
srcPose	double[]	输入	原始目标位姿(x,y,z（单位：

			mm);Rx,Ry,Rz (单位: °))
dstPose	double[]	输出	旋转后目标位姿(x,y,z (单位: mm);Rx,Ry,Rz (单位: °))

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
double[] axisPosition = new double[3] { 0, 0, 0 };
double[] axisDirection = new double[3] { 0, 0, 100 };
double angle = 180;
double[] srcPose = new double[6] { -66.01, -438.24, 887.68, -57.34, 0, -148.98 };
double[] dstPose = new double[6];
CRresult result = CS.cr_compute_rotate_around_axis(axisPosition, axisDirection, angle, srcPose,
dstPose);//目标点旋转变换
if (result != CRresult.sucess)//判断目标点旋转变换是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(dstPose[0].ToString());
```

3.5.2 运动学

3.5.2.1 正解

函数

CRresult cr_kineForward(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] srcJointPose, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] tarPose)

简介

正解。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
srcJointPose	double[]	输入	关节空间位置: 关节 1~6 (单位: °)
tarPose	double[]	输出	笛卡尔空间位姿计算结果: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)

返回值

数据类型为 CRresult, 若为 sucess 或 0, 则表示成功, 其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] srcJointPose = new double[6] { 60, 60, -60, 60, 60, 60 };
double[] tarPose = new double[6];
result = CS.cr_kineForward(robotHandle, srcJointPose, tarPose);
//正解
if (result != CRresult.sucess)//判断正解是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(tarPose[3].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.5.2.2 逆解

函数

```
CRresult cr_kineInverse(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)]
double[] srcPose, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] refJointPose,
[MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] tarJointPose)
```

简介

逆解。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
srcPose	double[]	输入	笛卡尔空间位姿值：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°）
refJointPose	double[]	输入	逆解计算用的参考关节空间位置：关节 1~6（单位：°）
tarJointPose	double[]	输出	计算成功时关节空间位置计算结果：关节

			1~6 (单位: °)
--	--	--	-------------

返回值

数据类型为 CResult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败, 查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] srcPose = new double[6] { -66.01, -438.24, 887.68, -57.34, 0, -148.98 };
double[] refJointPose = new double[6] { 0, 0, 0, 0, 0, 0 };
double[] tarJointPose = new double[6];
result = CS.cr_kineInverse(robotHandle, srcPose, refJointPose, tarJointPose);
//逆解
if (result != CResult.success)//判断逆解是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(tarJointPose[3].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接, 释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6 配置

3.6.1 安装

3.6.1.1 设置安装角度

函数

CResult cr_cfg_install_angle_set(int robotHandle, **InstallAngle** installAngle)

简介

设置安装角度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
installAngle	InstallAngle	输入	安装角度设置 (基座旋转角度范围: 0-360 基座倾斜角度范围: 0-180)

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
InstallAngle installAngle = new InstallAngle();
installAngle.tiltAngle = 1;
installAngle.baseAngle = 1;
result = CS.cr_cfg_install_angle_set(robotHandle, installAngle);
//设置安装角度
if (result != CRresult.success)//判断设置安装角度是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.1.2 读取安装角度

函数

CRresult cr_cfg_install_angle_get(int robotHandle, ref **InstallAngle** installAngle)

简介

读取安装角度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
installAngle	ref InstallAngle	输出	安装角度

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
InstallAngle installAngle = new InstallAngle();
result = CS.cr_cfg_install_angle_get(robotHandle, ref installAngle);
//读取安装角度
if (result != CResult.sucess)//判断读取安装角度是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
Console.WriteLine(installAngle.baseAngle.ToString());
Console.WriteLine(installAngle.tiltAngle.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.2 TCP

3.6.2.1 读取 TCP 个数

函数

CResult cr_cfg_tcp_count(int robotHandle, ref int count)

简介

读取 TCP 个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
count	ref int	输出	tcp 个数

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int count = -1;
result = CS.cr_cfg_tcp_count(robotHandle, ref count);
//读取 TCP 个数
if (result != CResult.success)//判断读取 TCP 个数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(count.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.2.2 读取 TCP 数据

函数

CResult cr_cfg_tcp_get(int robotHandle, int index, ref **TCPMsg** tcpMsg)

简介

读取 TCP 数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	tcp 索引（范围：0-19）

tcpMsg	TCPMsg	输出	tcp 数据
--------	--------	----	--------

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
TCPMsg tcpMsg = new TCPMsg();
tcpMsg.tcpOffset = new double[6];
tcpMsg.tcpName = new char[20];
result = CS.cr_cfg_tcp_get(robotHandle, 1, ref tcpMsg);
//读取 TCP 数据
if (result != CRresult.sucess)//判断读取 TCP 数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string TcpName = new string(tcpMsg.tcpName);
Console.WriteLine(TcpName);
Console.WriteLine(tcpMsg.tcpOffset[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.2.3 读取 TCP 数据(通过名称)

函数

CRresult cr_cfg_tcp_accord_name_get(int robotHandle, string tcpName, ref **TCPMsg** tcpMsg);

简介

读取 TCP 数据(通过名称)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
tcpName	string	输入	tcp 名称

tcpMsg	ref TCPMsg	输出	tcp 数据
--------	------------	----	--------

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
string tcpName = "TCP_1";
TCPMsg tcpMasg = new TCPMsg();
result = CS.cr_cfg_tcp_accord_name_get(robotHandle, tcpName, ref tcpMasg);
if (result != CRresult.sucess)//判断读取 TCP 数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string TcpName = new string(tcpMasg.tcpName);
Console.WriteLine(TcpName);
Console.WriteLine(tcpMasg.tcpOffset[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine(" 断开连接失败" + "\r\n");
}
```

3.6.2.4 增加 TCP

函数

CRresult cr_cfg_tcp_add(int robotHandle, **TCPMsg** tcpMsg)

简介

增加 TCP。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
tcpMsg	TCPMsg	输入	tcp 数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
TCPMsg tcpMsg = new TCPMsg();
tcpMsg.tcpOffset = new double[6];
tcpMsg.tcpName = new char[20];
tcpMsg.tcpName[0] = 'a';
tcpMsg.tcpName[1] = '2';
tcpMsg.tcpOffset[0] = 5;
tcpMsg.tcpOffset[1] = 7;
result = CS.cr_cfg_tcp_add(robotHandle, tcpMsg);
//增加 TCP
if (result != CRresult.success)//判断增加 TCP 是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string TcpName = new string(tcpMsg.tcpName);
Console.WriteLine(result.ToString());
Console.WriteLine(TcpName);
Console.WriteLine(tcpMsg.tcpOffset[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.2.5 删除 TCP

函数

CRresult cr_cfg_tcp_delete(int robotHandle, int index)

简介

删除 TCP。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	tcp 索引（范围：0-19）

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_cfg_tcp_delete(robotHandle, 1);
//删除 TCP
if (result != CResult.sucess)//判断删除 TCP 是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.2.6 编辑 TCP 数据

函数

CResult cr_cfg_tcp_set(int robotHandle, int index, **TCPMsg** tcpMsg)

简介

编辑 TCP 数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	tcp 索引（范围：0-19）
tcpMsg	TCPMsg	输入	tcp 数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
TCPMsg tcpMsg = new TCPMsg();
tcpMsg.tcpOffset = new double[6];
tcpMsg.tcpName = new char[20];
tcpMsg.tcpName[0] = 'y';
tcpMsg.tcpName[1] = '2';
tcpMsg.tcpOffset[0] = 5;
tcpMsg.tcpOffset[1] = 7;
result = CS.cr_cfg_tcp_set(robotHandle, 1, tcpMsg);
//编辑 TCP 数据
if (result != CRresult.sucess)//判断编辑 TCP 数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string TcpName = new string(tcpMsg.tcpName);
Console.WriteLine(result.ToString());
Console.WriteLine(TcpName);
Console.WriteLine(tcpMsg.tcpOffset[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.2.7 读取当前激活的 TCP 索引

函数

CRresult cr_cfg_tcp_active_get(int robotHandle, ref int index)

简介

读取当前激活的 TCP 索引。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	ref int	输出	tcp 索引 (范围: 0-19)

返回值

数据类型为 CResult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败, 查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int index = -1;
result = CS.cr_cfg_tcp_active_get(robotHandle, ref index);
//读取当前激活的 TCP 索引
if (result != CResult.success)//判断读取当前激活的 TCP 索引是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(index.ToString());
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接, 释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.2.8 激活 TCP

函数

CResult cr_cfg_tcp_active_set(int robotHandle, int index)

简介

激活 TCP。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

index	int	输入	tcp 索引（范围：0-19）
-------	-----	----	-----------------

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_cfg_tcp_active_set(robotHandle, 1);
//激活 TCP
if (result != CRresult.sucess)//判断激活 TCP 是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.2.9 计算 TCP 位置

函数

CRresult cr_compute_tcp_position(double[,] poses, int posesLen, double[] position, int positionLen, ref double err)

简介

计算 TCP 位置。

参数

参数名称	数据类型	输入/输出	描述
poses	double[,]	输入	多组位姿数据，每组位姿数据都是轴角： x, y, z（单位：mm）；Rx, Ry, Rz （单位：°）
posesLen	int	输入	位姿数量，至少为 4
positionLen	int	输入	xyz 的长度，至少为 3

position	double[]	输出	tcp 的 xyz 值: x, y, z (单位: mm)
err	ref double	输出	位置误差

返回值

数据类型为 CResult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败, 查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[,] poses = new double[,] { { -36.10, -4.42, 32.62, 13.54, -91.33, 39.52 },
                                     { -35.98, -11.94, 43.92, 5.07, -94.74, 39.53 },
                                     { -61.76, -16.04, 34.85, 18.68, -80.13, 39.52 },
                                     { -121.26, -33.49, 38.20, 19.91, -51.68, 39.31 } };

double[] position = new double[3];
double err = 0;
result = CS.cr_compute_tcp_position(poses, 4, position, 3, ref err);
//计算 TCP 位置
if (result != CResult.success)//判断计算 TCP 位置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(err.ToString());
for (int i = 0; i < 3; i++)
{
    Console.WriteLine(position[i].ToString());
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接, 释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.2.10 计算 TCP 角度

函数

```
CResult cr_compute_tcp_orientation(double[] csPose, int csLen, double[] pointPose, int
pointLen, double[] orientation, int orientationLen)
```

简介

计算 TCP 角度。

参数

参数名称	数据类型	输入/输出	描述
csPose	double[]	输入	参考坐标系的位姿：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°）
csLen	int	输入	参考坐标系位姿的长度，至少为 6
pointPose	double[]	输入	任意一点的位姿：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°）
pointLen	int	输入	点位姿的长度，至少为 6
orientationLen	int	输入	rxyz 的长度，至少为 3
orientation	double[]	输出	tcp 的 rxyz 值：Rx，Ry，Rz（单位：°）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] csPose = new double[6] { 0, 0, 0, 0, 0, 0 };
double[] pointPose = new double[6] { -36.10, -4.42, 32.62, 13.54, -91.33, 39.52 };
double[] orientation = new double[3];
result = CS.cr_compute_tcp_orientation(csPose, 6, pointPose, 6, orientation, 3);
//计算 TCP 角度
if (result != CResult.success)//判断计算 TCP 角度是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(orientation[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.3 负载

3.6.3.1 读取负载个数

函数

CRresult cr_cfg_payload_count(int robotHandle, ref int count)

简介

读取负载个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
count	ref int	输出	负载个数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int count = -1;
result = CS.cr_cfg_payload_count(robotHandle, ref count);
//读取负载个数
if (result != CRresult.sucess)//判断读取负载个数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(count.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.3.2 读取负载数据

函数

CRresult cr_cfg_payload_get(int robotHandle, int index, ref PayLoad payLoad)

简介

读取负载数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	负载索引（范围：0-19）
payLoad	ref PayLoad	输出	负载信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PayLoad payLoad = new PayLoad();
result = CS.cr_cfg_payload_get(robotHandle, 1, ref payLoad);
//读取负载数据
if (result != CRresult.success)//判断读取负载数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(payLoad.toolPayload.ToString());
Console.WriteLine(payLoad.payloadName[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.3.3 读取负载数据(通过名称)

函数

```
CRresult cr_cfg_payload_accord_name_get(int robotHandle, string payloadName, ref PayLoad
payLoad);
```

简介

读取负载数据(通过名称)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
payloadName	string	输入	负载名称
payLoad	ref PayLoad	输出	负载信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess) //判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
string payloadName = "Payload_1";
PayLoad payload = new PayLoad();
result = CS.cr_cfg_payload_accord_name_get(robotHandle, payloadName, ref payload);
if (result != CRresult.sucess) //判断读取负载数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string PayloadName = new string(payload.payloadName);
Console.WriteLine(payload.toolPayload.ToString());
Console.WriteLine(PayloadName);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess) //判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.3.4 增加负载

函数

```
CRresult cr_cfg_payload_add(int robotHandle, PayLoad payLoad)
```

简介

增加负载。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
payLoad	PayLoad	输入	负载信息

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PayLoad payLoad = new PayLoad();
payLoad.payloadName = new char[20];
payLoad.centerOfGravity = new double[3];
payLoad.payloadName[0] = 'c';
payLoad.payloadName[1] = '2';
payLoad.toolPayload = 5;
payLoad.centerOfGravity[0] = 8;
result = CS.cr_cfg_payload_add(robotHandle, payLoad);
//增加负载
if (result != CResult.success)//判断增加负载是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(payLoad.toolPayload.ToString());
Console.WriteLine(payLoad.payloadName[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.3.5 删除负载

函数

CRresult cr_cfg_payload_delete(int robotHandle, int index)

简介

删除负载。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	负载索引（范围：0-19）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_cfg_payload_delete(robotHandle, 1);
//删除负载
if (result != CRresult.sucess)//判断删除负载是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.3.6 编辑负载

函数

CRresult cr_cfg_payload_set(int robotHandle, int index, **PayLoad** payLoad)

简介

编辑负载。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	负载索引（范围：0-19）
payLoad	PayLoad	输入	负载信息

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PayLoad payLoad = new PayLoad();
payLoad.payloadName = new char[20];
payLoad.centerOfGravity = new double[3];
payLoad.payloadName[0] = 'o';
payLoad.payloadName[1] = '2';
payLoad.toolPayload = 5;
payLoad.centerOfGravity[0] = 8;
result = CS.cr_cfg_payload_set(robotHandle, 2, payLoad);
//编辑负载
if (result != CRresult.sucess)//判断编辑负载是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
Console.WriteLine(payLoad.toolPayload.ToString());
Console.WriteLine(payLoad.payloadName[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.3.7 读取当前激活负载索引

函数

CRresult cr_cfg_payload_active_get(int robotHandle, ref int index)

简介

读取当前激活负载索引。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	ref int	输出	负载索引（范围：0-19）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int index = -1;
result = CS.cr_cfg_payload_active_get(robotHandle, ref index);
//读取当前激活负载索引
if (result != CRresult.sucess)//判断读取当前激活负载索引是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(index.ToString());
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.3.8 激活负载

函数

CRresult cr_cfg_payload_active_set(int robotHandle, int index)

简介

激活负载。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	负载索引（范围：0-19）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_cfg_payload_active_set(robotHandle, 1);
//激活负载
if (result != CRresult.sucess)//判断激活负载是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.4 初始位/包装位

3.6.4.1 设置初始位

函数

`CRresult cr_cfg_home_pose_set(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] homePose, int len)`

简介

设置初始位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
homePose	double[]	输入	初始位姿数据
len	int	输入	数组长度，长度为 6

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] homePose = new double[6];
homePose[0] = 1;
homePose[1] = 1;
result = CS.cr_cfg_home_pose_set(robotHandle, homePose, 6);
//设置初始位
if (result != CRresult.success)//判断设置初始位是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(homePose[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.4.2 读取初始位

函数

```
CRresult cr_cfg_home_pose_get(int robotHandle, [MarshalAs(UnmanagedType.LPArray,
SizeConst = 6)] double[] homePose, int len)
```

简介

读取初始位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
homePose	double[]	输出	初始位姿数据
len	int	输入	数组长度，长度为 6

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] homePose = new double[6];
result = CS.cr_cfg_home_pose_get(robotHandle, homePose, 6);
//读取初始位
if (result != CRresult.success)//判断读取初始位是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(homePose[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.4.3 删除初始位

函数

CRresult cr_cfg_home_pose_delete(int robotHandle)

简介

删除初始位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_cfg_home_pose_delete(robotHandle);
//删除初始位
if (result != CRresult.sucess)//判断删除初始位是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.4.4 读取包装位

函数

CRresult cr_cfg_pack_pose_get(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] packPose, int len)

简介

读取包装位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
packPose	double[]	输出	包装位数据
len	int	输入	数组长度，长度为 6

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] packPose = new double[6];
result = CS.cr_cfg_pack_pose_get(robotHandle, packPose, 6);
//读取包装位
if (result != CRresult.success)//判断读取包装位是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(packPose[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.5 启动

3.6.5.1 设置机器人是否自动上电使能

函数

```
CRresult cr_cfg_poweron_auto_set(int robotHandle, int isAuto)
```

简介

设置机器人是否自动上电使能。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
isAuto	int	输入	是否自动上电 (0-不自动上电 1-自动上电)

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int isAuto = 1;
result = CS.cr_cfg_poweron_auto_set(robotHandle, isAuto);
//设置机器人是否自动上电使能
if (result != CRresult.sucess)//判断设置机器人自动上电使能是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(isAuto.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.5.2 读取机器人自动上电状态

函数

CRresult cr_cfg_poweron_auto_get(int robotHandle, ref int isAuto)

简介

读取机器人自动上电状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
isAuto	ref int	输出	是否自动上电的状态：0-否，1-是

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int isAuto = -1;
result = CS.cr_cfg_poweron_auto_get(robotHandle, ref isAuto);
//读取机器人自动上电状态
if (result != CResult.sucess)//判断读取机器人自动上电状态是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(isAuto.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.6 控制参数

3.6.6.1 设置机器人拖动阻尼

函数

CResult cr_cfg_joint_drag_damping_set(int robotHandle, int jointDargDamping, int arrSize)

简介

设置机器人拖动阻尼。

参数

参数名称	数据类型	输入/输出	描述
------	------	-------	----

robotHandle	int	输入	机械臂实例句柄
jointDargDamping	int	输入	机器人拖动阻尼数组(关节 1-6)
arrSize	int	输入	机器人拖动阻尼数组大小 (一般为 6)

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int[] doubjointDargDamping = new int[6] { 100, 100, 100, 100, 100, 100 };
result = CS.cr_cfg_joint_drag_damping_set(robotHandle, doubjointDargDamping, 6);
//设置机器人拖动阻尼
if (result != CResult.success)//判断设置机器人拖动阻尼是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.6.2 读取机器人拖动阻尼

函数

CResult cr_cfg_joint_drag_damping_get(int robotHandle, ref int jointDargDamping, int arrSize)

简介

读取机器人拖动阻尼。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
jointDargDamping	ref int	输出	机器人拖动阻尼数组(关节 1-6)
arrSize	int	输入	机器人拖动阻尼数组大小

			(一般为 6)
--	--	--	---------

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int[] doubleJointDargDamping = new int[6] { 0, 0, 0, 0, 0, 0 };
result = CS.cr_cfg_joint_drag_damping_set(robotHandle, doubleJointDargDamping, 6);
//读取机器人拖动阻尼
if (result != CResult.success)//判断读取机器人拖动阻尼是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(doubleJointDargDamping[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.6.3 设置转矩异常门限倍率

函数

CResult cr_cfg_torque_threshold_ratio_set(int robotHandle, double[] torqueThresholdRatio, int arrSize);

简介

设置转矩异常门限倍率。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
torqueThresholdRatio	double[]	输入	机器人转矩异常门限倍率数组(关节 1-6)
arrSize	int	输入	机器人转矩异常门限倍率数组大小(一般为 6)

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] torqueThresholdRatio = new double[6] { 0.1, 1, 1, 1, 1, 1 };
result = CS.cr_cfg_torque_threshold_ratio_set(robotHandle, torqueThresholdRatio, 6);
if (result != CRresult.sucess)//判断设置转矩异常门限倍率是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.6.4 读转矩异常门限倍率

函数

CRresult cr_cfg_torque_threshold_ratio_get(int robotHandle, double[] torqueThresholdRatio, int arrSize);

简介

读转矩异常门限倍率。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
torqueThresholdRatio	double[]	输出	机器人转矩异常门限倍率数组(关节 1-6)
arrSize	int	输入	机器人转矩异常门限倍率数组大小(一般为 6)

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] torqueThresholdRatio = new double[6] { 0, 0, 0, 0, 0, 0 };
result = CS.cr_cfg_torque_threshold_ratio_get(robotHandle, torqueThresholdRatio, 6);
if (result != CRresult.sucess)//判断设置转矩异常门限倍率是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(torqueThresholdRatio[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.6.5 设置扩展轴步长及速度倍率

函数

CRresult cr_cfg_exj_move_speed_ratio_set(int robotHandle, double exjMoveSpeedRatio);

简介

设置扩展轴步长及速度倍率。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
exjMoveSpeedRatio	double	输入	扩展轴步长及速度倍率

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
```

```
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double exjMoveSpeedRatio = 1;
result = CS.cr_cfg_exj_move_speed_ratio_set(robotHandle, exjMoveSpeedRatio);
if (result != CRresult.sucess)//判断设置扩展轴步长及速度倍率是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.6.6 读扩展轴步长及速度倍率

函数

`CRresult cr_cfg_exj_move_speed_ratio_get(int robotHandle, ref double exjMoveSpeedRatio);`

简介

读扩展轴步长及速度倍率。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
exjMoveSpeedRatio	ref double	输出	扩展轴步长及速度倍率

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double exjMoveSpeedRatio = new double();
result = CS.cr_cfg_exj_move_speed_ratio_get(robotHandle, ref exjMoveSpeedRatio);
if (result != CRresult.sucess)//判断读扩展轴步长及速度倍率是否成功
```

```
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(exjMoveSpeedRatio.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.7 机器人限值

3.6.7.1 设置限制模式

函数

CRresult cr_cfg_safety_limit_type_set(int robotHandle, **SafetyLimitsValuesType** safetyLimitsValuesType)

简介

设置限制模式。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
safetyLimitsValuesType	SafetyLimitsValuesType	输入	机器人限制等级

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyLimitsValuesType safetyLimitsValuesType = SafetyLimitsValuesType.LimitLevel_3;
result = CS.cr_cfg_safety_limit_type_set(robotHandle, safetyLimitsValuesType);
//设置限制模式
if (result != CRresult.sucess)//判断设置限制模式是否成功
{
```

```

        Console.WriteLine("接口运行失败" + "\r\n");
    }
    Console.WriteLine(result.ToString());
    Console.WriteLine(safetyLimitsValuesType.ToString());
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }

```

3.6.7.2 读取限制模式

函数

CRresult cr_cfg_safety_limit_type_get(int robotHandle, ref **SafetyLimitsValuesType** safetyLimitsValuesType)

简介

读取限制模式。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
safetyLimitsValuesType	ref SafetyLimitsValuesType	输出	限制模式

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyLimitsValuesType safetyLimitsValuesType = SafetyLimitsValuesType.LimitLevel_2;
result = CS.cr_cfg_safety_limit_type_get(robotHandle, ref safetyLimitsValuesType);
//读取限制模式
if (result != CRresult.sucess)//判断读取限制模式是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());

```

```
Console.WriteLine(safetyLimitsValuesType.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.7.3 设置自定义模式下参数

函数

CRresult cr_cfg_safety_limit_para_set(int robotHandle, **SafetyLimitsMode** mode, **SafetyLimitsValues** safetyLimitsValues)

简介

设置自定义模式下参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
mode	SafetyLimitsMode	输入	安全限值模式
safetyLimitsValues	SafetyLimitsValues	输入	模式参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyLimitsValues safetyLimitsValues = new SafetyLimitsValues();
SafetyLimitsMode mode = SafetyLimitsMode.mode_Reduced;
safetyLimitsValues.maxTcpSpeed = 500;
safetyLimitsValues.maxForce = 250;
safetyLimitsValues.maxElbowSpeed = 5000;
safetyLimitsValues.maxElbowForce = 100;
safetyLimitsValues.maxStoppingDistance = 150;
safetyLimitsValues.maxStoppingTime = 1000;
safetyLimitsValues.maxPower = 800;
safetyLimitsValues.maxMomentum = 50;
```

```
result = CS.cr_cfg_safety_limit_para_set(robotHandle, mode, safetyLimitsValues);
//设置自定义模式下参数
if (result != CRresult.sucess)//判断设置自定义模式下参数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.7.4 读取自定义模式下参数

函数

CRresult cr_cfg_safety_limit_para_get(int robotHandle, **SafetyLimitsMode** mode, ref **SafetyLimitsValues** safetyLimitsValues)

简介

读取自定义模式下参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
mode	SafetyLimitsMode	输入	安全限值模式
safetyLimitsValues	ref SafetyLimitsValues	输出	安全限值

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyLimitsValues safetyLimitsValues = new SafetyLimitsValues();
SafetyLimitsMode mode = SafetyLimitsMode.mode_Reduced;
result = CS.cr_cfg_safety_limit_para_get(robotHandle, mode, ref safetyLimitsValues);
```

```
//读取自定义模式下参数
if (result != CRresult.sucess)//判断读取自定义模式下参数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(safetyLimitsValues.maxTcpSpeed.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.7.5 设置碰撞后处理方式

函数

```
CRresult cr_cfg_safety_collihandle_type_set(int robotHandle,SafetyCollisionHandleMode
safetyCollisionHandleMode);
```

简介

设置碰撞后处理方式。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
safetyCollisionHandleMode	SafetyCollisionHandleMode	输入	碰撞后处理方式

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyCollisionHandleMode safetyCollisionHandleMode = new SafetyCollisionHandleMode();
safetyCollisionHandleMode = SafetyCollisionHandleMode.Collision_EnterReboundMode;
result = CS.cr_cfg_safety_collihandle_type_set(robotHandle, safetyCollisionHandleMode);
//将碰撞后处理方式设置为进入反弹模式
if (result != CRresult.sucess)//判断设置碰撞后处理方式是否成功
```



```
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.7.6 读取碰撞后处理方式

函数

```
CResult cr_cfg_safety_collihandle_type_get(int robotHandle, ref SafetyCollisionHandleMode
safetyCollisionHandleMode);
```

简介

读取碰撞后处理方式。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
safetyCollisionHandleMode	ref SafetyCollisionHandleMode	输出	碰撞后处理方式

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyCollisionHandleMode safetyCollisionHandleMode = new SafetyCollisionHandleMode();
result = CS.cr_cfg_safety_collihandle_type_get(robotHandle, ref safetyCollisionHandleMode);
//读取碰撞后处理方式
if (result != CRresult.sucess) //判断读取碰撞后处理方式是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("碰撞后处理方式:" + safetyCollisionHandleMode.ToString() );
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.8 关节限值

3.6.8.1 设置关节限值

函数

[CRresult](#) cr_cfg_safety_joint_limit_set(int robotHandle, [SafetyLimitsMode](#) mode, int jointIndex, [SafetyLimitsJointAngle](#) safetyLimitJoint)

简介

设置关节限值。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
mode	SafetyLimitsMode	输入	关节模式

jointIndex	int	输入	关节序号（范围：0-5）
safetyLimitJoint	SafetyLimitsJointAngle	输入	关节限值

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyLimitsJointAngle safetyLimitJoint = new SafetyLimitsJointAngle();
SafetyLimitsMode mode = SafetyLimitsMode.mode_Reduced;
safetyLimitJoint.maxJointPosition = 200;
safetyLimitJoint.minJointPosition = -200;
safetyLimitJoint.maxJointSpeed = 150;
result = CS.cr_cfg_safety_joint_limit_set(robotHandle, mode, 1, safetyLimitJoint);
//设置关节限值
if (result != CRresult.success)//判断设置关节限值是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(safetyLimitJoint.maxJointPosition.ToString());
Console.WriteLine(safetyLimitJoint.minJointPosition.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.8.2 读取关节限值

函数

CRresult cr_cfg_safety_joint_limit_get(int robotHandle, **SafetyLimitsMode** mode, int jointIndex, ref **SafetyLimitsJointAngle** safetyLimitJoint)

简介

读取关节限值。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
mode	SafetyLimitsMode	输入	关节模式
jointIndex	int	输入	关节序号（范围：0-5）
safetyLimitJoint	ref SafetyLimitsJointAngle	输出	关节限值

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyLimitsJointAngle safetyLimitJoint = new SafetyLimitsJointAngle();
SafetyLimitsMode mode = SafetyLimitsMode.mode_Normal;
result = CS.cr_cfg_safety_joint_limit_get(robotHandle, mode, 3, ref safetyLimitJoint);
//读取关节限值
if (result != CRresult.sucess)//判断读取关节限值是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(safetyLimitJoint.maxJointPosition.ToString());
Console.WriteLine(safetyLimitJoint.maxJointSpeed.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.9 安全平面

3.6.9.1 读取安全平面个数

函数

```
CRresult cr_cfg_safety_plane_count(int robotHandle, ref int count)
```

简介

读取安全平面个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
count	ref int	输出	安全平面数量

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int count = -1;
result = CS.cr_cfg_safety_plane_count(robotHandle, ref count);
//读取安全平面个数
if (result != CResult.success)//判断读取安全平面个数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(count.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.9.2 删除安全平面

函数

CResult cr_cfg_safety_plane_delete(int robotHandle, int index)

简介

删除安全平面。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	安全平面序号（范围：0-7）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_cfg_safety_plane_delete(robotHandle, 5);
//删除安全平面
if (result != CResult.success)//判断删除安全平面是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.9.3 读取安全平面信息

函数

CResult cr_cfg_safety_plane_get(int robotHandle, int index, ref **SafetyLimitsBoundaryPlane** safetyLimitsBoundaryPlane)

简介

读取安全平面信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	安全平面序号

			（范围：0-7）
safetyLimitsBoundaryPlane	SafetyLimitsBoundaryPlane	输出	安全平面信息

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyLimitsBoundaryPlane safetyLimitsBoundaryPlane = new SafetyLimitsBoundaryPlane();
result = CS.cr_cfg_safety_plane_get(robotHandle, 2, ref safetyLimitsBoundaryPlane);
//读取安全平面信息
if (result != CRresult.sucess)//判断读取安全平面信息是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(safetyLimitsBoundaryPlane.id.ToString());
Console.WriteLine(safetyLimitsBoundaryPlane.name[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.9.4 增加安全平面

函数

```
CRresult cr_cfg_safety_plane_add(int robotHandle, SafetyLimitsBoundaryPlane
safetyLimitsBoundaryPlane);
```

简介

增加安全平面。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
safetyLimitsBoundaryPlane	SafetyLimitsBoundaryPlane	输入	安全平面信息

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyLimitsBoundaryPlane safetyLimitsBoundaryPlane = new SafetyLimitsBoundaryPlane();
safetyLimitsBoundaryPlane.name = new char[32];
string name = "asdfv22";
char[] str = name.ToCharArray();
for(int i = 0; i < str.Length; i++)
{
    safetyLimitsBoundaryPlane.name[i] = str[i];
}
safetyLimitsBoundaryPlane.id = 4;
result = CS.cr_cfg_safety_plane_add(robotHandle, safetyLimitsBoundaryPlane);
//增加安全平面
if (result != CRresult.sucess)//判断增加安全平面是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(safetyLimitsBoundaryPlane.id.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.9.5 修改安全平面数据

函数

CRresult cr_cfg_safety_plane_set(int robotHandle, int index, [SafetyLimitsBoundaryPlane](#) safetyLimitsBoundaryPlane)

简介

修改安全平面数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	安全平面序号 (范围: 0-7)
safetyLimitsBoundaryPlane	SafetyLimitsBoundaryPlane	输入	安全平面信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyLimitsBoundaryPlane safetyLimitsBoundaryPlane = new SafetyLimitsBoundaryPlane();
safetyLimitsBoundaryPlane.name = new char[32];
string name = "yyyyyybafv22";
char[] str = name.ToCharArray();
for (int i = 0; i < str.Length; i++)
{
    safetyLimitsBoundaryPlane.name[i] = str[i];
}
safetyLimitsBoundaryPlane.id = 0;
result = CS.cr_cfg_safety_plane_set(robotHandle, 2, safetyLimitsBoundaryPlane);
//修改安全平面数据
if (result != CRresult.success)//判断修改安全平面数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(safetyLimitsBoundaryPlane.id.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.10 安全 I/O

3.6.10.1 设置可配置输入信号

函数

CRresult cr_cfg_safety_io_input_set(int robotHandle, int index, **SafetyIOInput** safetyinput)

简介

设置可配置输入信号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	可配置输入信号序号（范围：0-3）
safetyinput	SafetyIOInput	输入	可配置输入信号

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyIOInput safetyinput = SafetyIOInput.emergencyStop;
result = CS.cr_cfg_safety_io_input_set(robotHandle, 1, safetyinput);
//设置可配置输入信号
if (result != CRresult.sucess)//判断设置可配置输入信号是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(safetyinput.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.10.2 读取可配置输入信号

函数

CRresult cr_cfg_safety_io_input_get(int robotHandle, int index, ref **SafetyIOInput** safetyinput)

简介

读取可配置输入信号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	可配置输入信号序号（范围：0-3）
safetyinput	ref SafetyIOInput	输出	可配置输入信号

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyIOInput safetyinput = SafetyIOInput.emergencyStop;
result = CS.cr_cfg_safety_io_input_get(robotHandle, 1, ref safetyinput);
//读取可配置输入信号
if (result != CRresult.success)//判断读取可配置输入信号是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(safetyinput.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.10.3 设置可配置输出信号

函数

`CRresult cr_cfg_safety_io_output_set(int robotHandle, int index, SafetyIOOutput safetyoutput)`

简介

设置可配置输出信号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	可配置输出信号序号（范围：0-3）
safetyoutput	SafetyIOInput	输入	可配置输出信号

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyIOOutput safetyoutput = SafetyIOOutput.robotMoving;
result = CS.cr_cfg_safety_io_output_set(robotHandle, 2, safetyoutput);
//设置可配置输出信号
if (result != CRresult.success)//判断设置可配置输出信号是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(safetyoutput.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.10.4 读取可配置输出信号

函数

CRresult cr_cfg_safety_io_output_get(int robotHandle, int index, ref **SafetyIOOutput** safetyoutput)

简介

读取可配置输出信号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	可配置输出信号序号（范围：0-3）
safetyoutput	ref SafetyIOInput	输出	可配置输出信号

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SafetyIOOutput safetyoutput = SafetyIOOutput.robotMoving;
result = CS.cr_cfg_safety_io_output_get(robotHandle, 1, ref safetyoutput);
//读取可配置输出信号
if (result != CRresult.success)//判断读取可配置输出信号是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(safetyoutput.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.11 示教器控制

3.6.11.1 设置是否启用示教器

函数

CRresult cr_cfg_safety_tp_use_set(int robotHandle, bool isUse)

简介

设置是否启用示教器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
isUse	bool	输入	是否启用示教器 (0-禁用 1-启用)

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
bool isUse = false;
result = CS.cr_cfg_safety_tp_use_set(robotHandle, isUse);
//设置是否启用示教器
if (result != CRresult.success)//判断设置启用示教器是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(isUse.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.11.2 读取当前是否启用示教器

函数

CRresult cr_cfg_safety_tp_use_get(int robotHandle, ref bool isUse)

简介

读取当前是否启用示教器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
isUse	ref bool	输出	是否启用示教器 (0-禁用 1-启用)

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
bool isUse = true;
result = CS.cr_cfg_safety_tp_use_get(robotHandle, ref isUse);
//读取当前是否启用示教器
if (result != CRresult.sucess)//判断读取当前启用示教器是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(isUse.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12 IO 变量

3.6.12.1 设置数字输入信号配置

函数

`CRresult cr_cfg_var_di_set(int robotHandle, VarDigitalIOType type, int index, IOInputConfiguration inputConfiguration)`

简介

设置数字输入信号配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	VarDigitalIOType	输入	IO 类型
index	int	输入	IO 索引（范围：0-7）
inputConfiguration	IOInputConfiguration	输入	数字量输入配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
IOInputConfiguration inputConfiguration = new IOInputConfiguration();
VarDigitalIOType type = VarDigitalIOType.Digital;
InputAction inputActions = InputAction.InputAction_NONE;
inputConfiguration.inputActions = inputActions;
inputConfiguration.inputFilteringTime = 1;
inputConfiguration.inputName = new char[32];
inputConfiguration.inputName[0] = 'a';
inputConfiguration.inputName[1] = '1';
result = CS.cr_cfg_var_di_set(robotHandle, type, 1, inputConfiguration);
//设置数字输入信号配置
if (result != CRresult.success)//判断设置数字输入信号配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(inputConfiguration.inputFilteringTime.ToString());
result = CS.cr_destroy_robot(robotHandle);
```



```
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.2 读取数字输入信号配置

函数

CRresult cr_cfg_var_di_get(int robotHandle, **VarDigitalIOType** type, int index, ref **IOInputConfiguration** inputConfiguration)

简介

读取数字输入信号配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	VarDigitalIOType	输入	IO 类型
index	int	输入	IO 索引（范围：0-7）
inputConfiguration	ref IOInputConfiguration	输出	数字量输入配置

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
IOInputConfiguration inputConfiguration = new IOInputConfiguration();
VarDigitalIOType type = VarDigitalIOType.Digital;
result = CS.cr_cfg_var_di_get(robotHandle, type, 1, ref inputConfiguration);
//读取数字输入信号配置
if (result != CRresult.sucess)//判断读取数字输入信号配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(inputConfiguration.inputFilteringTime.ToString());
result = CS.cr_destroy_robot(robotHandle);
```

```
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.3 设置数字输出信号配置

函数

CRresult cr_cfg_var_do_set(int robotHandle, **VarDigitalIOType** type, int index, **IOOutputConfiguration** outputConfiguration)

简介

设置数字输出信号配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	VarDigitalIOType	输入	IO 类型
index	int	输入	IO 索引（范围：0-7）
outputConfiguration	IOOutputConfiguration	输入	数字量输出配置

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
VarDigitalIOType type = VarDigitalIOType.Digital;
int index = 1;
IOOutputConfiguration outputConfiguration = new IOOutputConfiguration();
outputConfiguration.outputName = new char[32];
outputConfiguration.outputActions = OutputAction.OutputAction_RobotON_LOW;
string name = "digital";
char[] str = name.ToCharArray();
for (int i = 0; i < str.Length; i++)
{
    outputConfiguration.outputName[i] = str[i];
}
```

```
outputConfiguration.outputOptions = OutputOptions.OutputOptions_Enable;
outputConfiguration.outputRule = OutputRule.OutputRule_Disable;
result = CS.cr_cfg_var_do_set(robotHandle, type, index, outputConfiguration);
//设置索引号为 1 的数字输出信号配置
if (result != CRresult.sucess)//判断设置数字输出信号配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.4 读取数字输出信号配置

函数

`CRresult cr_cfg_var_do_get(int robotHandle, VarDigitalIOType type, int index, ref IOOutputConfiguration outputConfiguration)`

简介

读取数字输出信号配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	VarDigitalIOType	输入	IO 类型
index	int	输入	IO 索引（范围：0-7）
outputConfiguration	ref IOOutputConfiguration	输出	数字量输出配置

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
```

```
IOOutputConfiguration outputConfiguration = new IOOutputConfiguration();
VarDigitalIOType type = VarDigitalIOType.Digital;
result = CS.cr_cfg_var_do_get(robotHandle, type, 1, ref outputConfiguration);
//读取数字输出信号配置
if (result != CRresult.sucess)//判断读取数字输出信号配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
Console.WriteLine(outputConfiguration.outputName[0].ToString());
Console.WriteLine(type.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.5 设置模拟量输入信号类型

函数

`CRresult cr_cfg_var_ai_set(int robotHandle, VarDigitalIOType type, int index, AnalogInputConfiguration analogInputConfiguration)`

简介

设置模拟量输入信号类型。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	VarDigitalIOType	输入	模拟量类型
index	int	输入	模拟量索引 (范围: 0-2)
analogInputConfiguration	AnalogInputConfiguration	输入	模拟量输入配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
AnalogInputConfiguration analogInputConfiguration = new AnalogInputConfiguration();
AnalogType analogInputType = AnalogType.Current;
VarDigitalIOType type = VarDigitalIOType.Analog;
analogInputConfiguration.analogInputType = analogInputType;
analogInputConfiguration.analogInputName = new char[32];
analogInputConfiguration.analogInputName[0] = 'a';
analogInputConfiguration.analogInputName[1] = '1';
result = CS.cr_cfg_var_ai_set(robotHandle, type, 1, analogInputConfiguration);
//设置模拟量输入信号类型
if (result != CRresult.success)//判断设置模拟量输入信号类型是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(analogInputConfiguration.analogInputName[0].ToString());
Console.WriteLine(type.ToString());
```

```
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.6 设置工具 IO 类型

函数

CRresult cr_cfg_var_tool_io_type_set(int robotHandle, int index, **ToolIoType** toolIoType)

简介

设置工具 IO 类型。调用一次或多次该接口后，都需要先调用 **cr_destroy_robot** 接口释放当前句柄，然后再调用 **cr_create_robot** 接口重新创建新句柄。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	IO 索引（4 路 IO 范围：0~3； 10 路 IO 范围：0~9）
toolIoType	ToolIoType	输入	工具 IO 类型

返回值

数据类型为 **CRresult**，若为 **success** 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess) //判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int index = 0;
ToolIoType toolIoType = ToolIoType.IO_OUTPUT;
result = CS.cr_cfg_var_tool_io_type_set(robotHandle, index, toolIoType);
if (result != CRresult.sucess) //判断设置工具 IO 类型是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(toolIoType);
result = CS.cr_destroy_robot(robotHandle);
```

```
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.7 读取工具 IO 类型

函数

CRresult cr_cfg_var_tool_io_type_get(int robotHandle, int index, ref ToolIoType toolIoType)

简介

读取工具 IO 类型。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	IO 索引（4 路 IO 范围：0~3； 10 路 IO 范围：0~9）
toolIoType	ref ToolIoType	输出	工具 IO 类型

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess) //判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int index = 1;
ToolIoType toolIoType = ToolIoType.IO_INPUT;
result = CS.cr_cfg_var_tool_io_type_get(robotHandle, index, ref toolIoType);
if (result != CRresult.sucess) //判断读取工具 IO 类型是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(toolIoType);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
```

```
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.8 读取模拟量输入信号类型

函数

CRresult cr_cfg_var_ai_get(int robotHandle, **VarDigitalIOType** type, int index, ref **AnalogInputConfiguration** analogInputConfiguration)

简介

读取模拟量输入信号类型。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	VarDigitalIOType	输入	模拟量类型
index	int	输入	模拟量索引 (范围: 0-2)
analogInputConfiguration	ref AnalogInputConfiguration	输出	模拟量输入配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
AnalogInputConfiguration analogInputConfiguration = new AnalogInputConfiguration();
VarDigitalIOType type = VarDigitalIOType.Analog;
result = CS.cr_cfg_var_ai_get(robotHandle, type, 1, ref analogInputConfiguration);
//读取模拟量输入信号类型
if (result != CRresult.success)//判断读取模拟量输入信号类型是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(analogInputConfiguration.analogInputName[0].ToString());
Console.WriteLine(type.ToString());
result = CS.cr_destroy_robot(robotHandle);
```



```
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.9 设置模拟量输出信号类型

函数

CRresult cr_cfg_var_ao_set(int robotHandle, int index, [AnalogOutputConfiguration](#) analogOutputConfiguration)

简介

设置模拟量输出信号类型。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	模拟量索引 (范围: 0)
analogOutputConfiguration	AnalogOutputConfiguration	输入	模拟量输出配置

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
AnalogOutputConfiguration analogOutputConfiguration = new AnalogOutputConfiguration();
AnalogType analogOutputType = AnalogType.Voltage;
analogOutputConfiguration.analogOutputType = analogOutputType;
analogOutputConfiguration.analogOutputName = new char[32];
analogOutputConfiguration.analogOutputName[0] = 'a';
analogOutputConfiguration.analogOutputName[1] = '1';
result = CS.cr_cfg_var_ao_set(robotHandle, 0, analogOutputConfiguration);
//设置模拟量输出信号类型
if (result != CRresult.sucess)//判断设置模拟量输出信号类型是否成功
```

```
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(analogOutputConfiguration.analogOutputName[0].ToString());
Console.WriteLine(analogOutputConfiguration.analogOutputType.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.10 读取模拟量输出信号类型

函数

CRresult cr_cfg_var_ao_get(int robotHandle, int index, ref **AnalogOutputConfiguration** analogOutputConfiguration)

简介

读取模拟量输出信号类型。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	模拟量索引 (范围：0)
analogOutputConfiguration	ref AnalogOutputConfiguration	输出	模拟量输出配置

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
AnalogOutputConfiguration analogOutputConfiguration = new AnalogOutputConfiguration();
result = CS.cr_cfg_var_ao_get(robotHandle, 0, ref analogOutputConfiguration);
//读取模拟量输出信号类型
if (result != CRresult.sucess)//判断读取模拟量输出信号类型是否成功
```

```
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(analogOutputConfiguration.analogOutputName[0].ToString());
Console.WriteLine(analogOutputConfiguration.analogOutputType.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.11 设置工具电压输出值

函数

CRresult cr_cfg_var_to_set(int robotHandle, **ToolPower** toolPower)

简介

设置工具电压输出值。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
toolPower	ToolPower	输入	电压值

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
ToolPower toolPower = ToolPower.Power_on;
result = CS.cr_cfg_var_to_set(robotHandle, toolPower);
//设置工具电压输出值
if (result != CRresult.sucess)//判断设置工具电压输出值是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(toolPower.ToString());
```

```
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.12 读取工具电压输出值

函数

CRresult cr_cfg_var_to_get(int robotHandle, ref **ToolPower** toolPower)

简介

读取工具电压输出值。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
toolPower	ref ToolPower	输出	电压值

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
ToolPower toolPower = ToolPower.Power_invalid;
result = CS.cr_cfg_var_to_get(robotHandle, ref toolPower);
//读取工具电压输出值
if (result != CRresult.sucess)//判断读取工具电压输出值是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(toolPower.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
```

```
Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.13 设置编码器配置

函数

CRresult cr_cfg_encoder_config_set(int robotHandle, int index, **EncoderConfiguration** encoderConfig)

简介

设置编码器配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	编码器索引
encoderConfig	EncoderConfiguration	输入	编码器状态

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
EncoderConfiguration encoderConfig = new EncoderConfiguration();
encoderConfig.eqepEnabled = 1;
encoderConfig.strobeInputMode = 0;
result = CS.cr_cfg_encoder_config_set(robotHandle, 0, encoderConfig);
if (result != CRresult.sucess)//判断设置编码器配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.12.14 读取编码器配置

函数

CRresult cr_cfg_encoder_config_get(int robotHandle, int index, ref **EncoderConfiguration** encoderConfig)

简介

读取编码器配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	编码器索引
encoderConfig	ref EncoderConfiguration	输出	编码器状态

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
EncoderConfiguration encoderConfig = new EncoderConfiguration();
result = CS.cr_cfg_encoder_config_get(robotHandle, 0, ref encoderConfig);
if (result != CRresult.sucess)//判断读取编码器配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(encoderConfig.eqepType);
Console.WriteLine(encoderConfig.eqepCountMode);
Console.WriteLine(encoderConfig.strobeInputMode);
Console.WriteLine(encoderConfig.eqepEnabled);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.13 通讯变量

3.6.13.1 设置位寄存器配置

函数

`CRresult cr_cfg_var_bit_reg_set(int robotHandle, int index, BitRegisterConfiguration bitRegisterConfiguration)`

简介

设置位寄存器配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	寄存器索引值 (范围: 0-999)
bitRegisterConfiguration	BitRegisterConfiguration	输入	寄存器配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int index = 1;
BitRegisterConfiguration bitRegisterConfiguration = new BitRegisterConfiguration();
bitRegisterConfiguration.GeneralPurposeBooleanRegisterName = new char[32];
bitRegisterConfiguration.GeneralPurposeBooleanRegisterInputActions =
    InputAction.InputAction_NONE;
bitRegisterConfiguration.GeneralPurposeBooleanRegisterOutputActions =
    OutputAction.OutputAction_RobotON_HIGH;
bitRegisterConfiguration.GeneralPurposeBooleanRegisterOutputRule =
    OutputRule.OutputRule_Disable;
string name = "WEI";
char[] str = name.ToCharArray();
for (int i = 0; i < str.Length; i++)
{
```

```

        bitRegisterConfiguration.GeneralPurposeBooleanRegisterName[i] = str[i];
    }
    result = CS.cr_cfg_var_bit_reg_set(robotHandle, index, bitRegisterConfiguration);
    //设置索引值为 1 的位寄存器配置
    if (result != CRresult.sucess)//判断设置位寄存器配置是否成功
    {
        Console.WriteLine("接口运行失败" + "\r\n");
    }
    Console.WriteLine(result.ToString());
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }
}

```

3.6.13.2 读取位寄存器配置

函数

CRresult cr_cfg_var_bit_reg_get(int robotHandle, int index, ref **BitRegisterConfiguration** bitRegisterConfiguration)

简介

读取位寄存器配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	寄存器索引值 (范围: 0-999)
bitRegisterConfiguration	ref BitRegisterConfiguration	输出	寄存器配置

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}

```



```
BitRegisterConfiguration bitRegisterConfiguration = new BitRegisterConfiguration();
result = CS.cr_cfg_var_bit_reg_get(robotHandle, 1, ref bitRegisterConfiguration);
//读取位寄存器配置
if (result != CRresult.sucess)//判断读取位寄存器配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(bitRegisterConfiguration.GeneralPurposeBOOLeanRegisterName[0].ToString());
Console.WriteLine(bitRegisterConfiguration.GeneralPurposeBOOLeanRegisterOutputActions.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.13.3 读取所有位寄存器配置

函数

CRresult cr_cfg_var_all_bit_reg_get(int robotHandle, BitRegisterConfiguration[] bitRegisterConfigurationArray, int arraySize)

简介

读取所有位寄存器配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
bitRegisterConfigurationArray	BitRegisterConfiguration[]	输出	存放寄存器配置的地址
arraySize	int	输入	读取的数量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
```

```
Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
BitRegisterConfiguration[] bitRegisterConfigurationArray = new BitRegisterConfiguration[100];
int arraySize = 100;
result = CS.cr_cfg_var_all_bit_reg_get(robotHandle, bitRegisterConfigurationArray, arraySize);
if (result != CRresult.sucess)//判断读取所有位寄存器配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.13.4 设置 16 位整数寄存器变量名

函数

CRresult cr_cfg_var_int16_reg_name_set(int robotHandle, int index, byte[] name, int len)

简介

设置 16 位整数寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	寄存器索引值（范围：0-999）
name	byte[]	输入	寄存器名字
len	int	输入	名字长度

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] name = new byte[32];
```

```
string str = "a22";
name = Encoding.Default.GetBytes(str);
result = CS.cr_cfg_var_int16_reg_name_set(robotHandle, 5, name, 6);
//设置 16 位整数寄存器变量名
if (result != CResult.sucess)//判断设置 16 位整数寄存器变量名是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("int16: " + Encoding.UTF8.GetString(name) + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.13.5 读取 16 位整数寄存器变量名

函数

CResult cr_cfg_var_int16_reg_name_get(int robotHandle, int index, byte[] name, int len)

简介

读取 16 位整数寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	寄存器索引值（范围：0-999）
name	byte[]	输出	寄存器名字
len	int	输入	名字长度

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] name = new byte[32];
```

```
result = CS.cr_cfg_var_int16_reg_name_get(robotHandle, 1, name, 6);
//读取 16 位整数寄存器变量名
if (result != CRresult.sucess)//判断读取 16 位整数寄存器变量名是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("int16: " + Encoding.UTF8.GetString(name) + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.13.6 读取所有 16 位整数寄存器变量名

函数

```
CRresult cr_cfg_var_all_int16_reg_name_get(int robotHandle, RegisterConfiguration[]
registerConfigurationArray, int arrSize)
```

简介

读取所有 16 位整数寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
registerConfigurationArray	RegisterConfiguration[]	输出	存放寄存器名字的地址
arrSize	int	输入	读取的名字数量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
RegisterConfiguration[] registerConfigurationArray = new RegisterConfiguration[100];
int arrSize = 100;
result = CS.cr_cfg_var_all_int16_reg_name_get(robotHandle, registerConfigurationArray,
```

```
arrSize);
if (result != CRresult.sucess)//判断读取所有 16 位整数寄存器变量名是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.13.7 设置 32 位整数寄存器变量名

函数

CRresult cr_cfg_var_int32_reg_name_set(int robotHandle, int index, byte[] name, int len)

简介

设置 32 位整数寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	寄存器索引值（范围：0-999）
name	byte[]	输入	寄存器名字
len	int	输入	名字长度

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] name = new byte[32];
string str = "k22";
name = Encoding.Default.GetBytes(str);
result = CS.cr_cfg_var_int32_reg_name_set(robotHandle, 2, name, 6);
//设置 32 位整数寄存器变量名
```

```
if (result != CRresult.sucess)//判断设置 32 位整数寄存器变量名是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("int32: " + Encoding.UTF8.GetString(name) + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.13.8 读取 32 位整数寄存器变量名

函数

CRresult cr_cfg_var_int32_reg_name_get(int robotHandle, int index, byte[] name, int len)

简介

读取 32 位整数寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	寄存器索引值（范围：0-999）
name	byte[]	输出	寄存器名字
len	int	输入	名字长度

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] name = new byte[32];
result = CS.cr_cfg_var_int32_reg_name_get(robotHandle, 1, name, 6);
//读取 32 位整数寄存器变量名
if (result != CRresult.sucess)//判断读取 32 位整数寄存器变量名是否成功
{
```

```
Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("int32: " + Encoding.UTF8.GetString(name) + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.13.9 读取所有 32 位整数寄存器变量名

函数

```
CRresult cr_cfg_var_all_int32_reg_name_get(int robotHandle, RegisterConfiguration[]
registerConfigurationArray, int arrSize)
```

简介

读取所有 32 位整数寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
registerConfigurationArray	RegisterConfiguration[]	输出	存放寄存器名字的地址
arrSize	int	输入	读取的名字数量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
RegisterConfiguration[] registerConfigurationArray = new RegisterConfiguration[100];
int arrSize = 100;
result = CS.cr_cfg_var_all_int32_reg_name_get(robotHandle, registerConfigurationArray,
arrSize);
if (result != CRresult.sucess)//判断读取所有 32 位整数寄存器变量名是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
```

```

    }
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }
}

```

3.6.13.10 设置浮点寄存器变量名

函数

CRresult cr_cfg_var_float_reg_name_set(int robotHandle, int index, byte[] name, int len)

简介

设置浮点寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	寄存器索引值（范围：0-999）
name	byte[]	输入	寄存器名字
len	int	输入	名字长度

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] name = new byte[32];
string str = "asda342";
name = Encoding.Default.GetBytes(str);
result = CS.cr_cfg_var_float_reg_name_set(robotHandle, 2, name, 6);
//设置浮点寄存器变量名
if (result != CRresult.sucess)//判断设置浮点寄存器变量名是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

```



```

}
Console.WriteLine("float: " + Encoding.UTF8.GetString(name) + "\r\n");
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
    
```

3.6.13.11 读取浮点寄存器变量名

函数

CRresult cr_cfg_var_float_reg_name_set(int robotHandle, int index, byte[] name, int len)

简介

读取浮点寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	寄存器索引值（范围：0-999）
name	byte[]	输出	寄存器名字
len	int	输入	名字长度

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] name = new byte[32];
result = CS.cr_cfg_var_float_reg_name_get(robotHandle, 1, name, 6);
//读取浮点寄存器变量名
if (result != CRresult.sucess)//判断读取浮点寄存器变量名是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("float: " + Encoding.UTF8.GetString(name) + "\r\n");
    
```

```
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.13.12 读取所有浮点寄存器变量名

函数

CRresult cr_cfg_var_all_float_reg_name_get(int robotHandle, RegisterConfiguration[] registerConfigurationArray, int arrSize)

简介

读取所有浮点寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
registerConfigurationArray	RegisterConfiguration[]	输出	存放寄存器名字的地址
arrSize	int	输入	读取的名字数量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
RegisterConfiguration[] registerConfigurationArray = new RegisterConfiguration[100];
int arrSize = 100;
result = CS.cr_cfg_var_all_float_reg_name_get(robotHandle, registerConfigurationArray, 100);
if (result != CRresult.sucess)//判断读取所有浮点寄存器变量名是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
```

```
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.14 安装变量

3.6.14.1 读取安装变量个数

函数

CRresult cr_cfg_var_install_count(int robotHandle, ref int count)

简介

读取安装变量个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
count	ref int	输出	安装变量数量

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int count = -1;
result = CS.cr_cfg_var_install_count(robotHandle, ref count);
//读取安装变量个数
if (result != CRresult.sucess)//判断读取安装变量个数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
Console.WriteLine(count.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
```

```
Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.14.2 读取安装变量数据

函数

CRresult cr_cfg_var_install_get(int robotHandle, int index, IntPtr variableMsg)

简介

读取安装变量数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	安装变量索引（范围：0-99）
variableMsg	IntPtr	输出	安装变量数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
IntPtr ptr = IntPtr.Zero;
int size = Marshal.SizeOf(typeof(VariableMsg));
ptr = Marshal.AllocHGlobal(size);
result = CS.cr_cfg_var_install_get(robotHandle, 1, ptr);
//读取安装变量数据
if (result != CRresult.success)//判断读取安装变量数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
VariableMsg variableMsg = (VariableMsg)Marshal.PtrToStructure(ptr, typeof(VariableMsg));
Console.WriteLine(variableMsg.variableName[0].ToString());
Console.WriteLine(variableMsg.variableID.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
}
```

```
Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.14.3 增加安装变量

函数

CRresult cr_cfg_var_install_add(int robotHandle, IntPtr variableMsg)

简介

增加安装变量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
variableMsg	IntPtr	输入	安装变量数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
VariableMsg variableMsg = new VariableMsg();
variableMsg.variableName = "bbbbbb";
variableMsg.variableID = 3;
variableMsg.variableType = VariableType.LUA_TNUMBER;
variableMsg.numberValue = 1;
int size = Marshal.SizeOf(variableMsg);
IntPtr ptr = Marshal.AllocHGlobal(size);
Marshal.StructureToPtr(variableMsg, ptr, true);
result = CS.cr_cfg_var_install_add(robotHandle, ptr);
//增加安装变量
if (result != CRresult.success)//判断增加安装变量是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("cr_set_intallVarValue;" + "result:" + result + "variableName:" +
variableMsg.variableName + "variableID:" + variableMsg.variableID + "\r\n");
Marshal.FreeHGlobal(ptr);
```

```
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.14.4 删除安装变量

函数

CResult cr_cfg_var_install_delete(int robotHandle, int index)

简介

删除安装变量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	安装变量索引（范围：0-99）

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_cfg_var_install_delete(robotHandle, 1);
//删除安装变量
if (result != CResult.sucess)//判断删除安装变量是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

```
}
```

3.6.14.5 修改安装变量

函数

CRresult cr_cfg_var_install_set(int robotHandle, int index, IntPtr variableMsg)

简介

修改安装变量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	安装变量索引（范围：0-99）
variableMsg	IntPtr	输入	安装变量数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
VariableMsg variableMsg = new VariableMsg();
variableMsg.variableName = "yyyyyyyb";
variableMsg.variableID = 2;
variableMsg.variableType = VariableType.LUA_TNUMBER;
variableMsg.numberValue = 1;
int size = Marshal.SizeOf(variableMsg);
IntPtr ptr = Marshal.AllocHGlobal(size);
Marshal.StructureToPtr(variableMsg, ptr, true);
result = CS.cr_cfg_var_install_set(robotHandle, 1, ptr);
//修改安装变量
if (result != CRresult.success)//判断修改安装变量是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("cr_set_intallVarValue;" + "result:" + result + "variableName:" +
variableMsg.variableName + "variableID:" + variableMsg.variableID + "\r\n");
Marshal.FreeHGlobal(ptr);
```

```
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15 坐标系

3.6.15.1 读取点坐标系个数

函数

CResult cr_cfg_cs_point_count(int robotHandle, ref int count)

简介

读取点坐标系个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
count	ref int	输出	点坐标系数量

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int count = -1;
result = CS.cr_cfg_cs_point_count(robotHandle, ref count);
//读取点坐标系个数
if (result != CResult.sucess)//判断读取点坐标系个数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
Console.WriteLine(count.ToString());
result = CS.cr_destroy_robot(robotHandle);
```



```
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.2 读取点坐标系数据

函数

CRresult cr_cfg_cs_point_get(int robotHandle, int index, ref **PointCSNode** pointCSNode)

简介

读取点坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	点坐标系索引（范围：0-9）
pointCSNode	ref PointCSNode	输出	点坐标系数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PointCSNode pointCSNode = new PointCSNode();
result = CS.cr_cfg_cs_point_get(robotHandle, 2, ref pointCSNode);
//读取点坐标系数据
if (result != CRresult.sucess)//判断读取点坐标系数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(pointCSNode.name[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
```

```
Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.3 读取点坐标系数数据(通过名称)

函数

```
CRresult cr_cfg_cs_point_accord_name_get(int robotHandle, string csName, ref PointCSNode
pointCSNode);
```

简介

读取点坐标系数数据(通过名称)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
csName	string	输入	点坐标系名称
pointCSNode	ref PointCSNode	输出	点坐标系数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success) //判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
string pointName = "point_1";
PointCSNode pointCSNode = new PointCSNode();
result = CS.cr_cfg_cs_point_accord_name_get(robotHandle, pointName, ref pointCSNode);
if (result != CRresult.success) //判断读取点坐标系数数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
String PointName = new string(pointCSNode.name);
Console.WriteLine(PointName);
Console.WriteLine(pointCSNode.isValid);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success) //判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

```
}
```

3.6.15.4 增加点坐标系

函数

CRresult cr_cfg_cs_point_add(int robotHandle, **PointCSNode** pointCSNode)

简介

增加点坐标系。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pointCSNode	PointCSNode	输入	点坐标系数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PointCSNode pointcsnode;

pointcsnode.point.toolAxisAngle = new double[6] { 1.8, 2.09, 90, 34.00, -90, 0 };
//第一个点关节角度
pointcsnode.point.toolPosition = new double[6];
result = CS.cr_kineForward(robotHandle, pointcsnode.point.toolAxisAngle,
pointcsnode.point.toolPosition);
//正解求第一个点位姿
if (result != CRresult.success)//判断正解是否成功
{
    Console.WriteLine("第一个点正解失败" + "\r\n");
}
pointcsnode.id = 1;

char[] str = ("point1").ToCharArray();
pointcsnode.name = new char[32];
for (int i = 0; i < str.Length; i++)
{
```

```

        pointcsnode.name[i] = str[i];
    }

    int res = (int)CS.cr_compute_cs_point(pointcsnode.point.toolPosition, 6,
    pointcsnode.point.toolPosition, 6); //计算点坐标系位姿
    if (res == 0) //判断这点能否解算出点坐标系位姿，即坐标系是否有效
    {
        pointcsnode.isValid = 1;
    }
    else
        pointcsnode.isValid = 0;

    result = CS.cr_kineInverse(robotHandle, pointcsnode.point.toolPosition,
    pointcsnode.point.toolAxisAngle, pointcsnode.point.toolAxisAngle);
    //逆解求点坐标系关节角度
    if (result != CRresult.sucess) //判断逆解是否成功
    {
        Console.WriteLine("逆解失败" + "\r\n");
    }

    result = CS.cr_cfg_cs_point_add(robotHandle, pointcsnode);
    //增加点坐标系
    if (result != CRresult.sucess) //判断增加点坐标系是否成功
    {
        Console.WriteLine("接口运行失败" + "\r\n");
    }
    Console.WriteLine("\r\n");
    Console.WriteLine(result.ToString());
    Console.WriteLine("\r\n");
    Console.WriteLine(pointcsnode.name[2].ToString());
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess) //判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }
}

```

3.6.15.5 删除点坐标系

函数

CRresult cr_cfg_cs_point_delete(int robotHandle, int index)

简介

删除点坐标系。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	点坐标系索引（范围：0-9）

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_cfg_cs_point_delete(robotHandle, 1);
//删除点坐标系
if (result != CResult.sucess)//判断删除点坐标系是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.6 修改点坐标系数据

函数

CResult cr_cfg_cs_point_set(int robotHandle, int index, **PointCSNode** pointCSNode)

简介

修改点坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	点坐标系索引（范围：0-9）
pointCSNode	PointCSNode	输入	点坐标系数据

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PointCSNode pointcsnode;

pointcsnode.point.toolAxisAngle = new double[6] { 1.8, 2.09, 90, 34.00, -90, 0 };
//第一个点关节角度
pointcsnode.point.toolPosition = new double[6];
result = CS.cr_kineForward(robotHandle, pointcsnode.point.toolAxisAngle,
pointcsnode.point.toolPosition);
//正解求第一个点位姿
if (result != CResult.success)//判断正解是否成功
{
    Console.WriteLine("第一个点正解失败" + "\r\n");
}
pointcsnode.id = 1;

char[] str = ("point1").ToCharArray();
pointcsnode.name = new char[32];
for (int i = 0; i < str.Length; i++)
{
    pointcsnode.name[i] = str[i];
}

int res = (int)CS.cr_compute_cs_point(pointcsnode.point.toolPosition, 6,
pointcsnode.point.toolPosition, 6); //计算点坐标系位姿
if (res == 0) //判断这点能否解算出点坐标系位姿，即坐标系是否有效
{
    pointcsnode.isValid = 1;
}
else
    pointcsnode.isValid = 0;

result = CS.cr_kineInverse(robotHandle, pointcsnode.point.toolPosition,
pointcsnode.point.toolAxisAngle, pointcsnode.point.toolAxisAngle);
//逆解求点坐标系关节角度
```

```

if (result != CRresult.sucess)//判断逆解是否成功
{
    Console.WriteLine("逆解失败" + "\r\n");
}

result = CS.cr_cfg_cs_point_set(robotHandle, 1, pointcsnode);
//修改点坐标系数数据
if (result != CRresult.sucess)//判断修改点坐标系数数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("\r\n");
Console.WriteLine(result.ToString());
Console.WriteLine("\r\n");
Console.WriteLine(pointcsnode.name[2].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
    
```

3.6.15.7 读取线坐标系个数

函数

CRresult cr_cfg_cs_line_count(int robotHandle, ref int count)

简介

读取线坐标系个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
count	ref int	输出	线坐标系个数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    
```

```

        Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
    }
    int count = -1;
    result = CS.cr_cfg_cs_line_count(robotHandle, ref count);
    //读取线坐标系个数
    if (result != CResult.sucess)//判断读取线坐标系个数是否成功
    {
        Console.WriteLine("接口运行失败" + "\r\n");
    }
    Console.WriteLine(result.ToString());
    Console.WriteLine(count.ToString());
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CResult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }
}

```

3.6.15.8 读取线坐标系数据

函数

CResult cr_cfg_cs_line_get(int robotHandle, int index, ref **LineCSNode** lineCSNode)

简介

读取线坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	线坐标系索引（范围：0-9）
lineCSNode	ref LineCSNode	输出	线坐标系数据

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}

```



```
LineCSNode lineCSNode = new LineCSNode();
result = CS.cr_cfg_cs_line_get(robotHandle, 1, ref lineCSNode);
//读取线坐标系数据
if (result != CRresult.sucess)//判断读取线坐标系数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(lineCSNode.name[0].ToString());
Console.WriteLine(lineCSNode.coordinateJointPos[5].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.9 读取线坐标系数据(通过名称)

函数

```
CRresult cr_cfg_cs_line_accord_name_get(int robotHandle, string csName, ref LineCSNode
lineCSNode);
```

简介

读取线坐标系数据(通过名称)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
csName	string	输入	线坐标系名称
lineCSNode	ref LineCSNode	输出	线坐标系数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess) //判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
string lineName = "line_1";
LineCSNode lineCSNode = new LineCSNode();
```

```
result = CS.cr_cfg_cs_line_accord_name_get(robotHandle, lineName, ref lineCSNode);
if (result != CRresult.sucess)//判断读取线坐标系数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string LineName = new string(lineCSNode.name);
Console.WriteLine(LineName);
Console.WriteLine(lineCSNode.coordinateJointPos[5].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.10 增加线坐标系

函数

CRresult cr_cfg_cs_line_add(int robotHandle, **LineCSNode** lineCSNode)

简介

增加线坐标系。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
lineCSNode	LineCSNode	输入	线坐标系数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
LineCSNode linecsnode;

linecsnode.firstPoint.point.toolAxisAngle = new double[6] { 1.8, 2.09, 90, 34.00, -90, 0 };
//第一个点关节角度
linecsnode.firstPoint.point.toolPosition = new double[6];
```

```
result = CS.cr_kineForward(robotHandle, linecsnode.firstPoint.point.toolAxisAngle,
linecsnode.firstPoint.point.toolPosition);
//正解求第一个点位姿
if (result != CRresult.sucess)//判断正解是否成功
{
    Console.WriteLine("第一个点正解失败" + "\r\n");
}

linecsnode.firstPoint.id = 1;
linecsnode.firstPoint.isValid = 1;
char[] str = ("first").ToCharArray();
linecsnode.firstPoint.name = new char[32];
for (int i = 0; i < str.Length; i++)
{
    linecsnode.firstPoint.name[i] = str[i];
}
linecsnode.secondPoint.point.toolAxisAngle = new double[6] { 5.96, 5.28, 90, 34.67, -90, 4.76 };
//第二个点关节角度
linecsnode.secondPoint.point.toolPosition = new double[6];
result = CS.cr_kineForward(robotHandle, linecsnode.secondPoint.point.toolAxisAngle,
linecsnode.secondPoint.point.toolPosition);
//正解求第二个点位姿
if (result != CRresult.sucess)//判断正解是否成功
{
    Console.WriteLine("第二个点正解失败" + "\r\n");
}

linecsnode.secondPoint.id = 2;
linecsnode.secondPoint.isValid = 1;
char[] str1 = ("second").ToCharArray();
linecsnode.secondPoint.name = new char[32];
for (int i = 0; i < str1.Length; i++)
{
    linecsnode.secondPoint.name[i] = str1[i];
}

linecsnode.id = 2;
char[] str2 = ("line1").ToCharArray();
linecsnode.name = new char[32];
for (int i = 0; i < str2.Length; i++)
{
    linecsnode.name[i] = str2[i];
}
linecsnode.coordinateJointPos = new double[6] { 0, 0, 0, 0, 0, 0 };
linecsnode.coordinatePose = new double[6];
```

```

int res = (int)CS.cr_compute_cs_line(linecsnode.firstPoint.point.toolPosition, 6,
linecsnode.secondPoint.point.toolPosition, 6, linecsnode.coordinatePose, 6);
//计算线坐标系位姿
if (res == 0) //判断这二点能否解算出线坐标系位姿，即坐标系是否有效
{
    linecsnode.isValid = 1;
}
else
    linecsnode.isValid = 0;

result = CS.cr_kineInverse(robotHandle, linecsnode.coordinatePose,
linecsnode.coordinateJointPos, linecsnode.coordinateJointPos);
//逆解求线坐标系关节角度
if (result != CRresult.sucess)//判断逆解是否成功
{
    Console.WriteLine("逆解失败" + "\r\n");
}

result = CS.cr_cfg_cs_line_add(robotHandle, linecsnode);
//增加线坐标系
if (result != CRresult.sucess)//判断增加线坐标系是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("\r\n");
Console.WriteLine(result.ToString());
Console.WriteLine("\r\n");
Console.WriteLine(linecsnode.name[2].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

3.6.15.11 删除线坐标系

函数

CRresult cr_cfg_cs_line_delete(int robotHandle, int index)

简介

删除线坐标系。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	线坐标系索引（范围：0-9）

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_cfg_cs_line_delete(robotHandle, 2);
//删除线坐标系
if (result != CResult.sucess)//判断删除线坐标系是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("\r\n");
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.12 修改线坐标系数据

函数

CResult cr_cfg_cs_line_set(int robotHandle, int index, [LineCSNode](#) lineCSNode)

简介

修改线坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	线坐标系索引（范围：0-9）

lineCSNode	LineCSNode	输入	线坐标系数据
------------	------------	----	--------

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
LineCSNode linecsnode;

linecsnode.firstPoint.point.toolAxisAngle = new double[6] { 1.8, 2.09, 90, 34.00, -90, 0 };
//第一个点关节角度
linecsnode.firstPoint.point.toolPosition = new double[6];
result = CS.cr_kineForward(robotHandle, linecsnode.firstPoint.point.toolAxisAngle,
linecsnode.firstPoint.point.toolPosition);
//正解求第一个点位姿
if (result != CRresult.sucess)//判断正解是否成功
{
    Console.WriteLine("第一个点正解失败" + "\r\n");
}

linecsnode.firstPoint.id = 1;
linecsnode.firstPoint.isValid = 1;
char[] str = ("first").ToCharArray();
linecsnode.firstPoint.name = new char[32];
for (int i = 0; i < str.Length; i++)
{
    linecsnode.firstPoint.name[i] = str[i];
}
linecsnode.secondPoint.point.toolAxisAngle = new double[6] { 5.96, 5.28, 90, 34.67, -90, 4.76 };
//第二个点关节角度
linecsnode.secondPoint.point.toolPosition = new double[6];
result = CS.cr_kineForward(robotHandle, linecsnode.secondPoint.point.toolAxisAngle,
linecsnode.secondPoint.point.toolPosition);
//正解求第二个点位姿
if (result != CRresult.sucess)//判断正解是否成功
{
    Console.WriteLine("第二个点正解失败" + "\r\n");
}
```

```

linecsnode.secondPoint.id = 2;
linecsnode.secondPoint.isValid = 1;
char[] str1 = ("second").ToCharArray();
linecsnode.secondPoint.name = new char[32];
for (int i = 0; i < str1.Length; i++)
{
    linecsnode.secondPoint.name[i] = str1[i];
}

linecsnode.id = 2;
char[] str2 = ("line1").ToCharArray();
linecsnode.name = new char[32];
for (int i = 0; i < str2.Length; i++)
{
    linecsnode.name[i] = str2[i];
}
linecsnode.coordinateJointPos = new double[6] { 0, 0, 0, 0, 0, 0 };
linecsnode.coordinatePose = new double[6];
int res = (int)CS.cr_compute_cs_line(linecsnode.firstPoint.point.toolPosition, 6,
linecsnode.secondPoint.point.toolPosition, 6, linecsnode.coordinatePose, 6);
//计算线坐标系位姿
if (res == 0) //判断这二点能否解算出线坐标系位姿，即坐标系是否有效
{
    linecsnode.isValid = 1;
}
else
    linecsnode.isValid = 0;

result = CS.cr_kineInverse(robotHandle, linecsnode.coordinatePose,
linecsnode.coordinateJointPos, linecsnode.coordinateJointPos);
//逆解求线坐标系关节角度
if (result != CResult.sucess)//判断逆解是否成功
{
    Console.WriteLine("逆解失败" + "\r\n");
}

result = CS.cr_cfg_cs_line_set(robotHandle, 2, linecsnode);
//修改线坐标系
if (result != CResult.sucess)//判断修改线坐标系数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("\r\n");
Console.WriteLine(result.ToString());

```

```
Console.WriteLine("\r\n");
Console.WriteLine(linecsnode.name[1].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.13 读取面坐标系个数

函数

CRresult cr_cfg_cs_plane_count(int robotHandle, ref int count)

简介

读取面坐标系个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
count	ref int	输出	面坐标系数量

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int count = -1;
result = CS.cr_cfg_cs_plane_count(robotHandle, ref count);
//读取面坐标系个数
if (result != CRresult.sucess)//判断读取面坐标系个数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
Console.WriteLine(count.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
```



```
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.14 读取面坐标系数据

函数

CRresult cr_cfg_cs_plane_get(int robotHandle, int index, ref **PlaneCSNode** planeCSNode)

简介

读取面坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	面坐标系索引（范围：0-9）
planeCSNode	ref PlaneCSNode	输出	面坐标系数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PlaneCSNode planeCSNode = new PlaneCSNode();
result = CS.cr_cfg_cs_plane_get(robotHandle, 1, ref planeCSNode);
//读取面坐标系数据
if (result != CRresult.sucess)//判断读取面坐标系数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(planeCSNode.id.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

```
}
```

3.6.15.15 读取面坐标系数据(通过名称)

函数

```
CRresult cr_cfg_cs_plane_accord_name_get(int robotHandle, string csName, ref PlaneCSNode planeCSNode);
```

简介

读取面坐标系数据(通过名称)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
csName	string	输入	面坐标系名称
planeCSNode	ref PlaneCSNode	输出	面坐标系数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess) //判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
string planeName = "plane_1";
PlaneCSNode planeCSNode = new PlaneCSNode();
result = CS.cr_cfg_cs_plane_accord_name_get(robotHandle, planeName, ref planeCSNode);
if (result != CRresult.sucess) //判断读取线坐标系数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
string PlaneName = new string(planeCSNode.name);
Console.WriteLine(PlaneName);
Console.WriteLine(planeCSNode.isValid);
result = CS.cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
if (result != CRresult.sucess) //判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.16 增加面坐标系

函数

CRresult cr_cfg_cs_plane_add(int robotHandle, **PlaneCSNode** planeCSNode)

简介

增加面坐标系。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
planeCSNode	PlaneCSNode	输入	面坐标系数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PlaneCSNode planeCSNode;

planeCSNode.firstPoint.point.toolAxisAngle = new double[6] { 1.8, 2.09, 90, 34.00, -90, 0 };
//第一个点关节角度
planeCSNode.firstPoint.point.toolPosition = new double[6];
result = CS.cr_kineForward(robotHandle, planeCSNode.firstPoint.point.toolAxisAngle,
planeCSNode.firstPoint.point.toolPosition);
//正解求第一个点位姿
if (result != CRresult.sucess)//判断正解是否成功
{
    Console.WriteLine("第一个点正解失败" + "\r\n");
}

planeCSNode.firstPoint.id = 1;
planeCSNode.firstPoint.isValid = 1;
char[] str = ("first").ToCharArray();
planeCSNode.firstPoint.name = new char[32];
for (int i = 0; i < str.Length; i++)
{
    planeCSNode.firstPoint.name[i] = str[i];
}
```

```

}
planecsnode.secondPoint.point.toolAxisAngle = new double[6] { 5.96, 5.28, 90, 34.67, -90,
4.76 };
//第二个点关节角度
planecsnode.secondPoint.point.toolPosition = new double[6];
resut = CS.cr_kineForward(robotHandle, planecsnode.secondPoint.point.toolAxisAngle,
planecsnode.secondPoint.point.toolPosition);
//正解求第二个点位姿
if (result != CRresult.sucess)//判断正解是否成功
{
    Console.WriteLine("第二个点正解失败" + "\r\n");
}

planecsnode.secondPoint.id = 2;
planecsnode.secondPoint.isValid = 1;
char[] str1 = ("second").ToCharArray();
planecsnode.secondPoint.name = new char[32];
for (int i = 0; i < str1.Length; i++)
{
    planecsnode.secondPoint.name[i] = str1[i];
}
planecsnode.thirdPoint.point.toolAxisAngle = new double[6] { 7.80, 7.30, 86.45, 34.67, -92.46,
6.22 };
//第三个点关节角度
planecsnode.thirdPoint.point.toolPosition = new double[6];
result = CS.cr_kineForward(robotHandle, planecsnode.thirdPoint.point.toolAxisAngle,
planecsnode.thirdPoint.point.toolPosition);
//正解求第三个点位姿
if (result != CRresult.sucess)//判断正解是否成功
{
    Console.WriteLine("第三个点正解失败" + "\r\n");
}

planecsnode.thirdPoint.id = 3;
planecsnode.thirdPoint.isValid = 1;
char[] str2 = ("third").ToCharArray();
planecsnode.thirdPoint.name = new char[32];
for (int i = 0; i < str2.Length; i++)
{
    planecsnode.thirdPoint.name[i] = str2[i];
}
planecsnode.id = 2;
char[] str3 = ("plane1").ToCharArray();
planecsnode.name = new char[32];
for (int i = 0; i < str3.Length; i++)

```

```

{
    planeconsnode.name[i] = str3[i];
}
planeconsnode.coordinateJointPos = new double[6] { 0, 0, 0, 0, 0, 0 };
planeconsnode.coordinatePose = new double[6];
int res = (int)CS.cr_compute_cs_plane(planeconsnode.firstPoint.point.toolPosition, 6,
planeconsnode.secondPoint.point.toolPosition, 6, planeconsnode.thirdPoint.point.toolPosition, 6,
planeconsnode.coordinatePose, 6); //计算面坐标系位姿
if (res == 0) //判断这三点能否解算出面坐标系位姿，即坐标系是否有效
{
    planeconsnode.isValid = 1;
}
else
    planeconsnode.isValid = 0;

result = CS.cr_kineInverse(robotHandle, planeconsnode.coordinatePose,
planeconsnode.coordinateJointPos, planeconsnode.coordinateJointPos);
//逆解求面坐标系关节角度
if (result != CRresult.sucess)//判断逆解是否成功
{
    Console.WriteLine("逆解失败" + "\r\n");
}

result = CS.cr_cfg_cs_plane_add(robotHandle, planeconsnode);
//增加面坐标系
if (result != CRresult.sucess)//判断增加面坐标系是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("\r\n");
Console.WriteLine(result.ToString());
Console.WriteLine("\r\n");
Console.WriteLine(planeconsnode.name[1].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

3.6.15.17 删除面坐标系

函数

```
CRresult cr_cfg_cs_plane_delete(int robotHandle, int index)
```

简介

删除面坐标系。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	面坐标系索引（范围：0-9）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_cfg_cs_plane_delete(robotHandle, 1);
//删除面坐标系
if (result != CResult.success)//判断删除面坐标系是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("\r\n");
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.18 修改面坐标系数据

函数

CResult cr_cfg_cs_plane_set(int robotHandle, int index, **PlaneCSNode** planeCSNode)

简介

修改面坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	面坐标系索引（范围：0-9）
planeCSNode	PlaneCSNode	输入	面坐标系数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PlaneCSNode planeCSNode;

planeCSNode.firstPoint.point.toolAxisAngle = new double[6] { 1.8, 2.09, 90, 34.00, -90, 0 };
//第一个点关节角度
planeCSNode.firstPoint.point.toolPosition = new double[6];
result = CS.cr_kineForward(robotHandle, planeCSNode.firstPoint.point.toolAxisAngle,
planeCSNode.firstPoint.point.toolPosition);
//正解求第一个点位姿
if (result != CRresult.sucess)//判断正解是否成功
{
    Console.WriteLine("第一个点正解失败" + "\r\n");
}

planeCSNode.firstPoint.id = 1;
planeCSNode.firstPoint.isValid = 1;
char[] str = ("first").ToCharArray();
planeCSNode.firstPoint.name = new char[32];
for (int i = 0; i < str.Length; i++)
{
    planeCSNode.firstPoint.name[i] = str[i];
}
planeCSNode.secondPoint.point.toolAxisAngle = new double[6] { 5.96, 5.28, 90, 34.67, -90,
4.76 };
//第二个点关节角度
planeCSNode.secondPoint.point.toolPosition = new double[6];
resut = CS.cr_kineForward(robotHandle, planeCSNode.secondPoint.point.toolAxisAngle,
planeCSNode.secondPoint.point.toolPosition);
```

```
//正解求第二个点位姿
if (result != CRresult.sucess)//判断正解是否成功
{
    Console.WriteLine("第二个点正解失败" + "\r\n");
}

planecsnode.secondPoint.id = 2;
planecsnode.secondPoint.isValid = 1;
char[] str1 = ("second").ToCharArray();
planecsnode.secondPoint.name = new char[32];
for (int i = 0; i < str1.Length; i++)
{
    planecsnode.secondPoint.name[i] = str1[i];
}
planecsnode.thirdPoint.point.toolAxisAngle = new double[6] { 7.80, 7.30, 86.45, 34.67, -92.46,
6.22 };
//第三个点关节角度
planecsnode.thirdPoint.point.toolPosition = new double[6];
result = CS.cr_kineForward(robotHandle, planecsnode.thirdPoint.point.toolAxisAngle,
planecsnode.thirdPoint.point.toolPosition);
//正解求第三个点位姿
if (result != CRresult.sucess)//判断正解是否成功
{
    Console.WriteLine("第三个点正解失败" + "\r\n");
}

planecsnode.thirdPoint.id = 3;
planecsnode.thirdPoint.isValid = 1;
char[] str2 = ("third").ToCharArray();
planecsnode.thirdPoint.name = new char[32];
for (int i = 0; i < str2.Length; i++)
{
    planecsnode.thirdPoint.name[i] = str2[i];
}
planecsnode.id = 2;
char[] str3 = ("plane1").ToCharArray();
planecsnode.name = new char[32];
for (int i = 0; i < str3.Length; i++)
{
    planecsnode.name[i] = str3[i];
}
planecsnode.coordinateJointPos = new double[6] { 0, 0, 0, 0, 0, 0 };
planecsnode.coordinatePose = new double[6];
int res = (int)CS.cr_compute_cs_plane(planecsnode.firstPoint.point.toolPosition, 6,
planecsnode.secondPoint.point.toolPosition, 6, planecsnode.thirdPoint.point.toolPosition, 6,
```



```

planeCSnode.coordinatePose, 6); //计算面坐标系位姿
if (res == 0) //判断这三点能否解算出面坐标系位姿，即坐标系是否有效
{
    planeCSnode.isValid = 1;
}
else
    planeCSnode.isValid = 0;

result = CS.cr_kineInverse(robotHandle, planeCSnode.coordinatePose,
planeCSnode.coordinateJointPos, planeCSnode.coordinateJointPos);
//逆解求面坐标系关节角度
if (result != CRresult.sucess)//判断逆解是否成功
{
    Console.WriteLine("逆解失败" + "\r\n");
}

result = CS.cr_cfg_cs_plane_set(robotHandle, 1, planeCSnode);
//修改面坐标系数据
if (result != CRresult.sucess)//判断修改面坐标系数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("\r\n");
Console.WriteLine(result.ToString());
Console.WriteLine("\r\n");
Console.WriteLine(planeCSnode.name[1].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

3.6.15.19 读取基坐标系数据

函数

CRresult cr_cfg_cs_base_get(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] csPose, int len)

简介

读取基坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
------	------	-------	----

robotHandle	int	输入	机械臂实例句柄
csPose	double[]	输出	基坐标系数据
len	int	输入	数据长度，长度为 6

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] csPose = new double[6];
result = CS.cr_cfg_cs_base_get(robotHandle, csPose, 6);
//读取基坐标系数据
if (result != CResult.success)//判断读取基坐标系数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.20 读取工具坐标系数据

函数

CResult cr_cfg_cs_tool_get(int robotHandle, [MarshalAs(UnmanagedType.LPArray, SizeConst = 6)] double[] csPose, int len)

简介

读取工具坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
csPose	double[]	输出	工具坐标系数据

len	int	输入	数据长度，长度为 6
-----	-----	----	------------

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] csPose = new double[6];
result = CS.cr_cfg_cs_tool_get(robotHandle, csPose, 6);
//读取工具坐标系数据
if (result != CRresult.sucess)//判断读取工具坐标系数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.21 计算点坐标系

函数

CRresult cr_compute_cs_point(double[] pointPose, int pLen, double[] pointCS, int csLen)

简介

计算点坐标系。

参数

参数名称	数据类型	输入/输出	描述
pointPose	double[]	输入	计算点坐标系的位姿：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）
pLen	int	输入	位姿的长度，至少为 6
pointCS	double[]	输出	点坐标系的位姿：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）

csLen	int	输入	点坐标系的长度，至少为 6
-------	-----	----	---------------

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] pointPose = new double[6] { -66, -438, 887, -57, 0, -148 };
double[] pointCS = new double[6];
result = CS.cr_compute_cs_point(pointPose, 6, pointCS, 6);
//计算点坐标系
if (result != CRresult.sucess)//判断计算点坐标系是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(pointCS[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.22 计算线坐标系

函数

CRresult cr_compute_cs_line(double[] point1Pose, int p1Len, double[] point2Pose, int p2Len, double[] lineCS, int csLen)

简介

计算线坐标系。

参数

参数名称	数据类型	输入/输出	描述
point1Pose	double[]	输入	计算线坐标系的位姿 1: x, y, z（单位: mm）; Rx, Ry, Rz（单位: °）
p1Len	int	输入	位姿的长度，至少为 6

point2Pose	double[]	输入	计算线坐标系的位姿 2: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
p2Len	int	输入	位姿的长度, 至少为 6
lineCS	double[]	输出	线坐标系的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
csLen	int	输入	线坐标系的长度, 至少为 6

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败, 查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] point1Pose = new double[6] { -66, -438, 887, -57, 0, -148 };
double[] point2Pose = new double[6] { -76, -438, 887, -57, 0, -148 };
double[] lineCS = new double[6];
result = CS.cr_compute_cs_line(point1Pose, 6, point2Pose, 6, lineCS, 6);
//计算线坐标系
if (result != CRresult.success)//判断计算线坐标系是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(lineCS[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接, 释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.15.23 计算面坐标系

函数

```
CRresult cr_compute_cs_plane(double[] point1Pose, int p1Len, double[] point2Pose, int p2Len,
double[] point3Pose, int p3Len, double[] planeCS, int csLen)
```

简介

计算面坐标系。

参数

参数名称	数据类型	输入/输出	描述
point1Pose	double[]	输入	计算面坐标系的位姿 1: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
p1Len	int	输入	位姿的长度, 至少为 6
point2Pose	double[]	输入	计算面坐标系的位姿 2: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
p2Len	int	输入	位姿的长度, 至少为 6
point3Pose	double[]	输入	计算面坐标系的位姿 3: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
p3Len	int	输入	位姿的长度, 至少为 6
planeCS	double[]	输出	面坐标系的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
csLen	int	输入	面坐标系的长度, 至少为 6

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败, 查看本地 ip 或者是否已有其他连接" + "\r\n");
}
double[] point1Pose = new double[6] { -70, -436.75, 893, 27.15, 22, 150.76 };
double[] point2Pose = new double[6] { -75, -436.75, 893, 27.15, 22, 150.76 };
double[] point3Pose = new double[6] { -70, -440, 893, 27.15, 22, 150.76 };
double[] planeCS = new double[6];
result = CS.cr_compute_cs_plane(point1Pose, 6, point2Pose, 6, point3Pose, 6, planeCS, 6);
//计算面坐标系
if (result != CRresult.success)//判断计算面坐标系是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(planeCS[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接, 释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

```
}
```

3.6.16 串口通讯

3.6.16.1 设置串口配置

函数

CRresult cr_cfg_comm_serial_setting_set(int robotHandle, **SerialType** serialType, SerialCommSettings serialCommSettings)

简介

设置串口配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
serialType	SerialType	输入	串口类型
serialCommSettings	SerialCommSettings	输入	串口配置数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SerialType serialType = SerialType.rs485_1_CommSettings;
SerialCommSettings serialCommSettings = new SerialCommSettings();
serialCommSettings.baudRate = BaudRate.BaudRate115200;
serialCommSettings.parity = ParityCheck.odd;
serialCommSettings.serialCommType = SerialCommType.Modbus_ASCII;
serialCommSettings.dataBits = 7;
serialCommSettings.retryNumber = 2;
serialCommSettings.timeout = 1000;
serialCommSettings.modbusSlaveNo = 1;
result = CS.cr_cfg_comm_serial_setting_set(robotHandle, serialType, serialCommSettings);
//设置串口配置
if (result != CRresult.sucess)//判断设置串口配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
```

```

}
Console.WriteLine(serialType.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

3.6.16.2 读取串口配置

函数

CRresult cr_cfg_comm_serial_setting_get(int robotHandle, **SerialType** serialType, ref **SerialCommSettings** serialCommSettings)

简介

读取串口配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
serialType	SerialType	输入	串口类型
serialCommSettings	ref SerialCommSettings	输出	串口配置数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SerialType serialType = SerialType.rs485_1_CommSettings;
SerialCommSettings serialCommSettings = new SerialCommSettings();
result = CS.cr_cfg_comm_serial_setting_get(robotHandle, serialType, ref serialCommSettings);
//读取串口配置
if (result != CRresult.sucess)//判断读取串口配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(serialType.ToString());

```



```
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.17 网络

3.6.17.1 设置控制柜网络配置

函数

CRresult cr_cfg_comm_ethernet_ip_set(int robotHandle, **IPConfig** ipconfig)

简介

设置控制柜网络配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
ipconfig	IPConfig	输入	网络地址配置

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
IPConfig ipconfig = new IPConfig();
ipconfig.ipAddr = new int[4] { 192, 168, 6, 7 };
ipconfig.subnetMask = new int[4] { 255, 255, 255, 0 };
result = CS.cr_cfg_comm_ethernet_ip_set(robotHandle, ipconfig);
//设置控制柜网络配置
if (result != CResult.sucess)//判断设置控制柜网络配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(ipconfig.ipAddr[0].ToString());
```

```
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.17.2 读取控制柜网络配置

函数

CRresult cr_cfg_comm_ethernet_ip_get(int robotHandle, ref **IPConfig** ipconfig)

简介

读取控制柜网络配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
ipconfig	ref IPConfig	输出	网络地址配置

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
IPConfig ipconfig = new IPConfig();
result = CS.cr_cfg_comm_ethernet_ip_get(robotHandle, ref ipconfig);
//读取控制柜网络配置
if (result != CRresult.sucess)//判断读取控制柜网络配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(ipconfig.ipAddr[0].ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
```

```
Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.17.3 设置控制柜从站配置

函数

CRresult cr_cfg_comm_ethernet_modbus_slave_num_set(int robotHandle, int modbusSlaveNo)

简介

设置控制柜从站配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
modbusSlaveNo	int	输入	从站号

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int modbusSlaveNo = 1200;
result = CS.cr_cfg_comm_ethernet_modbus_slave_num_set(robotHandle, modbusSlaveNo);
//设置控制柜从站配置
if (result != CRresult.success)//判断设置控制柜从站配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(modbusSlaveNo.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.17.4 读取控制柜从站配置

函数

CRresult cr_cfg_comm_ethernet_modbus_slave_num_get(int robotHandle, ref int modbusSlaveNo)

简介

读取控制柜从站配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
modbusSlaveNo	ref int	输出	从站号

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int modbusSlaveNo = -1;
result = CS.cr_cfg_comm_ethernet_modbus_slave_num_get(robotHandle, ref modbusSlaveNo);
//读取控制柜从站配置
if (result != CRresult.success)//判断读取控制柜从站配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(modbusSlaveNo.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.18 modbus 主站

3.6.18.1 读取从站个数

函数

CRresult cr_cfg_comm_modbus_slave_count(int robotHandle, **SerialType** type, ref int count)

简介

读取从站个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
count	ref int	输出	从站数量

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SerialType type = SerialType.rs232_CommSettings;
int count = -1;
result = CS.cr_cfg_comm_modbus_slave_count(robotHandle, type, ref count);
//读取从站个数
if (result != CRresult.sucess)//判断读取从站个数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(count.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.18.2 读取从站数据

函数

`CRresult cr_cfg_comm_modbus_slave_get(int robotHandle, SerialType type, int index, ref ModbusMasterConfig modbusConfig)`

简介

读取从站数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
index	int	输入	从站索引（范围：0-19）
modbusConfig	ref ModbusMasterConfig	输出	从站配置信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SerialType type = SerialType.rs232_CommSettings;
ModbusMasterConfig modbusConfig = new ModbusMasterConfig();
result = CS.cr_cfg_comm_modbus_slave_get(robotHandle, type, 1, ref modbusConfig);
//读取从站数据
if (result != CRresult.success)//判断读取从站数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(modbusConfig.id.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.18.3 添加从站

函数

CRresult cr_cfg_comm_modbus_slave_add(int robotHandle, **SerialType** type, ref **ModbusMasterConfig** modbusConfig)

简介

添加从站。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
modbusConfig	ref ModbusMasterConfig	输入	从站配置信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SerialType type = SerialType.ethernet_CommSettings;
ModbusMasterConfig modbusConfig = new ModbusMasterConfig();
modbusConfig.autoConnect = 0;
modbusConfig.id = 1;
modbusConfig.modbusMasterOperate = ModbusMasterOperate.Stop;
modbusConfig.scanTime = 10;
int[] slaveIP = new int[4];
slaveIP[0] = 192;
modbusConfig.slaveIP = slaveIP;
modbusConfig.slaveNo = 2;
result = CS.cr_cfg_comm_modbus_slave_add(robotHandle, type, ref modbusConfig);
//添加从站
if (result != CRresult.success)//判断添加从站是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(modbusConfig.id.ToString());
result = CS.cr_destroy_robot(robotHandle);
```

```
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.18.4 删除从站

函数

CRresult cr_cfg_comm_modbus_slave_delete(int robotHandle, **SerialType** type, int index)

简介

删除从站。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
index	int	输入	从站索引（范围：0-19）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SerialType type = SerialType.rs232_CommSettings;
result = CS.cr_cfg_comm_modbus_slave_delete(robotHandle, type, 1);
//删除从站
if (result != CRresult.sucess)//判断删除从站是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
```



```
Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.18.5 修改从站数据

函数

CRresult cr_cfg_comm_modbus_slave_set(int robotHandle, **SerialType** type, int index, **ModbusMasterConfig** modbusConfig)

简介

修改从站数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
index	int	输入	从站索引（范围：0-19）
modbusConfig	ModbusMasterConfig	输入	从站配置信息

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SerialType type = SerialType.ethernet_CommSettings;
ModbusMasterConfig modbusConfig = new ModbusMasterConfig();
modbusConfig.autoConnect = 0;
modbusConfig.id = 1;
modbusConfig.modbusMasterOperate = ModbusMasterOperate.Stop;
modbusConfig.scanTime = 10;
int[] slaveIP = new int[4];
slaveIP[0] = 192;
modbusConfig.slaveIP = slaveIP;
modbusConfig.slaveNo = 3;
result = CS.cr_cfg_comm_modbus_slave_set(robotHandle, type, 1, modbusConfig);
//修改从站数据
if (result != CRresult.sucess)//判断修改从站数据是否成功
{
```

```

        Console.WriteLine("接口运行失败" + "\r\n");
    }
    Console.WriteLine(modbusConfig.id.ToString());
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CResult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }

```

3.6.18.6 从站连接/断开

函数

[CResult](#) cr_cfg_comm_modbus_slave_operate(int robotHandle, [SerialType](#) type, int index, [ModbusMasterOperate](#) modbusOperate)

简介

从站连接/断开。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
index	int	输入	从站索引（范围：0-19）
modbusOperate	ModbusMasterOperate	输入	从站操作

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
ModbusMasterOperate modbusOperate = ModbusMasterOperate.Start;
SerialType type = SerialType.rs232_CommSettings;
result = CS.cr_cfg_comm_modbus_slave_operate(robotHandle, type, 1, modbusOperate);
//从站连接/断开
if (result != CResult.sucess)//判断从站连接/断开是否成功
{

```

```

        Console.WriteLine("接口运行失败" + "\r\n");
    }
    Console.WriteLine(result.ToString());
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }

```

3.6.18.7 获取从站连接状态

函数

CRresult cr_cfg_comm_modbus_slave_connect_get(int robotHandle, **SerialType** type, int index, ref int status)

简介

获取从站连接状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
index	int	输入	从站索引（范围：0-19）
status	ref int	输出	从站连接状态

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int status = 1;
SerialType type = SerialType.ethernet_CommSettings;
result = CS.cr_cfg_comm_modbus_slave_connect_get(robotHandle, type, 0, ref status);
if (result != CRresult.sucess)//判断获取从站连接状态是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

```

```

}
Console.WriteLine(status.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

3.6.18.8 读取地址映射数量

函数

CRresult cr_cfg_comm_modbus_addr_map_count(int robotHandle, **SerialType** type, int serialIndex, ref int count)

简介

读取地址映射数量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
serialIndex	int	输入	从站索引（范围：0-19）
count	ref int	输出	地址映射数量

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int count = -1;
SerialType type = SerialType.rs232_CommSettings;
result = CS.cr_cfg_comm_modbus_addr_map_count(robotHandle, type, 1, ref count);
//读取地址映射数量
if (result != CRresult.sucess)//判断读取地址映射数量是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}

```

```

}
Console.WriteLine(count.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
    
```

3.6.18.9 读取地址映射数据

函数

CRresult cr_cfg_comm_modbus_addr_map_get(int robotHandle, **SerialType** type, int serialIndex, int index, ref **ModbusAddrMap** addrMap)

简介

读取地址映射数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
serialIndex	int	输入	从站索引（范围：0-19）
index	int	输入	地址映射索引（范围：0-99）
addrMap	ref ModbusAddrMap	输出	地址映射数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SerialType type = SerialType.rs232_CommSettings;
ModbusAddrMap addrMap = new ModbusAddrMap();
result = CS.cr_cfg_comm_modbus_addr_map_get(robotHandle, type, 1, 1, ref addrMap);
//读取地址映射数据
if (result != CRresult.sucess)//判断读取地址映射数据是否成功
{
    
```

```

        Console.WriteLine("接口运行失败" + "\r\n");
    }
    Console.WriteLine(addrMap.slaveStartAddr.ToString());
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CResult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }

```

3.6.18.10 添加地址映射

函数

CResult cr_cfg_comm_modbus_addr_map_add(int robotHandle, **SerialType** type, int serialIndex, **ModbusAddrMap** addrMap)

简介

添加地址映射。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
serialIndex	int	输入	从站索引（范围：0-19）
addrMap	ModbusAddrMap	输入	地址映射数据

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SerialType type = SerialType.rs232_CommSettings;
ModbusAddrMap addrMap = new ModbusAddrMap();
addrMap.functionNo = ModbusMasterFunctionNo.ReadInt16;
addrMap.masterOffsetNo = 1;
addrMap.masterRegType = CommuVarType.CommuVarType_Float;
addrMap.masterStartAddr = 2;
addrMap.slaveStartAddr = 1;

```

```
result = CS.cr_cfg_comm_modbus_addr_map_add(robotHandle, type, 2, addrMap);
//添加地址映射
if (result != CRresult.sucess)//判断添加地址映射是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(addrMap.slaveStartAddr.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.18.11 删除地址映射

函数

CRresult cr_cfg_comm_modbus_addr_map_delete(int robotHandle, **SerialType** type, int serialIndex, int index)

简介

删除地址映射。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
serialIndex	int	输入	从站索引（范围：0-19）
index	int	输入	地址映射索引（范围：0-99）

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SerialType type = SerialType.rs232_CommSettings;
result = CS.cr_cfg_comm_modbus_addr_map_delete(robotHandle, type, 1, 1);
```

```
//删除地址映射
if (result != CRresult.sucess)//判断删除地址映射是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.18.12 修改地址映射数据

函数

`CRresult cr_cfg_comm_modbus_addr_map_set(int robotHandle, SerialType type, int serialIndex, int index, ModbusAddrMap addrMap)`

简介

修改地址映射数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
serialIndex	int	输入	从站索引（范围：0-19）
index	int	输入	地址映射索引（范围：0-99）
addrMap	ModbusAddrMap	输入	地址映射数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
SerialType type = SerialType.rs232_CommSettings;
ModbusAddrMap addrMap = new ModbusAddrMap();
```



```
addrMap.functionNo = ModbusMasterFunctionNo.ReadInt16;
addrMap.masterOffsetNo = 1;
addrMap.masterRegType = CommuVarType.CommuVarType_Float;
addrMap.masterStartAddr = 2;
addrMap.slaveStartAddr = 1;
result = CS.cr_cfg_comm_modbus_addr_map_set(robotHandle, type, 1, 1, addrMap);
//修改地址映射数据
if (result != CRresult.sucess)//判断修改地址映射数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(addrMap.slaveStartAddr.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.19 力控

3.6.19.1 设置力控配置

函数

CRresult cr_force_cfg_set(int robotHandle, **ForceConfig** forceConfig)

简介

设置力控配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
forceConfig	ForceConfig	输入	力控配置参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
```

```
        Console.WriteLine("连接失败, 查看本地 ip 或者是否已有其他连接" + "\r\n");
    }
    ForceConfig forceConfig = new ForceConfig();

    forceConfig.commuConfig = new CommuConfig();
    forceConfig.commuConfig.autoConnect = 1;
    //自动与传感器通信, 传感器开始接收数据
    forceConfig.commuConfig.mannaulOperate = 0;
    //自动与传感器通信, 传感器开始接收数据
    forceConfig.commuConfig.paraBuf = 0;
    //预留参数

    forceConfig.forceCtrlTcpImpl = new ForceToolSetting();
    byte[] tmp_bytes = Encoding.UTF8.GetBytes("ForcePayload7");
    //激活的力控传感器负载的名称
    forceConfig.forceCtrlTcpImpl.activeForcePayload_name = new byte[20];
    for (int i = 0; i < tmp_bytes.Length; i++)
    {
        forceConfig.forceCtrlTcpImpl.activeForcePayload_name[i] = tmp_bytes[i];
    }
    tmp_bytes = Encoding.UTF8.GetBytes("TCP_4");
    //激活的力控传感器 TCP 的名称
    forceConfig.forceCtrlTcpImpl.activeForceTCP_name = new byte[20];
    for (int i = 0; i < tmp_bytes.Length; i++)
    {
        forceConfig.forceCtrlTcpImpl.activeForceTCP_name[i] = tmp_bytes[i];
    }

    forceConfig.forceCtrlTcpImpl.availableForcePayLoad = new ForcePayLoad[20];
    //当前待选的力控传感器的配置信息
    forceConfig.forceCtrlTcpImpl.availableForcePayLoad[0].forceCenterOfGravity = new double[3]
    { 1, 2, 3 };
    tmp_bytes = Encoding.UTF8.GetBytes("ForcePayload7");
    forceConfig.forceCtrlTcpImpl.availableForcePayLoad[0].forcePayloadName = new byte[20];
    for (int i = 0; i < tmp_bytes.Length; i++)
    {
        forceConfig.forceCtrlTcpImpl.availableForcePayLoad[0].forcePayloadName[i] =
        tmp_bytes[i];
    }
    forceConfig.forceCtrlTcpImpl.availableForcePayLoad[0].forceToolPayload = 1.3;
    forceConfig.forceCtrlTcpImpl.availableForcePayLoad[0].forcePayloadId = 0;
    forceConfig.forceCtrlTcpImpl.availableForcePayLoad[1].forceCenterOfGravity = new double[3]
    { 1, 2, 3 };
    tmp_bytes = Encoding.UTF8.GetBytes("ForcePayload6");
```

```
forceConfig.forceCtrlTcpImpl.availableForcePayLoad[1].forcePayloadName = new byte[20];
for (int i = 0; i < tmp_bytes.Length; i++)
{
    forceConfig.forceCtrlTcpImpl.availableForcePayLoad[1].forcePayloadName[i] =
tmp_bytes[i];
}
forceConfig.forceCtrlTcpImpl.availableForcePayLoad[1].forceToolPayload = 1.3;
forceConfig.forceCtrlTcpImpl.availableForcePayLoad[1].forcePayloadId = 1;
forceConfig.forceCtrlTcpImpl.availableForcePayLoad[2].forceCenterOfGravity = new double[3]
{ 0, 0, 0 };
tmp_bytes = Encoding.UTF8.GetBytes("ForcePayload5");
forceConfig.forceCtrlTcpImpl.availableForcePayLoad[2].forcePayloadName = new byte[20];
for (int i = 0; i < tmp_bytes.Length; i++)
{
    forceConfig.forceCtrlTcpImpl.availableForcePayLoad[2].forcePayloadName[i] =
tmp_bytes[i];
}
forceConfig.forceCtrlTcpImpl.availableForcePayLoad[2].forceToolPayload = 0.5;
forceConfig.forceCtrlTcpImpl.availableForcePayLoad[2].forcePayloadId = 2;

forceConfig.forceCtrlTcpImpl.availableForcePayLoadLen = 3;
//当前待选的力控传感器负载的配置信息的个数

forceConfig.forceCtrlTcpImpl.availableForceTCP = new ForceTCPMsg[20];
//当前待选的力控传感器 TCP 的配置信息
forceConfig.forceCtrlTcpImpl.availableForceTCP[0].forceTcpOffset = new double[6] { 0, 0, 0, 0,
0, -110 };
forceConfig.forceCtrlTcpImpl.availableForceTCP[0].forceTcpId = 0;
tmp_bytes = Encoding.UTF8.GetBytes("TCP_4");
forceConfig.forceCtrlTcpImpl.availableForceTCP[0].forceTcpName = new byte[20];
for (int i = 0; i < tmp_bytes.Length; i++)
{
    forceConfig.forceCtrlTcpImpl.availableForceTCP[0].forceTcpName[i] = tmp_bytes[i];
}

forceConfig.forceCtrlTcpImpl.availableForceTCP[1].forceTcpOffset = new double[6] { 0, 0, 0,
180, 0, 0 };
forceConfig.forceCtrlTcpImpl.availableForceTCP[1].forceTcpId = 1;
tmp_bytes = Encoding.UTF8.GetBytes("ForceTcp5");
forceConfig.forceCtrlTcpImpl.availableForceTCP[1].forceTcpName = new byte[20];
for (int i = 0; i < tmp_bytes.Length; i++)
{
    forceConfig.forceCtrlTcpImpl.availableForceTCP[1].forceTcpName[i] = tmp_bytes[i];
}
```

```

forceConfig.forceCtrlTcpImpl.availableForceTCP[2].forceTcpOffset = new double[6] { 0, 0, 0,
180, 0, 0 };
forceConfig.forceCtrlTcpImpl.availableForceTCP[2].forceTcpId = 2;
tmp_bytes = Encoding.UTF8.GetBytes("ForceTcp4");
forceConfig.forceCtrlTcpImpl.availableForceTCP[2].forceTcpName = new byte[20];
for (int i = 0; i < tmp_bytes.Length; i++)
{
    forceConfig.forceCtrlTcpImpl.availableForceTCP[2].forceTcpName[i] = tmp_bytes[i];
}

forceConfig.forceCtrlTcpImpl.availableForceTCPLen = 3;
//当前待选的力控传感器 TCP 的配置信息的个数

forceConfig.sensorMessage = new SensorMessage();
forceConfig.sensorMessage.productNo = 1;
//传感器产品号，预留参数
forceConfig.sensorMessage.sequenceNo = 0;
//传感器序列号，预留参数
forceConfig.sensorMessage.venderNo = 0;
//传感器品牌
result = CS.cr_force_cfg_set(robotHandle, forceConfig);
//设置力控配置
if (result != CRresult.sucess)//判断设置力控配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("result = " + result);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
    
```

3.6.19.2 读力控配置

函数

[CRresult](#) cr_force_cfg_get(int robotHandle, ref [ForceConfig](#) forceConfig)

简介

读力控配置。

参数

参数名称	数据类型	输入/输出	描述
------	------	-------	----

robotHandle	int	输入	机械臂实例句柄
forceConfig	ref ForceConfig	输出	力控配置参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
//读力控配置
ForceConfig forceConfig = new ForceConfig();
result = CS.cr_force_cfg_get(robotHandle, ref forceConfig);
if (result != CRresult.sucess)//判断读力控配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("result = " + result);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.19.3 开启力控并下发相关设置

函数

CRresult cr_force_open(int robotHandle, **ForceSetting** forceSetting)

简介

开启力控并下发相关设置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
forceSetting	ForceSetting	输入	力控设置参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
//开启力控并下发相关设置
ForceSetting forceSetting = new ForceSetting
{
    flexibleAxis = new int[6] { 0, 0, 1, 0, 0, 0 },
    //z 轴可拖动
    forceBaseType = 0,
    //力控类型: 0-基于 TCP 力传感器,1-基于动力学,2-基于底座力传感器
    forceCtrlType = 1
    //力控用法: 0-运动力控,1-力控拖动
};
result = CS.cr_force_open(robotHandle, forceSetting);
//开启力控并下发相关设置
if (result != CRresult.sucess)//判断开启力控并下发相关设置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("result = " + result);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.19.4 关闭力控

函数

CRresult cr_force_close(int robotHandle)

简介

关闭力控。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_force_close(robotHandle);
//关闭力控
if (result != CRresult.sucess)//判断关闭力控是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("result = " + result);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.19.5 设置力控控制参数

函数

CRresult cr_force_para_set(int robotHandle, **ForceCtlPara** forceCtlPara)

简介

设置力控控制参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
forceCtlPara	ForceCtlPara	输入	力控控制参数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
```

```
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
//设置力控控制参数
ForceCtlPara forceCtlPara = new ForceCtlPara
{
    damping = new double[6] { 1, 1, 0.1, 1, 1, 1 },
    //x,y,Rx,Ry,Rz 阻尼系数均为 1,z 阻尼系数为 0.1
    dampingLen = 6,
    //0、1 或 6
    limits = new double[6] { 0.1, 0.1, 0.005, 0.002, 0.002, 0.002 },
    //x,y 方向速度保护为 0.1m/s，z 方向速度保护为 0.005m/s,Rx,Ry,Rz 偏差保护为
    0.002rad
    limitsLen = 6,
    //0 或 6
    mass = new double[6] { 1, 1, 0.5, 1, 1, 1 },
    //x,y,Rx,Ry,Rz 质量系数均为 1,z 质量系数为 0.5
    massLen = 6,
    //0、1 或 6
    stiffness = new double[6] { 1, 1, 0.1, 1, 1, 1 },
    //x,y,Rx,Ry,Rz 刚度均为 1,z 刚度为 0.1
    stiffnessLen = 6,
    //0、1 或 6
    taskFrame = new double[6] { 0, 0, 0, 0, 0, 0 },
    //力控坐标系为基坐标系
    wrench = new double[6] { 0, 0, 10, 0, 0, 0 }
    //z 轴正向施加 10N 的力
};
result = CS.cr_force_para_set(robotHandle, forceCtlPara);
//设置力控控制参数
if (result != CRresult.sucess)//判断设置力控控制参数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("result = " + result);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```



```
}
```

3.6.19.6 读力控控制参数

函数

```
CRresult cr_force_para_get(int robotHandle, ref ForceCtlPara forceCtlPara)
```

简介

读力控控制参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
forceCtlPara	ref ForceCtlPara	输出	力控控制参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
ForceCtlPara forceCtlPara = new ForceCtlPara();
result = CS.cr_force_para_get(robotHandle, ref forceCtlPara);
//读力控控制参数
if (result != CRresult.success)//判断读力控控制参数是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("result = " + result + ", massLen = " + forceCtlPara.massLen);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.19.7 读力控数据

函数

CRresult cr_force_data_get(int robotHandle, ref **ForceData** forceData)

简介

读力控数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
forceData	ref ForceData	输出	力控数据

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
ForceData forceData = new ForceData();
result = CS.cr_force_data_get(robotHandle, ref forceData);
//读力控数据
if (result != CRresult.sucess)//判断读力控数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine("result = " + result + ", controlMode = " +
forceData.controlMode.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.20 扩展轴

3.6.20.1 设置扩展轴使能状态

函数

CRresult cr_set_exj_enableStatus(int robotHandle, int index, int status)

简介

设置扩展轴使能状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	扩展轴索引
status	int	输入	扩展轴状态

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int status = 1;
result = CS.cr_set_exj_enableStatus(robotHandle, 0, status);
if (result != CRresult.sucess)//判断设置扩展轴使能状态是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.20.2 读取扩展轴使能状态

函数

CRresult cr_get_exj_enableStatus(int robotHandle, int index, ref int status)

简介

读取扩展轴使能状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
index	int	输入	扩展轴索引
status	ref int	输出	扩展轴状态

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int status1 = 0;
result = CS.cr_get_exj_enableStatus(robotHandle, 0, ref status1);
if (result != CRresult.sucess)//判断读取扩展轴使能状态是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(status1.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.20.3 读取机械臂和扩展轴数据

函数

CRresult cr_get_robot_exj_data(int robotHandle, ref **RobotExjData** robotExjData)

简介

读取机械臂和扩展轴数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄
robotExjData	RobotExjData*	输出	待读取的值

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
RobotExjData robotExjData = new RobotExjData();
result = CS.cr_get_robot_exj_data(robotHandle, ref robotExjData);
if (result != CRresult.success)//判断读取机械臂和扩展轴数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(robotExjData.actualExjointPos[0]);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.20.4 设置扩展轴配置

函数

CRresult cr_cfg_exj_set(int robotHandle, string exjName, **ExjConfig** exjConfig)

简介

设置扩展轴配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
exjName	string	输入	扩展轴名称
exjConfig	ExjConfig	输入	扩展轴配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
ExjConfig exjConfig = new ExjConfig();
exjConfig.name = new byte[50];
byte[] tmp_name = Encoding.UTF8.GetBytes("Exj_1");
for (int i = 0; i < tmp_name.Length; i++)
{
    exjConfig.name[i] = tmp_name[i];
}
exjConfig.type = ExternalJointType.line_externalJoint;
exjConfig.ratio = 100;
exjConfig.minlimit = -99999;
exjConfig.maxlimit = 99999;
exjConfig.zeroPosition = 2;
exjConfig.masterSlaveRate = 1;
exjConfig.exjToRef = new double[6] { 0, 0, 0, 0, 0, 0 };
exjConfig.masterExjName = new byte[50];
byte[] tmp_exjname = Encoding.UTF8.GetBytes("");
for (int i = 0; i < tmp_exjname.Length; i++)
{
    exjConfig.masterExjName[i] = tmp_exjname[i];
}
result = CS.cr_cfg_exj_set(robotHandle, "Exj_0", exjConfig);
if (result != CRresult.sucess)//判断设置扩展轴配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
```

```
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.20.5 获取扩展轴配置

函数

CRresult cr_cfg_exj_get(int robotHandle, string exjName, ref **ExjConfig** exjConfig)

简介

获取扩展轴配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
exjName	string	输入	扩展轴名称
exjConfig	ref ExjConfig	输出	扩展轴配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
ExjConfig exjConfig = new ExjConfig();
string exjName = "Exj_1";
result = CS.cr_cfg_exj_get(robotHandle, exjName, ref exjConfig);
if (result != CRresult.sucess)//判断获取扩展轴配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(exjConfig.ratio);
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
```

```
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.20.6 增加扩展轴

函数

CResult cr_cfg_exj_add(int robotHandle, **ExjConfig** exjConfig)

简介

增加扩展轴。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
exjConfig	ExjConfig	输入	扩展轴配置

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
ExjConfig exjConfig = new ExjConfig();
exjConfig.name = new byte[50];
byte[] tmp_name = Encoding.UTF8.GetBytes("Exj_03");
for (int i = 0; i < tmp_name.Length; i++)
{
    exjConfig.name[i] = tmp_name[i];
}
exjConfig.type = ExternalJointType.line_externalJoint;
exjConfig.ratio = 100;
exjConfig.minlimit = -99999;
exjConfig.maxlimit = 99999;
exjConfig.zeroPosition = 2;
exjConfig.masterSlaveRate = 1;
exjConfig.exjToRef = new double[6] { 0, 0, 0, 0, 0, 0 };
exjConfig.masterExjName = new byte[50];
```



```
byte[] tmp_exjname = Encoding.UTF8.GetBytes("");
for (int i = 0; i < tmp_exjname.Length; i++)
{
    exjConfig.masterExjName[i] = tmp_exjname[i];
}
result = CS.cr_cfg_exj_add(robotHandle, exjConfig);
if (result != CResult.success)//判断获取扩展轴配置是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.6.20.7 删除扩展轴

函数

CResult cr_cfg_exj_delete(**RobotHandle** robotHandle)

简介

删除扩展轴（只能删除最后一个）。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
result = CS.cr_cfg_exj_delete(robotHandle);
if (result != CResult.success)//判断获取扩展轴配置是否成功
{
}
```

```

        Console.WriteLine("接口运行失败" + "\r\n");
    }
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }

```

3.7 通讯

3.7.1 EtherCAT

3.7.1.1 ethercat 扫描

函数

CRresult cr_action_ethercat_scan(int robotHandle, ref **EtherCATSlaveData** etherCATSlaveData, ref int count)

简介

ethercat 扫描。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
etherCATSlaveData	ref EtherCATSlaveData	输出	从站信息
count	ref int	输出	ethercat 扫描的从站个数

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
EtherCATSlaveData[] tmp = new EtherCATSlaveData[10];
int size = Marshal.SizeOf(typeof(EtherCATSlaveData));
IntPtr ptr = Marshal.AllocHGlobal(size * 20);

```

```
int count = -1;
result = CS.cr_action_ethercat_scan(robotHandle, ptr, ref count);
//ethercat 扫描
if (result != CRresult.sucess)//判断 ethercat 扫描是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
for (int i = 0; i < count; i++)
{
    IntPtr ptr_s = ptr + i * size;
    tmp[i] = (EtherCATSlaveData)Marshal.PtrToStructure(ptr_s, typeof(EtherCATSlaveData));
}
Marshal.FreeHGlobal(ptr);
string hexadecimalNumber = tmp[0].revisionNo.ToString("X");
Console.WriteLine(hexadecimalNumber.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.1.2 读 SDO

函数

CRresult cr_get_sdo(int robotHandle, int slaveNo, short objectIndex, short objectsubIndex, byte[] dataBuff, int dataBuffLength, ref int dataTyeLength, ref bool commuResult, ref int abort_code)

简介

读 SDO。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
slaveNo	int	输入	从站号
objectIndex	short	输入	索引号
objectsubIndex	short	输入	子索引号
dataBuff	byte[]	输出	读取的数据
dataBuffLength	int	输入	dataBuff 开辟的实际字节空间大小
dataTyeLength	ref int	输出	被读取值的数据类型对应长度
commuResult	ref bool	输出	通讯结果
abort_code	ref int	输出	中止代码

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] dataBuff = new byte[100];
int slaveNo = 2;
short objectIndex = 0x201D;
short objectsubIndex = 0;
int dataBuffLength = 100;
int dataTyeLength = 0;
bool commuResult = true;
int abort_code = 0;
result = CS.cr_get_sdo(robotHandle, slaveNo, objectIndex, objectsubIndex, dataBuff,
dataBuffLength, ref dataTyeLength, ref commuResult, ref abort_code);
//读 SDO
if (result != CResult.success)//判断读 SDO 是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
int Data = BitConverter.ToInt32(dataBuff, 0);
Console.WriteLine(Data.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.1.3 写 SDO

函数

```
CResult cr_set_sdo(int robotHandle, int slaveNo, short objectIndex, short objectsubIndex, byte[]
dataBuff, int dataBuffLength, int dataTyeLength, ref bool commuResult, ref int abort_code)
```

简介

写 SDO。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
slaveNo	int	输入	从站号
objectIndex	short	输入	索引号
objectsubIndex	short	输入	子索引号
dataBuff	byte[]	输入	写入的数据
dataBuffLength	int	输入	dataBuff 开辟的实际字节空间大小
dataTyeLength	int	输入	写入值的数据类型对应长度
commuResult	ref bool	输出	通讯结果
abort_code	ref int	输出	中止代码

返回值

数据类型为 CResult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] dataBuff = new byte[100];
dataBuff = BitConverter.GetBytes(1000);
int slaveNo = 2;
short objectIndex = 0x201D;
short objectsubIndex = 0;
int dataBuffLength = 100;
int dataTyeLength = 10;
bool commuResult = true;
int abort_code = 0;
result = CS.cr_set_sdo(robotHandle, slaveNo, objectIndex, objectsubIndex, dataBuff,
dataBuffLength, dataTyeLength, ref commuResult, ref abort_code);
//写 SDO
if (result != CResult.sucess)//判断写 SDO 是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
int Data = BitConverter.ToInt32(dataBuff, 0);
Console.WriteLine(Data.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
```

```
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.1.4 读 ESC 寄存器

函数

CRresult cr_get_escReg(int robotHandle, int slaveNo, int regOffset, int regNum, byte[] dataBuff, ref int dataBuffLength, ref bool commuResult, ref int abort_code)

简介

读 ESC 寄存器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
slaveNo	int	输入	从站号
oregOffset	int	输入	寄存器首地址
regNum	int	输入	寄存器个数
dataBuff	byte[]	输出	读取的数据
dataBuffLength	ref int	输出	dataBuff 字节空间大小
commuResult	ref bool	输出	通讯结果
abort_code	ref int	输出	中止代码

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int slaveNo = 1;
int regOffset = 0x00;
int regNum = 2;
byte[] dataBuff = new byte[100];
int dataBuffLength = 10;
bool commuResult = true;
int abort_code = 0;
```

```
result = CS.cr_get_escReg(robotHandle, slaveNo, regOffset, regNum, dataBuff, ref
dataBuffLength, ref commuResult, ref abort_code);
//读 ESC 寄存器
if (result != CRresult.sucess)//判断读 ESC 寄存器是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
int Data = BitConverter.ToInt32(dataBuff, 0);
Console.WriteLine(Data.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.1.5 写 ESC 寄存器

函数

CRresult cr_set_escReg(int robotHandle, int slaveNo, int regOffset, int regNum, byte[] dataBuff, int dataBuffLength, ref bool commuResult, ref int abort_code)

简介

写 ESC 寄存器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
slaveNo	int	输入	从站号
oregOffset	int	输入	寄存器首地址
regNum	int	输入	寄存器个数
dataBuff	byte[]	输入	待写入的数据
dataBuffLength	int	输入	dataBuff 字节空间大小
commuResult	ref bool	输出	通讯结果
abort_code	ref int	输出	中止代码

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
```

```

if (result != CResult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int slaveNo = 1;
int regOffset = 0x10;
int regNum = 2;
byte[] dataBuff = new byte[100];
dataBuff = BitConverter.GetBytes(3);
int dataBuffLength = 10;
bool commuResult = true;
int abort_code = 0;
result = CS.cr_set_escReg(robotHandle, slaveNo, regOffset, regNum, dataBuff, dataBuffLength,
ref commuResult, ref abort_code);
//写 ESC 寄存器
if (result != CResult.sucess)//判断写 ESC 寄存器是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
int Data = BitConverter.ToInt32(dataBuff, 0);
Console.WriteLine(Data.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
    
```

3.7.1.6 读 FOE

函数

CResult cr_get_foe(int robotHandle, int slaveNo, byte[] fileName, int fileNameLength, byte[] fileData, ref int fileDataLength, ref int abort_code)

简介

读 FOE。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
slaveNo	int	输入	从站号
fileName	byte[]	输入	待读取的文件名
fileNameLength	int	输入	待读取的文件名字节长度

fileData	byte[]	输出	读取数据
fileDataLength	ref int	输出	读取数据字节长度
abort_code	ref int	输出	中止代码

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int slaveNo = 2;
byte[] fileName = new byte[100];
int fileNameLength = 10;
byte[] fileData = new byte[60000];
int fileDataLength = 1000;
int abort_code = 0;
result = CS.cr_get_foe(robotHandle, slaveNo, fileName, fileNameLength, fileData, ref
fileDataLength, ref abort_code);
//读 FOE
if (result != CResult.success)//判断读 FOE 是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
int Data = BitConverter.ToInt32(fileData, 0);
Console.WriteLine(Data.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.1.7 写 FOE

函数

```
CResult cr_set_foe(int robotHandle, int slaveNo, byte[] fileName, int fileNameLength, byte[]
fileData, int fileDataLength, ref int abort_code)
```

简介

写 FOE。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
slaveNo	int	输入	从站号
fileName	byte[]	输入	待读取的文件名
fileNameLength	int	输入	待读取的文件名字节长度
fileData	byte[]	输入	待写入数据
fileDataLength	int	输入	待写入数据字节长度
abort_code	ref int	输出	中止代码

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int slaveNo = 2;
byte[] fileName = new byte[100];
int fileNameLength = 10;
byte[] fileData = new byte[60000];
int fileDataLength = 1000;
int abort_code = 0;
result = CS.cr_set_foe(robotHandle, slaveNo, fileName, fileNameLength, fileData,
fileDataLength, ref abort_code);
//写 FOE
if (result != CResult.success)//判断写 FOE 是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
int Data = BitConverter.ToInt32(fileData, 0);
Console.WriteLine(Data.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
```

```
Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.1.8 读 EEPROM

函数

CRresult cr_get_sii(int robotHandle, int slaveNo, int offset, int dataSize, byte[] databuff, ref int databuffLength)

简介

读 EEPROM。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
slaveNo	int	输入	从站号
offset	int	输入	写操作首地址(按字取址)
dataSize	int	输入	写操作数据个数(字长)
databuff	byte[]	输出	待读取数据
databuffLength	ref int	输出	读取到的数据字节长度

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] dataBuff = new byte[2048];
int databuffLength = 0;
for (int i = 0; i < 1024; i++)
{
    byte[] newbuf = new byte[2];
    result = CS.cr_get_sii(robotHandle, 3, i, 1, newbuf, ref databuffLength); //读 EEPROM
    if (result != CRresult.sucess) //判断读 EEPROM 是否成功
    {
        Console.WriteLine("接口运行失败" + "\r\n");
    }
    Array.Copy(newbuf, 0, dataBuff, 2 * i, 2);
}
```

```

        Console.WriteLine(dataBuff[2*i]);
        Console.WriteLine(dataBuff[2*i+1]);
    }
    result = CS.cr_destroy_robot(robotHandle);
    //机器人断开连接，释放句柄
    if (result != CRresult.sucess)//判断机器人断开连接是否成功
    {
        Console.WriteLine("断开连接失败" + "\r\n");
    }

```

3.7.1.9 写 EEPROM

函数

CRresult cr_set_sii(int robotHandle, int slaveNo, int offset, int dataSize, byte[] dataBuff, int dataBuffLength)

简介

写 EEPROM。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
slaveNo	int	输入	从站号
offset	int	输入	写操作首地址(按字取址)
dataSize	int	输入	写操作数据个数(字长)
databuff	byte[]	输入	待写入数据
databuffLength	int	输入	写入的数据字节长度

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```

int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
byte[] dataBuff = new byte[2048];
using (FileStream file = new FileStream("./CGX ETF Module.bin", FileMode.Open,
FileAccess.Read))
{

```

```

if (file != null)
{
    Console.WriteLine("open success");
    file.Read(dataBuff, 0, dataBuff.Length);
    file.Close();

    byte[] newbuf = new byte[2];
    //逐个字写入对应从站号
    for (int i = 0; i < 1024; i++)
    {
        Array.Copy(dataBuff, 2*i, newbuf, 0, 2);
        result = CS.cr_set_sii(robotHandle, 8, i, 1, newbuf, 2); //写 EEPROM
        Console.WriteLine("i = " + i + " result = " + result);
        if (result != CResult.sucess) //判断写 EEPROM 是否成功
        {
            Console.WriteLine("接口运行失败" + "\r\n");
        }
    }
}
else
{
    Console.WriteLine("open fail");
}
}

result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.sucess) //判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
}
    
```

3.7.1.10 读 ESC 状态

函数

CResult cr_get_escState(int robotHandle, int slaveNo, ref **EscState** actualDLLState, ref bool commuResult)

简介

读 ESC 状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄

slaveNo	int	输入	从站号
actualDLLState	ref EscState	输出	当前 ESC 状态
commuResult	ref bool	输出	通讯结果

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int slaveNo = 1;
bool commuResult = true;
EscState actualDLLState = EscState.ECT_STATE_INIT;
result = CS.cr_get_escState(robotHandle, slaveNo, ref actualDLLState, ref commuResult);
//读 ESC 状态
if (result != CResult.success)//判断读 ESC 状态是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(actualDLLState.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.1.11 写 ESC 状态

函数

CResult cr_set_escState(int robotHandle, int slaveNo, **EscState** requestDLLState, ref bool commuResult)

简介

写 ESC 状态。

参数

参数名称	数据类型	输入/输出	描述
------	------	-------	----

robotHandle	int	输入	机械臂实例句柄
slaveNo	int	输入	从站号
requestDLLState	EscState	输入	待写入 ESC 状态
commuResult	ref bool	输出	通讯结果

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int slaveNo = 2;
bool commuResult = true;
EscState actualDLLState = EscState.ECT_STATE_INIT;
result = CS.cr_set_escState(robotHandle, slaveNo, actualDLLState, ref commuResult);
//写 ESC 状态
if (result != CResult.success)//判断写 ESC 状态是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(actualDLLState.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.1.12 周期通讯激活

函数

CResult cr_ecatActive(int robotHandle, int slaveNo)

简介

周期通讯激活。

参数

参数名称	数据类型	输入/输出	描述
------	------	-------	----

robotHandle	int	输入	机械臂实例句柄
slaveNo	int	输入	从站号

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int slaveNo = 1;
result = CS.cr_ecatActive(robotHandle, slaveNo);
//周期通讯激活
if (result != CResult.success)//判断周期通讯激活是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
Console.WriteLine(result.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.1.13 读取数据从 PDO 映射地址

函数

```
CResult cr_get_pdo(int robotHandle, int offAddress, byte[] pdoData, ref int pdoDataLength, ref
Ect_DataType pdoDataType, ref Ect_PdoType pdoType)
```

简介

读取数据从 PDO 映射地址。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
offAddress	int	输入	映射地址
pdoData	byte[]	输出	读出的数据

pdoDataLength	ref int	输出	读出的数据字节长度
pdoDataType	ref Ect_DataType	输出	数据类型
pdoType	ref Ect_PdoType	输出	pdo 类型

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CResult.success)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int offAddress = 221;
byte[] pdoData = new byte[100];
int pdoDataLength = 10;
Ect_DataType pdoDataType = Ect_DataType.Ect_DINT;
Ect_PdoType pdoType = Ect_PdoType.Ect_RxPdo;
result = CS.cr_get_pdo(robotHandle, offAddress, pdoData, ref pdoDataLength, ref pdoDataType,
ref pdoType);
//读取数据从 PDO 映射地址
if (result != CResult.success)//判断读取数据从 PDO 映射地址是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
int Data = BitConverter.ToInt32(pdoData, 0);
Console.WriteLine(Data.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CResult.success)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.1.14 写入数据到 PDO 映射地址

函数

```
CResult cr_set_pdo(int robotHandle, int offAddress, byte[] pdoData, int pdoDataLength,
Ect_DataType pdoDataType, Ect_PdoType pdoType)
```

简介

写入数据到 PDO 映射地址。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
offAddress	int	输入	映射地址
pdoData	byte[]	输入	写入的数据
pdoDataLength	int	输入	写入的数据字节长度
pdoDataType	Ect_DataType	输入	数据类型
pdoType	Ect_PdoType	输入	pdo 类型

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int offAddress = 204;
byte[] pdoData = new byte[100];
pdoData = BitConverter.GetBytes(9);
int pdoDataLength = 10;
Ect_DataType pdoDataType = Ect_DataType.Ect_SINT;
Ect_PdoType pdoType = Ect_PdoType.Ect_RxPdo;
result = CS.cr_set_pdo(robotHandle, offAddress, pdoData, pdoDataLength, pdoDataType,
pdoType);
//写入数据从 PDO 映射地址
if (result != CRresult.sucess)//判断写入数据到 PDO 映射地址是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
int Data = BitConverter.ToInt32(pdoData, 0);
Console.WriteLine(Data.ToString());
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.1.15 读多个 Pdo 映射地址数据

函数

CRresult cr_get_pdo_array(int robotHandle, PdoTypeData[] pdoArray, int pdoArrayLen)

简介

读多个 Pdo 映射地址数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
pdoArray	PdoTypeData[]	输入	pdo 映射地址数组
pdoArrayLen	int	输入	pdo 映射地址数组大小

返回值

数据类型为 CRresult，若为 sucess 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle=-1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
PdoTypeData[] pdoArray = new PdoTypeData[9];
for (int i = 0; i < pdoArray.Length; i++)
{
    pdoArray[i].offAddress = i;
}
result = CS.cr_get_pdo_array(robotHandle, pdoArray, pdoArray.Length);
//读多个 Pdo 映射地址数据
if (result != CRresult.sucess)//读多个 Pdo 映射地址数据是否成功
{
    Console.WriteLine("接口运行失败" + "\r\n");
}
for (int i = 0; i < pdoArray.Length; i++)
{
    Console.WriteLine(pdoArray[i].offAddress + "\t" +
        BitConverter.ToUInt16(pdoArray[i].pdoData, 0) + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
```

```
if (result != CRresult.sucess)//判断机器人断开连接是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.2 CANOpen

3.7.2.1 CANopen 写 pdo 数据

函数

```
CRresult cr_co_pdo_set(int robotHandle, int offAddress, byte[] coPdoData, int coPdoDataLength,
CoDataType coPdoDataType, CoPdoType coPdoType)
```

简介

CANopen 写 pdo 数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
offAddress	int	输入	映射地址
coPdoData	byte[]	输入	写入的数据
coPdoDataLength	int	输入	写入的数据字节长度
coPdoDataType	CoDataType	输入	数据类型
coPdoType	CoPdoType	输入	pdo 类型

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess)//判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int offAddress = 4;
byte[] coPdoData = new byte[100];
coPdoData = BitConverter.GetBytes(75);
int coPdoDataLength = 10;
CoDataType coPdoDataType = CoDataType.EDS_DT_U16;
CoPdoType coPdoType = CoPdoType.CO_RX_PDO;
result = CS.cr_co_pdo_set(robotHandle, offAddress, coPdoData, coPdoDataLength,
```

```
coPdoDataType, coPdoType); //判断 CANopen 写 pdo 数据是否成功
if (result != CRresult.sucess)
{
    Console.WriteLine("接口运行失败 cr_co_pdo_set" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接，释放句柄
if (result != CRresult.sucess) //CANopen 写 pdo 数据是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}
```

3.7.2.2 CANopen 读 pdo 数据

函数

CRresult cr_co_pdo_get(int robotHandle, int offAddress, byte[] coPdoData, ref int coPdoDataLength, ref CoPdoType coPdoDataType, ref CoPdoType coPdoType)

简介

CANopen 读 pdo 数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	int	输入	机械臂实例句柄
offAddress	int	输入	映射地址
coPdoData	byte[]	输出	读出的数据
coPdoDataLength	ref int	输出	读出的数据字节长度
coPdoDataType	CoDataType*	输出	数据类型
coPdoType	CoPdoType*	输出	pdo 类型

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
int robotHandle = -1;
CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
//机器人连接
if (result != CRresult.sucess) //判断机器人连接是否成功
{
    Console.WriteLine("连接失败，查看本地 ip 或者是否已有其他连接" + "\r\n");
}
int offAddress = 4;
byte[] coPdoData = new byte[100];
```

```

int coPdoDataLength = 10;
CoDataType coPdoDataType = CoDataType.EDS_DT_U16;
CoPdoType coPdoType = CoPdoType.CO_RX_PDO;
result = CS.cr_co_pdo_get(robotHandle, offAddress, coPdoData, ref coPdoDataLength, ref
coPdoDataType, ref coPdoType);//判断 CANopen 读 pdo 数据是否成功
if (result != CRresult.sucess)
{
    Console.WriteLine("接口运行失败 cr_co_pdo_set" + "\r\n");
}
result = CS.cr_destroy_robot(robotHandle);
//机器人断开连接, 释放句柄
if (result != CRresult.sucess)//CANopen 读 pdo 数据是否成功
{
    Console.WriteLine("断开连接失败" + "\r\n");
}

```

四、综合示例

4.1 机械臂使能

简介

对机械臂进行操作需要使机械臂处于使能状态。本示例的功能为：在机械臂处于未上电状态时，通过 sdk 接口使机械臂上电使能完成。

示例

```

public static int robotHandle = -1;
static void Main(string[] args)
{
    CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
    if (result == CRresult.sucess)
    {
        RobotModes robotMode = RobotModes.BackDrive;
        result = CS.cr_get_robotMode(robotHandle, ref robotMode); //读机械臂当前状态
        if (result == CRresult.sucess && robotMode == RobotModes.JointPowerOff) //判断状态,robotMode 为 6 时是本体未上电状态才可以进行上电
        {
            result = CS.cr_poweron(robotHandle); //机械臂上电
            if (result == CRresult.sucess)
            {
                while(result == CRresult.sucess)
                {

```

```

        result = CS.cr_get_robotMode(robotHandle, ref robotMode);
        Thread.Sleep(50);
        if (robotMode == RobotModes.JointIdle)           //判断状态,robotMode
为 8 时是机器人上电完成后再进行使能
            break;
    }
    result = CS.cr_enable(robotHandle); //机械臂使能
    if (result == CRresult.sucess)
    {
        while (result == CRresult.sucess)
        {
            result = CS.cr_get_robotMode(robotHandle, ref robotMode);
            Thread.Sleep(50);
            if (robotMode == RobotModes.ProgramStop)       //判断状
态,robotMode 为 103 时机器人使能完成
                break;
        }
        Console.WriteLine($"行号: {GetLineNum().ToString()},使能完成");
    }
    else
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
    }
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}
result = CS.cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != CRresult.sucess)
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},机器人断开连接失败");
}

```

```

        Console.WriteLine($"按下任意键退出。");
        Console.ReadKey();
    }

    private static int GetLineNum()
    {
        StackTrace st = new StackTrace(1, true);

        return st.GetFrame(0).GetFileLineNumber();
    }

```

4.2 工程控制

简介

本示例介绍了对工程进行控制的方法，包括下载、上传、运行工程。

注：本示例中下载的工程会将机械臂的 4、5、6 关节各运动 5°，故使用本示例前，需注意机械臂运动过程中是否会和其他物体发生碰撞，以及是否会超出关节角度范围限值。

示例

```

public static int robotHandle = -1;
static void Main(string[] args)
{
    CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
    if (result == CResult.sucess)
    {
        Lua_ScriptStatus scriptstatus = Lua_ScriptStatus.lua_Script_load;
        result = CS.cr_get_lua_scriptstatus(robotHandle, ref scriptstatus); //读脚本运行状态
        if (result == CResult.sucess && scriptstatus == Lua_ScriptStatus.lua_Script_stop) //当
脚本运行状态处于脚本程序停止时才可进行程序下载
        {
            string crpFilepathname = ".././../program/demo.crp"; //crp 文件路径
            string crscriptFilepathname = ".././../program/demo.crscript"; //脚本程序路径
            result = CS.cr_downloadProject(robotHandle, crpFilepathname,
crscriptFilepathname); //下载程序
            if (result == CResult.sucess)
            {
                result = CS.cr_play(robotHandle); //运行程序
                if (result == CResult.sucess)
                {
                    while (result == CResult.sucess)
                    {
                        RobotModes robotMode = RobotModes.BackDrive;
                        result = CS.cr_get_robotMode(robotHandle, ref robotMode);
                        Thread.Sleep(50);

```



```

        if (robotMode == RobotModes.ProgramStop) //读机械臂当前状态,程序停止状态下才可以上传程序
        {
            break;
        }
        if (result == CResult.sucess)
        {
            string FilePath = "../program/"; //文件保存路径
            string filename = "demo"; // .crp 和脚本程序的文件名
            result = CS.cr_uploadProject(robotHandle, FilePath, filename); //工程文件上传

            if (result == CResult.sucess)
            {
                Console.WriteLine($"行号: {GetLineNum().ToString()},已成功上传工程文件");
            }
            else
            {
                Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
            }
        }
        else
        {
            Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
        }
    }
    else
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
    }
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {scriptstatus.ToString()}");
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}

```

```

    }
    else
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
    }
    result = CS.cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
    if (result != CResult.sucess)
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},机器人断开连接失败");
    }
    Console.WriteLine($"按下任意键退出。");
    Console.ReadKey();
}

private static int GetLineNum()
{
    StackTrace st = new StackTrace(1, true);

    return st.GetFrame(0).GetFileLineNumber();
}

```

4.3 移动控制

简介

本示例介绍了 moveJ、moveL、cr_move_joint、cr_move_line 四个接口的使用方法，moveJ 和 moveL 是非阻塞移动，cr_move_joint 和 cr_move_line 可以设置是否需要阻塞移动。

注：使用本示例前，需注意机械臂运动过程中是否会和其他物体发生碰撞。

示例

```

public static int robotHandle = -1;
static void Main(string[] args)
{
    CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
    if (result == CResult.sucess)
    {
        PointControlPara pointControlPara = new PointControlPara
        {
            speed = new double[6] { 30, 30, 30, 30, 30, 30 },
            acc = new double[6] { 30, 30, 30, 30, 30, 30 },
            jerk = new double[6] { 60, 60, 60, 60, 60, 60 },
            pose = new double[6] { 0, 0, 0, 0, 0, 0 }, //在 MoveJ 中无影响
            jointpos = new double[6] { 0, 0, 90, 0, -90, 0 }, //目标点示例数据
            tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 },
            coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 },
        }
    }
}

```

```

        tcpID = -1,
        coordinateType = CoordinateType.jointCoordinate,
        pointTransType = PointTransType.pointTransStop,
        pointTransRadius = 0,
        poseTranType = PoseTranType.poseTranMoveToTargetPose,
        motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc
    };
    result = CS.cr_moveJ(robotHandle, pointControlPara); //轴空间运动
    if (result == CResult.sucess)
    {
        Thread.Sleep(50);
        int isRobotMoving = 1;
        while (result == CResult.sucess)
        {
            result = CS.cr_get_robotMoveStatus(robotHandle, ref isRobotMoving);
            //读机械臂运动状态,如果运动已经停止,代表上一次运动已经结束,则
跳出循环进行下一次运动
            Thread.Sleep(50);
            if (isRobotMoving == 0)
            {
                break;
            }
        }
        result = CS.cr_get_tcpActualPose(robotHandle, pointControlPara.pose); //读实际
TCP 位置
        if (result == CResult.sucess)
        {
            pointControlPara.pose[2] = pointControlPara.pose[2] - 150; //Z 方向
向下运动 150mm
            pointControlPara.coordinateType = CoordinateType.baseCoordinate;
            result = CS.cr_moveL(robotHandle, pointControlPara); //直线运动
            if (result == CResult.sucess)
            {
                Thread.Sleep(50);
                while (result == CResult.sucess)
                {
                    result = CS.cr_get_robotMoveStatus(robotHandle, ref
isRobotMoving);
                    //读机械臂运动状态,如果运动已经停止,代表上一次运动已经
结束,则跳出循环进行下一次运动
                    Thread.Sleep(50);
                    if (isRobotMoving == 0)
                    {
                        break;

```

```

    }
}
pointControlPara.pose[2] = pointControlPara.pose[2] + 150;
PointControlParaSimple pointControlParaSimple = new
PointControlParaSimple
{
    speed = new double[6] { 30, 30, 30, 30, 30, 30 },
    acc = new double[6] { 30, 30, 30, 30, 30, 30 },
    pose = pointControlPara.pose,
    jointpos = new double[6] { 0, 0, 90, 0, -90, 0 }, //目标点示例数
据

    tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 },
    coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 },
    coordinateType = CoordinateType.baseCoordinate,
    pointTransType = PointTransType.pointTransStop,
    motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc
};
result = CS.cr_move_pointControlPara_transfer(robotHandle,
pointControlParaSimple, ref pointControlPara);
//PointControlParaSimple 数据转换为 PointControlPara
result = CS.cr_move_line(robotHandle, pointControlPara, true);
//阻塞式的直线运动，由于是阻塞的直线运动，则运动会在运动到
位后在进行后续的程序运行，因此不用像之前做判断
if (result == CRresult.sucess)
{
    pointControlParaSimple.jointpos = new double[6] { 90, 0, 90, 0, -
90, 0 };

    pointControlParaSimple.coordinateType =
CoordinateType.jointCoordinate;
    result = CS.cr_move_pointControlPara_transfer(robotHandle,
pointControlParaSimple, ref pointControlPara);
//PointControlParaSimple 数据转换为 PointControlPara
result = CS.cr_move_joint(robotHandle, pointControlPara, true);
//阻塞式的轴空间运动，由于是阻塞的直线运动，则运动会在
运动到位后在进行后续的程序运行，因此不用像之前做判断
if (result == CRresult.sucess)
{
    result = CS.cr_get_tcpActualPose(robotHandle,
pointControlPara.pose); //读实际 TCP 位置
    pointControlPara.pose[2] = pointControlPara.pose[2] - 150;
    pointControlParaSimple.pose = pointControlPara.pose;
    pointControlParaSimple.coordinateType =
CoordinateType.baseCoordinate;
    result = CS.cr_move_pointControlPara_transfer(robotHandle,
pointControlParaSimple, ref pointControlPara);

```

```
//PointControlParaSimple 数据转换为 PointControlPara
result = CS.cr_moveL(robotHandle, pointControlPara); //直
线运动

if (result == CRresult.sucess)
{
    Thread.Sleep(50);
    while (result == CRresult.sucess)
    {
        result = CS.cr_get_robotMoveStatus(robotHandle,
ref isRobotMoving);

        //读机械臂运动状态,如果运动已经停止,代表上
        一次运动已经结束,则跳出循环进行下一次运动
        Thread.Sleep(50);
        if (isRobotMoving == 0)
        {
            break;
        }
    }
    pointControlPara.pose[2] = pointControlPara.pose[2] +
150;

    result =
CS.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple, ref
pointControlPara);

    //PointControlParaSimple 数据转换为 PointControlPara
    result = CS.cr_move_line(robotHandle, pointControlPara,
false);

    //非阻塞式直线运动, 下发运动后会立马执行下一条
指令

    if (result == CRresult.sucess)
    {
        while (result == CRresult.sucess)
        {
            result =
CS.cr_get_robotMoveStatus(robotHandle, ref isRobotMoving);

            //读机械臂运动状态,如果运动已经停止,代
            表上一次运动已经结束,则跳出循环进行下一次运动
            Thread.Sleep(50);
            if (isRobotMoving == 0)
            {
                break;
            }
        }
        Console.WriteLine($"行号:
{GetLineNum().ToString()},运动已完成");
```

```

        }
        else
        {
            Console.WriteLine($"行号:
{GetLineNum().ToString()},结果: {result}");
        }
    }
    else
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},
结果: {result}");
    }
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结
果: {result}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}

```

```

    }
    result = CS.cr_destroy_robot(robotHandle); //机器人断开连接，释放句柄
    if (result != CResult.sucess)
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},机器人断开连接失败");
    }
    Console.WriteLine($"按下任意键退出。");
    Console.ReadKey();
}
private static int GetLineNum()
{
    StackTrace st = new StackTrace(1, true);

    return st.GetFrame(0).GetFileLineNumber();
}

```

4.4 轨迹再现

简介

本示例介绍了记录一段轨迹并把轨迹下载到控制柜中的方法。

注：本示例将轨迹下载到索引 2 中，如果控制器中已存在索引为 2 的轨迹，将新记录的轨迹下载到索引 2 后，原轨迹将会被覆盖。使用本示例前，需注意机械臂运动过程中是否会和其他物体发生碰撞。

示例

```

public static int robotHandle = -1;
static void Main(string[] args)
{
    CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
    if (result == CResult.sucess)
    {
        PointControlParaSimple pointControlParaSimple = new PointControlParaSimple
        {
            speed = new double[6] { 30, 30, 30, 30, 30, 30 },
            acc = new double[6] { 30, 30, 30, 30, 30, 30 },
            pose = new double[6] { 0, 0, 0, 0, 0, 0 },
            jointpos = new double[6] { 0, 0, 90, 0, -90, 0 }, //目标点示例数据
            tcpOffset = new double[6] { 0, 0, 0, 0, 0, 0 },
            coordinatePose = new double[6] { 0, 0, 0, 0, 0, 0 },
            coordinateType = CoordinateType.jointCoordinate,
            pointTransType = PointTransType.pointTransStop,
            motiontriggerMode = MotiontriggerMode.MovetriggerbyOnlyRpc
        };
        PointControlPara pointControlPara = new PointControlPara();
    }
}

```

```

    result = CS.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple,
ref pointControlPara);
    //PointControlParaSimple 数据转换为 PointControlPara
    result = CS.cr_move_joint(robotHandle, pointControlPara, true); //轴空间运动
    if (result == CRresult.sucess)
    {
        RecordPathPara recordPathPara = new RecordPathPara
        {
            sampleTime = 2,
            recordControl = 1
        };
        result = CS.cr_path_recordPara_set(robotHandle, recordPathPara); //设置轨迹记
录参数

        Thread.Sleep(500);
        pointControlParaSimple.jointpos = new double[6] { 90, 0, 90, 0, -90, 0 };
        result = CS.cr_move_pointControlPara_transfer(robotHandle,
pointControlParaSimple, ref pointControlPara);
        //PointControlParaSimple 数据转换为 PointControlPara
        result = CS.cr_move_joint(robotHandle, pointControlPara, true); //轴空间运动
        if (result == CRresult.sucess)
        {
            recordPathPara.recordControl = 0;
            result = CS.cr_path_recordPara_set(robotHandle, recordPathPara); //设置轨
迹记录参数

            if (result == CRresult.sucess)
            {
                int index = -1;
                PathData pathData = new PathData();
                pathData.pathPoints = new IntPtr();
                int size = Marshal.SizeOf(typeof(PathPoint));
                pathData.pathPoints = Marshal.AllocHGlobal(size * 10000);
                PathPoint[] pathPoints = new PathPoint[10000];
                result = CS.cr_path_upload(robotHandle, index, ref pathData);
                if (result == CRresult.sucess)
                {
                    int[] allPathIndex = new int[11];
                    int pathIndexLen = -1;
                    result = CS.cr_path_all_index_get(robotHandle, allPathIndex, ref
pathIndexLen);

                    //读取控制柜当前存在的轨迹索引
                    for (int i = 0; i < pathIndexLen; i++)
                    {
                        if (allPathIndex[i] == 2) //将下载的轨迹索引设置为 2, 故
需要判断其是否已经存在

```



```

        {
            Console.WriteLine($"行号: {GetLineNum().ToString()},
存在轨迹索引为 2 的轨迹");

            PathDownloadData pathDownloadData = new
            PathDownloadData();

            pathDownloadData.pathData = new PathData();
            pathDownloadData.pathPara = new PathPara();

            pathDownloadData.pathData.pathPoints = new IntPtr();
            pathDownloadData.pathData.pathPoints =
            Marshal.AllocHGlobal(size * 10000);

            for (int points_count = 0; points_count < 10000;
points_count++)//轨迹点指针内容转化
            {
                IntPtr newPtr = IntPtr.Add(pathData.pathPoints,
points_count * size);

                pathPoints[points_count] =
                (PathPoint)Marshal.PtrToStructure(newPtr, typeof(PathPoint));
            }

            pathDownloadData.pathData.moveTime =
            pathData.moveTime;

            pathDownloadData.pathData.pathPointsNum =
            pathData.pathPointsNum;

            pathDownloadData.pathPara.index = 2; //轨迹下载参数
            设置

            pathDownloadData.pathPara.moveType = 1;

            for (int points_count = 0; points_count < 10000;
points_count++)//将新的轨迹点位转回指针
            {
                Marshal.StructureToPtr(pathPoints[points_count],
IntPtr.Add(pathDownloadData.pathData.pathPoints, points_count * size), false);
            }

            result = CS.cr_path_download(robotHandle,
            pathDownloadData); //下载轨迹到索引 2
            if (result == CResult.sucess)
            {
                Console.WriteLine($"行号:
{GetLineNum().ToString()},已成功下载索引为 2 的轨迹");
            }
            else
            {

```

```

        Console.WriteLine($"行号:
{GetLineNum().ToString()},结果: {result}");
    }
    break;
}
if (i == (pathIndexLen - 1))
{
    PathDownloadData pathDownloadData = new
PathDownloadData();

    pathDownloadData.pathData = new PathData();
    pathDownloadData.pathPara = new PathPara();

    pathDownloadData.pathData.pathPoints = new IntPtr();
    pathDownloadData.pathData.pathPoints =
Marshal.AllocHGlobal(size * 10000);

    for (int points_count = 0; points_count < 10000;
points_count++)//轨迹点指针内容转化
    {
        IntPtr newPtr = IntPtr.Add(pathData.pathPoints,
points_count * size);

        pathPoints[points_count] =
(PathPoint)Marshal.PtrToStructure(newPtr, typeof(PathPoint));
    }

    pathDownloadData.pathData.moveTime =
pathData.moveTime;

    pathDownloadData.pathData.pathPointsNum =
pathData.pathPointsNum;

    pathDownloadData.pathPara.index = 2; //轨迹下载参数
设置

    pathDownloadData.pathPara.moveType = 1;

    for (int points_count = 0; points_count < 10000;
points_count++)//将新的轨迹点位转回指针
    {
        Marshal.StructureToPtr(pathPoints[points_count],
IntPtr.Add(pathDownloadData.pathData.pathPoints, points_count * size), false);
    }

    result = CS.cr_path_download(robotHandle,
pathDownloadData); //下载轨迹到索引 2
    if (result == CResult.sucess)
    {
        Console.WriteLine($"行号:

```

```

{GetLineNum().ToString()},{已成功下载索引为 2 的轨迹"};

        }
        else
        {
            Console.WriteLine($"行号:
{GetLineNum().ToString()},{结果: {result}}");
        }
    }
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},{结果:
{result}}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},{结果:
{result}}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},{结果: {result}}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},{结果: {result}}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},{结果: {result}}");
}
}
else
{
    Console.WriteLine($"按下任意键退出。");
    Console.ReadKey();
}
}

```

```
private static int GetLineNum()
{
    StackTrace st = new StackTrace(1, true);

    return st.GetFrame(0).GetFileLineNumber();
}
```

4.5 安装变量操作

简介

本示例介绍了与安装变量相关的接口的使用方法，包括读取安装变量个数、读取安装变量、修改安装变量、增加安装变量。

示例

```
public static int robotHandle = -1;
static void Main(string[] args)
{
    CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
    if (result == CResult.sucess)
    {
        int count = -1;
        result = CS.cr_cfg_var_install_count(robotHandle, ref count); //读取安装变量个数
        if (result == CResult.sucess)
        {
            VariableMsg variableMsg = new VariableMsg
            {
                variableName = "i_var_1",
                variableID = 3,
                variableType = VariableType.LUA_TBOOLEAN,
                boolValue = 1
            };
            if (count > 0)
            {
                for (int i = 0; i < count; i++)
                {
                    IntPtr ptr = IntPtr.Zero;
                    int size = Marshal.SizeOf(typeof(VariableMsg));
                    ptr = Marshal.AllocHGlobal(size);
                    result = CS.cr_cfg_var_install_get(robotHandle, i, ptr);
                    VariableMsg tmp_variableMsg =
                    (VariableMsg)Marshal.PtrToStructure(ptr, typeof(VariableMsg));
                    if (tmp_variableMsg.stringValue == variableMsg.stringValue) //判断是
                    否存在我想添加的同名安装变量，如果有就修改，没有就增加
                }
            }
        }
    }
}
```

```

        int size1 = Marshal.SizeOf(variableMsg);
        IntPtr ptr1 = Marshal.AllocHGlobal(size);
        Marshal.StructureToPtr(variableMsg, ptr1, true);
        result = CS.cr_cfg_var_install_set(robotHandle, i, ptr1);
        if (result == CRresult.sucess)
        {
            Console.WriteLine($"行号: {GetLineNum().ToString()},已成功修改安装变量");
        }
        else
        {
            Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
        }
    }
    else
    {
        int size1 = Marshal.SizeOf(variableMsg);
        IntPtr ptr1 = Marshal.AllocHGlobal(size);
        Marshal.StructureToPtr(variableMsg, ptr1, true);
        result = CS.cr_cfg_var_install_add(robotHandle, ptr1); //增加安装
        if (result == CRresult.sucess)
        {
            Console.WriteLine($"行号: {GetLineNum().ToString()},已成功增加安装变量");
        }
        else
        {
            Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
        }
    }
}
}
else
    //如果不存在安装变量，则添加安装变量
    {
        int size = Marshal.SizeOf(variableMsg);
        IntPtr ptr = Marshal.AllocHGlobal(size);
        Marshal.StructureToPtr(variableMsg, ptr, true);
        result = CS.cr_cfg_var_install_add(robotHandle, ptr);
        if (result == CRresult.sucess)
        {

```

```

        Console.WriteLine($"行号: {GetLineNum().ToString()},已成功增加安
装变量");
    }
    else
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
    }
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}
result = CS.cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != CRresult.sucess)
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},机器人断开连接失败");
}
Console.WriteLine($"按下任意键退出。");
Console.ReadKey();
}
private static int GetLineNum()
{
    StackTrace st = new StackTrace(1, true);

    return st.GetFrame(0).GetFileLineNumber();
}

```

4.6 点坐标系操作

简介

本示例介绍了与点坐标系操作相关的接口的使用方法, 包括读取点坐标系个数、读取点坐标系、修改安装变量、增加安装变量、删除点坐标系。

示例

```

public static int robotHandle = -1;
static void Main(string[] args)
{

```

```

CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
if (result == CResult.sucess)
{
    int count = -1;
    result = CS.cr_cfg_cs_point_count(robotHandle, ref count); //读取点坐标系个数
    if (result == CResult.sucess)
    {
        char[] str = ("point1").ToCharArray();
        PointCSNode pointcsnode = new PointCSNode
        {
            point = new PoseMessage
            {
                toolAxisAngle = new double[6] { 30, 30, 30, 30, 30, 30 },
                toolPosition = new double[6]
            },
            id = 1,
            isValid = 0,
            name = new char[32],
        };
        for (int i = 0; i < str.Length; i++)
        {
            pointcsnode.name[i] = str[i];
        }
        if (count > 0)
        {
            for (int i = 0; i < count; i++)
            {
                PointCSNode tmp_pointcsnode = new PointCSNode();
                result = CS.cr_cfg_cs_point_get(robotHandle, i, ref tmp_pointcsnode);
                if (result == CResult.sucess)
                {
                    if (tmp_pointcsnode.name == pointcsnode.name)
                    {
                        result = CS.cr_kineForward(robotHandle,
pointcsnode.point.toolAxisAngle, pointcsnode.point.toolPosition); //正解求点的位姿
                        if (result == CResult.sucess)
                        {
                            result =
CS.cr_compute_cs_point(pointcsnode.point.toolPosition, 6, pointcsnode.point.toolPosition, 6);
                            if (result == CResult.sucess)
                            {
                                pointcsnode.isValid = 1;
                                result = CS.cr_cfg_cs_point_set(robotHandle,
i, pointcsnode); //修改点坐标系成功
                                if (result == CResult.sucess)

```

```

        {
            Console.WriteLine($"行号：
{GetLineNum().ToString()},修改点坐标系成功");
        }
        else
        {
            Console.WriteLine($"行号：
{GetLineNum().ToString()},结果： {result}");
        }
    }
    else
    {
        Console.WriteLine($"行号：
{GetLineNum().ToString()},结果： {result}");
    }
}
else
{
    Console.WriteLine($"行号： {GetLineNum().ToString()},
结果： {result}");
}
break;
}
if (i == (count - 1))
{
    result = CS.cr_kineForward(robotHandle,
pointcsnode.point.toolAxisAngle, pointcsnode.point.toolPosition);//正解求点的位姿
    if (result == CRresult.sucess)
    {
        result =
CS.cr_compute_cs_point(pointcsnode.point.toolPosition, 6, pointcsnode.point.toolPosition, 6);
        if (result == CRresult.sucess)
        {
            pointcsnode.isValid = 1;
            result = CS.cr_cfg_cs_point_add(robotHandle,
pointcsnode);//增加点坐标系数据

            if (result == CRresult.sucess)
            {
                Console.WriteLine($"行号：
{GetLineNum().ToString()},增加点坐标系成功");
            }
            else
            {
                Console.WriteLine($"行号：

```



```

{GetLineNum().ToString()},结果: {result}");
        }
    }
    else
    {
        Console.WriteLine($"行号:
{GetLineNum().ToString()},结果: {result}");
    }
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},
结果: {result}");
}
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
}
}
}
else
{
    result = CS.cr_kineForward(robotHandle, pointcsnode.point.toolAxisAngle,
pointcsnode.point.toolPosition);
    if (result == CRresult.sucess)
    {
        result = CS.cr_compute_cs_point(pointcsnode.point.toolPosition, 6,
pointcsnode.point.toolPosition, 6);
        if (result == CRresult.sucess)
        {
            pointcsnode.isValid = 1;
            result = CS.cr_cfg_cs_point_add(robotHandle, pointcsnode);//增加
点坐标系数据

            if (result == CRresult.sucess)
            {
                Console.WriteLine($"行号: {GetLineNum().ToString()},增加
点坐标系成功");
            }
        }
        else
        {
            Console.WriteLine($"行号: {GetLineNum().ToString()},结

```

```

果: {result}");
        }
    }
    else
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
    }
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
}
}
int index = 0;
result = CS.cr_cfg_cs_point_get(robotHandle, index, ref pointesnode);
//读取索引号为 0 的点坐标系数据
if (result == CRresult.sucess)
{
    result = CS.cr_cfg_cs_point_delete(robotHandle, index);
    //删除索引号为 0 的点坐标系
    if (result == CRresult.sucess)
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},已成功删除点
坐标系");
    }
    else
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
    }
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}
else

```

```

    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
    }
    result = CS.cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
    if (result != CResult.sucess)
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},机器人断开连接失败");
    }
    Console.WriteLine($"按下任意键退出。");
    Console.ReadKey();
}
private static int GetLineNum()
{
    StackTrace st = new StackTrace(1, true);

    return st.GetFrame(0).GetFileLineNumber();
}

```

4.7 输出弹窗

简介

本示例介绍了读取输出弹窗的方法, 首先通过加载运行程序来创建一个输出弹窗, 再判断弹窗是否存在, 若存在则读取输出弹窗信息。

示例

```

public static int robotHandle = -1;
static void Main(string[] args)
{
    CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
    if (result == CResult.sucess)
    {
        string programfile = @"function mainFuncProgram()
                                --start robotConfig
                                TCP_1 = { 0,0,0,0,0,0 }
                                Payload_1= { 0,{0,0,0} }
                                set_tcp_payload(Payload_1[1], Payload_1[2])
                                --end robotConfig
                                function RobotProgram()
                                while (true)
                                do
                                --popup_out:text
                                popup_message("hello", "message", false, false, true)
                                wait(1)--sync
                                end

```

```

        end
        RobotProgram_Result = task_create(RobotProgram)
        function PauseFuncProgram()
        while (true)
        do
        wait(100)--sync
        end
        end
        PauseFuncProgram_Result = task_create(PauseFuncProgram)
        end
        mainFuncProgram_Result = task_create(mainFuncProgram)";
result = CS.cr_downloadProgram(robotHandle, programfile);
if (result == CRresult.sucess)
{
    result = CS.cr_play(robotHandle);//运行程序
    if (result == CRresult.sucess)
    {
        bool hasPopup = false;
        result = CS.cr_script_popup_exist(robotHandle, ref hasPopup);
        if (result == CRresult.sucess)
        {
            if (hasPopup == true)
            {
                PopUpMsg popupMsg = new PopUpMsg();
                result = CS.cr_script_popup_msg_get(robotHandle, ref popupMsg);
                if (result == CRresult.sucess)
                {
                    Console.WriteLine($"行号: {GetLineNum().ToString()},弹窗
信息:{Encoding.UTF8.GetString(popupMsg.var_data)}");
                    result = CS.cr_stop(robotHandle); //停止程序运行
                }
            }
            else
            {
                Console.WriteLine($"行号: {GetLineNum().ToString()},结
果: {result}");
            }
        }
        else
        {
            Console.WriteLine($"行号: {GetLineNum().ToString()},没有弹窗
信息");
        }
    }
    else
    {

```

```

        Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
    }
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}
}
result = CS.cr_destroy_robot(robotHandle); //机器人断开连接, 释放句柄
if (result != CResult.sucess)
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},机器人断开连接失败");
}
Console.WriteLine($"按下任意键退出。");
Console.ReadKey();
}
private static int GetLineNum()
{
    StackTrace st = new StackTrace(1, true);

    return st.GetFrame(0).GetFileLineNumber();
}

```

4.8 数学变量

简介

本示例介绍了将当前基坐标系下的 TCP 位姿转换到点坐标系下的过程。

示例

```

public static int robotHandle = -1;
static void Main(string[] args)
{
    CResult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
    if (result == CResult.sucess)

```

```
{
    int count = -1;
    result = CS.cr_cfg_cs_point_count(robotHandle, ref count); //读取点坐标系个数
    if (result == CRresult.sucess)
    {
        char[] str = ("point1").ToCharArray();
        PointCSNode pointcsnode = new PointCSNode
        {
            point = new PoseMessage
            {
                toolAxisAngle = new double[6] { 30, 30, 30, 30, 30, 30 },
                toolPosition = new double[6]
            },
            id = 1,
            isValid = 0,
            name = new char[32],
        };
        for (int i = 0; i < str.Length; i++)
        {
            pointcsnode.name[i] = str[i];
        }
        if (count > 0)
        {
            for (int i = 0; i < count; i++)
            {
                PointCSNode tmp_pointcsnode = new PointCSNode();
                result = CS.cr_cfg_cs_point_get(robotHandle, i, ref tmp_pointcsnode);
                if (result == CRresult.sucess)
                {
                    if (tmp_pointcsnode.name == pointcsnode.name)
                    {
                        result = CS.cr_kineForward(robotHandle,
pointcsnode.point.toolAxisAngle, pointcsnode.point.toolPosition); //正解求点的位姿
                        if (result == CRresult.sucess)
                        {
                            result =
CS.cr_compute_cs_point(pointcsnode.point.toolPosition, 6, pointcsnode.point.toolPosition, 6);
                            if (result == CRresult.sucess)
                            {
                                pointcsnode.isValid = 1;
                                result = CS.cr_cfg_cs_point_set(robotHandle, i,
pointcsnode); //修改点坐标系成功

                                if (result == CRresult.sucess)
                                {
                                    Console.WriteLine($"行号:
```

```
{GetLineNum().ToString()},修改点坐标系成功");
        }
        else
        {
            Console.WriteLine($"行号:
{GetLineNum().ToString()},结果: {result}");
        }
    }
    else
    {
        Console.WriteLine($"行号:
{GetLineNum().ToString()},结果: {result}");
    }
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},
结果: {result}");
}
break;
}
if (i == (count - 1))
{
    result = CS.cr_kineForward(robotHandle,
pointcsnode.point.toolAxisAngle, pointcsnode.point.toolPosition);//正解求点的位姿
    if (result == CRresult.sucess)
    {
        result =
CS.cr_compute_cs_point(pointcsnode.point.toolPosition, 6, pointcsnode.point.toolPosition, 6);
        if (result == CRresult.sucess)
        {
            pointcsnode.isValid = 1;
            result = CS.cr_cfg_cs_point_add(robotHandle,
pointcsnode);//增加点坐标系数据
            if (result == CRresult.sucess)
            {
                Console.WriteLine($"行号:
{GetLineNum().ToString()},增加点坐标系成功");
            }
        }
        else
        {
            Console.WriteLine($"行号:
{GetLineNum().ToString()},结果: {result}");
        }
    }
}
```

```

        }
        else
        {
            Console.WriteLine($"行号:
{GetLineNum().ToString()},结果: {result}");
        }
    }
    else
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},
结果: {result}");
    }
}
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
}
}
else
{
    result = CS.cr_kineForward(robotHandle, pointcsnode.point.toolAxisAngle,
pointcsnode.point.toolPosition);//正解求点的位姿
    if (result == CRresult.sucess)
    {
        result = CS.cr_compute_cs_point(pointcsnode.point.toolPosition, 6,
pointcsnode.point.toolPosition, 6);
        if (result == CRresult.sucess)
        {
            pointcsnode.isValid = 1;
            result = CS.cr_cfg_cs_point_add(robotHandle, pointcsnode);//增加
点坐标系数据
        }
        else
        {
            Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
        }
    }
    else
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},结果:

```



```

{result}");
        }
    }
    Thread.Sleep(500);
    if (result == CRresult.sucess)
    {
        double[] pose = new double[6];
        result = CS.cr_get_tcpActualPose(robotHandle, pose);
        if (result == CRresult.sucess)
        {
            double[] basePose = pose;
            double[] userPose = pointcsnode.point.toolPosition;
            double[] poseInUser = new double[6];
            result = CS.cr_compute_pose_base_to_user(basePose, 6, userPose, 6,
poseInUser, 6);//将点坐标系设置为用户坐标系
            if (result == CRresult.sucess)
            {
                Console.WriteLine($"行号: {GetLineNum().ToString()},将点坐标
系设置为用户坐标系成功");
            }
            else
            {
                Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
            }
        }
        else
        {
            Console.WriteLine($"行号: {GetLineNum().ToString()},结果:
{result}");
        }
    }
    else
    {
        Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
    }
}
else
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},结果: {result}");
}

result = CS.cr_destroy_robot(robotHandle);//机器人断开连接, 释放句柄
if (result != CRresult.sucess)

```

```
{
    Console.WriteLine($"行号: {GetLineNum().ToString()},机器人断开连接失败");
}
Console.WriteLine($"按下任意键退出。");
Console.ReadKey();
}
private static int GetLineNum()
{
    StackTrace st = new StackTrace(1, true);

    return st.GetFrame(0).GetFileLineNumber();
}
```

4.9 TCP 操作

简介

本示例介绍了与 TCP 操作相关的接口的使用方法，包括读取 TCP、修改 TCP、增加 TCP、删除 TCP、激活 TCP、读取当前激活的 TCP 索引。

示例

```
public static int robotHandle = -1;
static void Main(string[] args)
{
    CRresult result = CS.cr_create_robot(ref robotHandle, "192.168.6.6", 2323, "123");
    if (result == CRresult.sucess)
    {
        int index = -1;
        result = CS.cr_cfg_tcp_active_get(robotHandle, ref index); //读取当前激活的 TCP 索引
        if (result == CRresult.sucess)
        {
            TCPMsg tcpMsg = new TCPMsg
            {
                tcpOffset = new double[6],
                tcpName = new char[20],
            };
            result = CS.cr_cfg_tcp_get(robotHandle, index, ref tcpMsg); //需要先存在索引号为 0 的 TCP
            if (result == CRresult.sucess)
            {
                tcpMsg.tcpOffset = new double[6] { 3, 10, 9, 90, 80, 70 };
                result = CS.cr_cfg_tcp_set(robotHandle, index, tcpMsg); //编辑索引号为 0 的 TCP 数据
                if (result == CRresult.sucess)
```

```

        {
            char[] str = ("TCP").ToCharArray();
            TCPMsg tcpMsg1 = new TCPMsg
            {
                tcpOffset = new double[6] { 5, 10, 15, 10, 20, 30 },
                tcpName = new char[20]
            };
            for (int i = 0; i < str.Length; i++)
            {
                tcpMsg1.tcpName[i] = str[i];
            }
            result = CS.cr_cfg_tcp_add(robotHandle, tcpMsg1); //增加 TCP
            if (result == CRresult.sucess)
            {
                index = 1;
                result = CS.cr_cfg_tcp_active_set(robotHandle, index); //激活索引号为 1 的 TCP

                if (result == CRresult.sucess)
                {
                    result = CS.cr_cfg_tcp_active_get(robotHandle, ref index);
                    if (result == CRresult.sucess)
                    {
                        Console.WriteLine($"行号： {GetLineNum().ToString()},
                        当前激活的 TCP 为： {index}");

                        index = 0;
                        result = CS.cr_cfg_tcp_delete(robotHandle, index); //删除索引号为 0 的 TCP

                        if (result == CRresult.sucess)
                        {
                            Console.WriteLine($"行号：
                            {GetLineNum().ToString()},已成功删除索引号为{index}的 TCP");
                        }
                        else
                        {
                            Console.WriteLine($"行号：
                            {GetLineNum().ToString()},结果： {result}");
                        }
                    }
                    else
                    {
                        Console.WriteLine($"行号： {GetLineNum().ToString()},
                        结果： {result}");
                    }
                }
            }
        }
    }

```



```
return st.GetFrame(0).GetFileLineNumber();  
}
```

附录 A 速度和加速度范围

各机型对应速度范围

机型	关节速度范围限制(°/s)	TCP 速度范围限制(mm/s)
C3	1-180	0.01-1000
C4/C6/C6-DK	1-180	0.01-2000
G3	1-360	0.01-2000
G3a	1-270	0.01-1500
G4/G6/G7	1-360	0.01-3000
G4a/G6a	1-270	0.01-2000
G9/G10/G12	1-240	0.01-3000
G9a/G12a	1-180	0.01-2000
G18	1-240	0.01-2000
G18a	1-180	0.01-1500
G20	1-270	0.01-4000
G30	1-270	0.01-3000
X6	1-360	0.01-3000
X9/X12/X18	1-240	0.01-3000

各机型对应加速度范围

关节加速度限值统一为 0.01~2500 °/s²;

TCP 加速度限值统一为 0.01~20000 mm/s²。

各机型对应点动速度和加速度范围

关节点动速度限值统一为 $0.1\sim7.5\text{ }^{\circ}/\text{s}$;

关节点动加速度限值统一为 $0.01\sim45\text{ }^{\circ}/\text{s}^2$ 。

TCP 位置点动速度限值统一为 $0.1\sim60\text{ mm/s}$ ，姿态点动速度限值统一为 $0.1\sim7.5\text{ }^{\circ}/\text{s}$;

TCP 位置点动加速度限值统一为 $0.01\sim360\text{ mm/s}^2$ ，姿态点动加速度限值统一为 $0.01\sim45\text{ }^{\circ}/\text{s}^2$ 。